Kauno technologijos universitetas
Informatikos fakultetas

Objektinis programavimas 2 (P175B123)

Laboratorinių darbų ataskaita

Adomas Danilevičius IFA-3/1

Studentas

Doc. Giedrius Ziberkas

Dėstytojas

Kaunas 2025

TURINYS

1. Rekursija (L1) 4

1.1. Darbo užduotis 4

1.2. Grafinės naudotojo sąsajos schema 4

1.3. Sąsajoje panaudotų komponentų keičiamos savybės 4

1.4. Klasių diagrama 5

1.5. Programos naudotojo vadovas 6

1.6. Programos tekstas 6

1.7. Pradiniai duomenys ir rezultatai 11

1.8. Dėstytojo pastabos 12

2. Dinaminis atminties valdymas (L2) 13

2.1. Darbo užduotis 13

2.2. Grafinės naudotojo sąsajos schema 13

2.3. Sąsajoje panaudotų komponentų keičiamos savybės 13

2.4. Klasių diagrama 14

2.5. Programos naudotojo vadovas 14

2.6. Programos tekstas 14

2.7. Pradiniai duomenys ir rezultatai 38

2.8. Dėstytojo pastabos 41

3. Bendrinės klasės ir testavimas (L3) 42

3.1. Darbo užduotis 42

3.2. Grafinės naudotojo sąsajos schema 42

3.3. Sąsajoje panaudotų komponentų keičiamos savybės 42

3.4. Klasių diagrama 42

3.5. Programos naudotojo vadovas 42

3.6. Programos tekstas 42

3.7. Pradiniai duomenys ir rezultatai 42

3.8. Dėstytojo pastabos 43
4. Polimorfizmas ir išimčių valdymas (L4) 44
4.1. Darbo užduotis 44
4.2. Grafinės naudotojo sąsajos schema 44
4.3. Sąsajoje panaudotų komponentų keičiamos savybės 44
4.4. Klasių diagrama 44
4.5. Programos naudotojo vadovas 44
4.6. Programos tekstas 44
4.7. Pradiniai duomenys ir rezultatai 44
4.8. Dėstytojo pastabos 45
5. Deklaratyvusis programavimas (L5) 46
5.1. Darbo užduotis 46
5.2. Grafinės naudotojo sąsajos schema 46
5.3. Sąsajoje panaudotų komponentų keičiamos savybės 46
5.4. Klasių diagrama 46
5.5. Programos naudotojo vadovas 46
5.6. Programos tekstas 46
5.7. Pradiniai duomenys ir rezultatai 46
5.8. Dėstytojo pastabos 47

1. Rekursija (L1)

1.1. Darbo užduotis

Turime kvadrato formos languoto popieriaus lapą. Į lapo centre esantį langelių linijų susikirtimą statoma skriestuvo kojelė. Brėžiamas apskritimas, siekiantis lapo kraštus. Sunumeruokite, pradedant nuo 1, languoto popieriaus lapo langelius, pilnai patenkančius į apskritimo vidų, o kitiems langeliams priskirkite po 0. Numeravimo tvarka nėra svarbi. Atvaizduokite languoto popieriaus lapą matrica. Spindulio r ($r \in N$) dydis nurodomas langelių kiekiu.

Duomenys. r įvedamas klaviatūra ($1 \leq r \leq 16$).

Rezultatai. Spausdinkite gautą matricą.

1.2. Grafinės naudotojo sąsajos schema

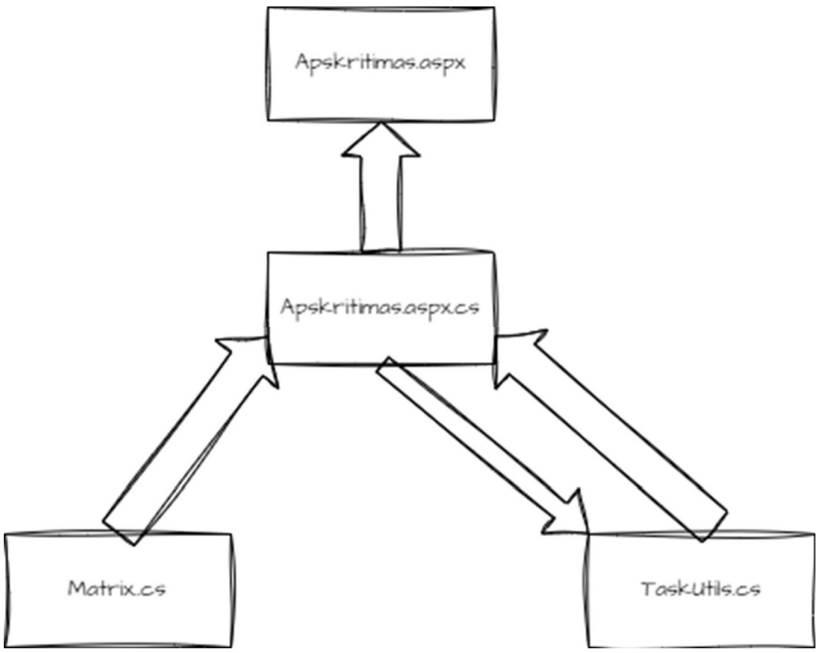
Pav. 1 Grafinės naudotojo sąsajos schema

1.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė
Table1	BorderColor	Black
Table1	BorderStyle	Solid
Table1	BorderWidth	1px
Table1	GridLines	Both
ValidationSummary1	DisplayMode	SingleParagapth
ValidationSummary1	ForeColor	Red
Button1	Text	„Vykdyti“
RangeValidator1	ErrorMessage	„Apskritimo spindulys privalo būti intervale [1;16]“
RangeValidator1	ForeColor	Red
RangeValidator1	ControlToValidate	TextBox1
RangeValidator1	MaximumValue	16
RangeValidator1	MinimumValue	1
RangeValidator1	Type	Integer
TextBox1	Width	24px

TextBox1	Text	8

1.4. Klasių diagrama



Pav. 2 Klasės diagrama

Matrix
-Matrix [,] : integer
+ (In Value : integer) : Matrix +Add(In Value : integer, integer, integer) +Get(In Value : integer, integer) : integer {query} +GetLength(In Value : integer, integer) : integer {query} -AutoFill()

TaskUtils
+CalculateCenter(In Value : integer) : integer {query} +FillOut(In Value : Matrix, integer, integer, [,]boolean, integer, integer, integer, integer) : Matrix

1.5. Programos naudotojo vadovas

Paleidus programą lange įrašyta numatyta reikšmė 8, tačiau ją galima keisti nuo 1 iki 16. Įrašius tinkamą reikšmę, lentelėje bus matomas apskritimas iš skaičių. Visi langeliai, kurie nėra užpildyti 0, patenka į apskritimo plotą.

1.6. Programos tekstas

TaskUtils.cs

```
//Adomas Dnailevičius IFA-3/1
using System;
using System.Web;
using System.Web.UI.WebControls;

namespace Lab1
{
    /// <summary>
    /// Backend class for Apskritimas.aspx form.
    /// </summary>

    public partial class Apskritimas : System.Web.UI.Page
    {

        /// <summary>
        /// Page load method
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>

        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                HttpContext.Current.Session["Matrix"] = new Classes.Matrix(8);
                GetMatrix();
            }
        }

        /// <summary>
        /// Method for actions after Button1 click.
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>

        protected void Button1_Click(object sender, EventArgs e)
        {
            HttpContext.Current.Session["Matrix"] = new
Classes.Matrix(int.Parse(TextBox1.Text));
            FillTable();
        }

        /// <summary>
        /// Method for filling Table1 in Apskritimas.aspx from matrix.
        /// </summary>
```

```
        private void FillTable()
        {
            Classes.Matrix matrix = GetMatrix();
            Table1.Rows.Clear();
            int tableSize = int.Parse(TextBox1.Text) * 50;
            Table1.Height = new Unit(tableSize, UnitType.Pixel);
            Table1.Width = new Unit(tableSize, UnitType.Pixel);
            Table1.Style["table-layout"] = "fixed";
            Table1.Style["border-collapse"] = "collapse";
            Unit CenterX = Classes.TaskUtils.CalculateCenter(Table1.Width);
            Unit CenterY = Classes.TaskUtils.CalculateCenter(Table1.Height);
            int rows = matrix.GetLength(0);
            int cols = matrix.GetLength(1);
            int[,] visited = new int[rows, cols];
            int counter = 0;

            HttpContext.Current.Session["Matrix"] =
Classes.TaskUtils.FillOut(matrix, CenterX, CenterY, visited, 0, 0, CenterX, ref
counter);

            for (int i = 0; i < rows; i++)
            {
                TableRow row = new TableRow();
                for (int j = 0; j < cols; j++)
                {
                    TableCell cell = new TableCell();
                    cell.HorizontalAlign = HorizontalAlign.Center;
                    cell.Text = matrix.Get(i, j).ToString();
                    cell.Attributes["style"] = "width:25px; height:25px;
padding:0; margin:0; border:1px solid black; box-sizing:border-box;
overflow:hidden; text-align:center;";

                    row.Cells.Add(cell);
                }
                Table1.Rows.Add(row);
            }
        }

        /// <summary>
        /// Gets Matrix saved in session.
        /// </summary>
        /// <returns>Matrix that is saved in session.</returns>

        private Classes.Matrix GetMatrix()
        {
            return (Classes.Matrix)HttpContext.Current.Session["Matrix"];
        }



    }
}
```

Matrix.cs

```
//Adomas Danilevičius IFA-3/1
namespace Lab1.Classes
{
    /// <summary>
    /// Class that contains two dimentional array.
    /// </summary>

    public class Matrix
    {
```

```
        private int[,] matrix; //Private two dimentional array.

        /// <summary>
        /// Constructor for two dimentional array.
        /// </summary>
        /// <param name="s">Size to set of square two dimentional array.</param>

        public Matrix(int s)
        {
            matrix = new int[s * 2, s * 2];
            AutoFill();
        }

        /// <summary>
        /// Adds or changes the number in specified place.
        /// </summary>
        /// <param name="x">X coordinate.</param>
        /// <param name="y">Y coordinate</param>
        /// <param name="number"></param>

        public void Add(int x, int y, int number)
        {
            matrix[x, y] = number;
        }

        /// <summary>
        /// Gets specific element from array.
        /// </summary>
        /// <param name="x">X coordinate.</param>
        /// <param name="y">Y coordinate</param>
        /// <returns></returns>

        public int Get(int x, int y)
        {
            return matrix[x, y];
        }

        /// <summary>
        /// Gets the size of an array.
        /// </summary>
        /// <param name="dim">Dimention specification (0 is length and 1 is
width).</param>
        /// <returns>Dimention or array.</returns>

        public int GetLength(int dim)
        {
            return matrix.GetLength(dim);
        }

        /// <summary>
        /// Fills up two dimentional array with 0s for easier checking.
        /// </summary>

        private void AutoFill()
        {
            for (int i = 0; i < matrix.GetLength(0); i++)
            {
                for (int j = 0; j < matrix.GetLength(1); j++)
                {
                    matrix[i, j] = 0;
                }
            }

        }
    }
```

```
}
```

Apskritimas.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Apskritimas.aspx.cs"
Inherits="Lab1.Apskritimas" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <strong>Apskritimas<br />
            </strong>
             <asp:ValidationSummary ID="ValidationSummary1" runat="server"
DisplayMode="SingleParagraph" ForeColor="Red" />
            <br />
            <asp:Table ID="Table1" runat="server" BorderColor="Black"
BorderStyle="Solid" BorderWidth="1px" GridLines="Both">
            </asp:Table>
            <br />
            <br />
            Nustatykite apskritimo spindulį:
            <asp:TextBox ID="TextBox1" runat="server"
Width="24px">8</asp:TextBox>
            <asp:RangeValidator ID="RangeValidator1" runat="server"
ControlToValidate="TextBox1" ErrorMessage="Apskritimo spindulys privalo būti
intervale [1;16]" ForeColor="Red" MaximumValue="16" MinimumValue="1"
Type="Integer">*</asp:RangeValidator>
            <br />
            <br />
            <br />
    
            <asp:Button ID="Button1" runat="server" OnClick="Button1_Click"
Text="Vykdyti" />
 
            <br />

        </div>
    </form>
</body>
</html>
```

Apskritimas.aspx.cs

```
//Adomas Danilevičius IFA-3/1
using System;
using System.Web;
using System.Web.UI.WebControls;

namespace Lab1
{
    /// <summary>
    /// Backend class for Apskritimas.aspx form.
    /// </summary>

    public partial class Apskritimas : System.Web.UI.Page
    {
```

```
        /// <summary>
        /// Page load method
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>

        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                HttpContext.Current.Session["Matrix"] = new Classes.Matrix(8);
                GetMatrix();
            }
        }

        /// <summary>
        /// Method for actions after Button1 click.
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>

        protected void Button1_Click(object sender, EventArgs e)
        {
            HttpContext.Current.Session["Matrix"] = new
Classes.Matrix(int.Parse(TextBox1.Text));
            FillTable();
        }

        /// <summary>
        /// Method for filling Table1 in Apskritimas.aspx from matrix.
        /// </summary>

        private void FillTable()
        {
            Classes.Matrix matrix = GetMatrix();
            Table1.Rows.Clear();
            int tableSize = int.Parse(TextBox1.Text) * 50;
            Table1.Height = new Unit(tableSize, UnitType.Pixel);
            Table1.Width = new Unit(tableSize, UnitType.Pixel);
            Table1.Style["table-layout"] = "fixed";
            Table1.Style["border-collapse"] = "collapse";
            Unit CenterX = Classes.TaskUtils.CalculateCenter(Table1.Width);
            Unit CenterY = Classes.TaskUtils.CalculateCenter(Table1.Height);
            int rows = matrix.GetLength(0);
            int cols = matrix.GetLength(1);
            int[,] visited = new int[rows, cols];
            int counter = 0;

            HttpContext.Current.Session["Matrix"] =
Classes.TaskUtils.FillOut(matrix, CenterX, CenterY, visited, 0, 0, CenterX, ref
counter);

            for (int i = 0; i < rows; i++)
            {
                TableRow row = new TableRow();
                for (int j = 0; j < cols; j++)
                {
                    TableCell cell = new TableCell();
                    cell.HorizontalAlign = HorizontalAlign.Center;
                    cell.Text = matrix.Get(i, j).ToString();
                    cell.Attributes["style"] = "width:25px; height:25px;
padding:0; margin:0; border:1px solid black; box-sizing:border-box;
overflow:hidden; text-align:center;";
```

```
                row.Cells.Add(cell);
            }
            Table1.Rows.Add(row);
        }
    }

    /// <summary>
    /// Gets Matrix saved in session.
    /// </summary>
    /// <returns>Matrix that is saved in session.</returns>

    private Classes.Matrix GetMatrix()
    {
        return (Classes.Matrix)HttpContext.Current.Session["Matrix"];
    }


}
                                                    }
```

1.7. Pradiniai duomenys ir rezultatai

Duomenys nr. 1

Duomenys ir rezultatai kai r=5.

Apskritimas

0	0	0	0	0	0	0	0	0	0
0	0	55	56	57	58	59	60	0	0
0	47	48	49	50	51	52	53	54	0
0	39	40	41	42	43	44	45	46	0
0	31	32	33	34	35	36	37	38	0
0	23	24	25	26	27	28	29	30	0
0	15	16	17	18	19	20	21	22	0
0	7	8	9	10	11	12	13	14	0
0	0	1	2	3	4	5	6	0	0
0	0	0	0	0	0	0	0	0	0

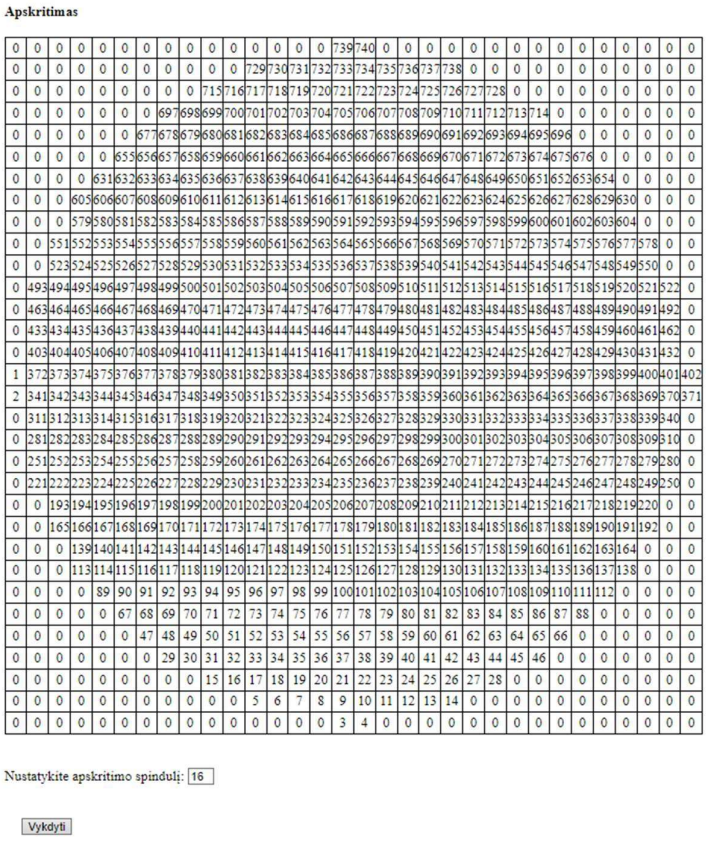
Nustatykite apskritimo spindulį: 5

Vykdyti

Pav. 3 Rezultatai

Duomenys nr. 2

Duomenys ir rezultatai kai r=16.



Apskritimas

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	739	740	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	729	730	731	732	733	734	735	736	737	738	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	715	716	717	718	719	720	721	722	723	724	725	726	727	728	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	697	698	699	700	701	702	703	704	705	706	707	708	709	710	711	712	713	714	0	0	0	0	0	0
0	0	0	0	0	0	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	0	0	0	0	0	0
0	0	0	0	0	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	0	0	0	0	0
0	0	0	0	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	0	0	0	0
0	0	0	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	0	0	0
0	0	0	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600	601	602	603	604	0	0	0
0	0	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	0	0
0	0	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	0	0
0	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	0
0	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	0
0	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	0
0	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	0
1	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402
2	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371
0	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	0
0	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	0
0	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	0
0	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	0
0	0	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	0	0
0	0	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	0	0
0	0	0	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	0	0	0
0	0	0	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	0	0	0
0	0	0	0	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	0	0	0	0
0	0	0	0	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	0	0	0	0	0	0
0	0	0	0	0	0	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	0	0	0	0	0	0
0	0	0	0	0	0	0	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	15	16	17	18	19	20	21	22	23	24	25	26	27	28	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	5	6	7	8	9	10	11	12	13	14	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Nustatykite apskritimo spindulį: 16

Vykdyti

Pav. 4 Rezultatai

1.8. Dėstytojo pastabos

- Nėra partial klasės Apskritimas.aspx.cs klasei;

2. Dinaminis atminties valdymas (L2)

2.1. Darbo užduotis

LD_6. Premijos.

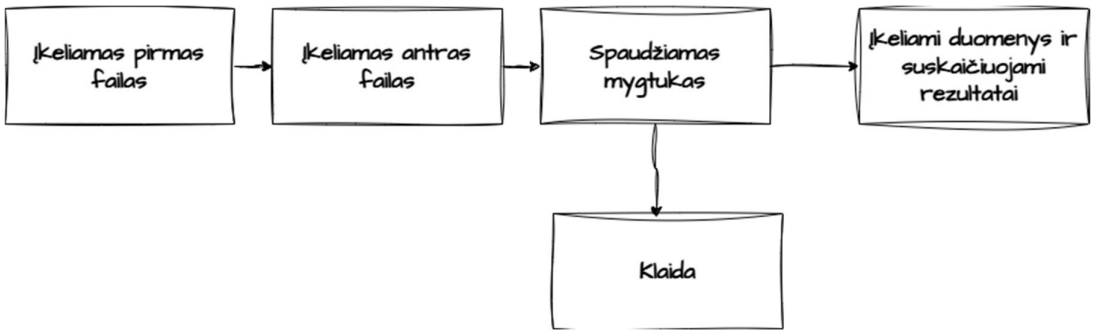
Moksliniai darbuotojai atliko darbus 4 skirtingose temose. Visos temos gavo premijas. Remiantis darbuotojų indėliais, suskaičiuokite kiekvienam darbuotojui priklausančios premijos dydį pagal kiekvieną temą atskirai ir bendrą premijų sumą. Darbuotojo indėlis rodo, kokia premijos dalis jam priklauso. Indėlis gali būti išreikštas bet kokiu skaičiumi. Bet tas pats matas naudojamas visiems darbuotojams. Neišdalinkite pinigų daugiau, negu turite, ir turite išdalinti visus pinigus. Sudarykite sąrašą darbuotojų (pavardė, vardas, darbuotojo indėliai, darbuotojo premijų dydžiai pagal temas, bendra premijų sumą), kurie uždirbo mažiau už vidurkį. Duomenys:

- Tekstiniame faile U6a.txt duota informacija apie darbuotojus: darbuotojų asmens kodai, pavardės, vardai, banko pavadinimas, sąskaitos numeris.

- Tekstiniame faile U6b.txt duota informacija apie indėlius į darbą: pirmoje eilutėje - premijų dydžiai, tolesnėse eilutėse - darbuotojų asmens kodai, darbuotojų indėliai, kurie išreikšti naudingumo koeficientu, į eilinę temą.

Informacija apie darbuotoją užima vieną eilutę. Spausdinamas sąrašas turi būti surikiuotas pagal darbuotojų pavardes, vardus abėcėlės tvarka ir bendrą premijų sumą didėjimo tvarka. Sudarykite nurodytos temos (įvedama klaviatūra) darbuotojų uždarbių sąrašą (pavardė, vardas, darbuotojo indėliai, darbuotojo premijų dydžiai pagal temas, bendra premijų sumą).

2.2. Grafinės naudotojo sąsajos schema

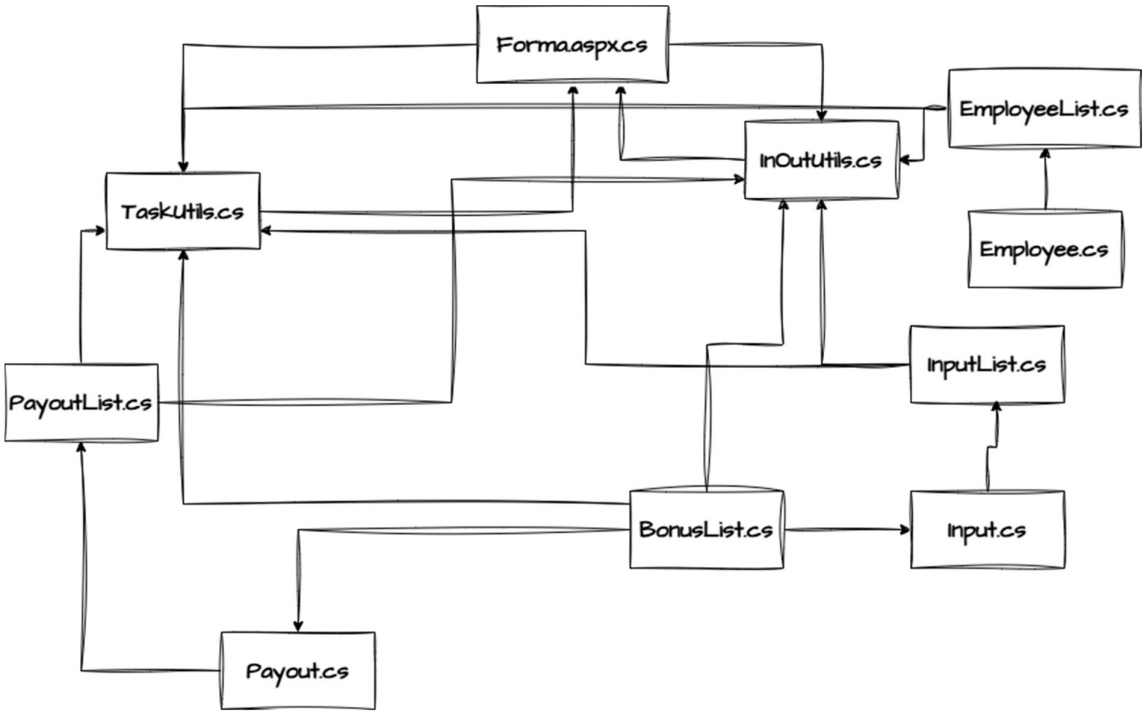


Pav. 5 GUI diagrama

2.3. Sąsajoje panaudotų komponentų keičiamos savybės

Visos savybės keičiamos styles.css faile.

2.4. Klasių diagrama



Pav. 6 Klasių diagrama

2.5. Programos naudotojo vadovas

Naudotojo sąsajoje reikia įkelti 2 failus. Viename faile turi būti duomenys apie darbuotojus : „Asmens kodas;Pavardė;Vardas;Banko pavadinimas;Sąskaitos numeris“. Kitame faile turi būti įkelti duomenys apie premijas. Pirmoje eilutėje: „Premija už tema1; Premija už tema2; Premija už tema3; Premija už tema4“. Likusiose duomenys apie indėlį: „Asmens kodas; Indėlis1; Indėlis2; Indėlis3; Indėlis4“. Įkėlus tokia formato failus ir paspaudus mygtuką programa suskaičiuoja išmokas.

2.6. Programos tekstas

Input.cs

```
using Lab2_v2.Bonuses;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab2_v2.Einput
```

```
{
    /// <summary>
    /// Input object class.
    /// </summary>
       public class Input
       {
             public int Id { get; set; }

         public BonusList Bonuses { get; set; }

         /// <summary>
         /// Constructor for object.
         /// </summary>
         /// <param name="id">ID of a person.</param>
         /// <param name="bonuses">Bonus linked list for storing bonus amounts.</param>
         public Input(int id, BonusList bonuses)
         {
             this.Id = id;
             this.Bonuses = bonuses;
         }

    }
                                                  }
```

Employee.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab2_v2.Employees
{
    /// <summary>
    /// Employee object class.
    ///  </summary>
    public class Employee
       {
             public int Id { get; set; }
         public string LastName { get; set; }
         public string Name { get; set; }
         public string BankName { get; set; }
         public string AccountNumber { get; set; }

         /// <summary>
         /// Constructor for object.
         /// </summary>
         /// <param name="id">Id of a person.</param>
         /// <param name="lastname">Last name of a person.</param>
         /// <param name="name">Name of a person.</param>
         /// <param name="bankname">Bank name of a person.</param>
         /// <param name="accountnumber">Account number of person.</param>

         public Employee(int id, string lastname, string name, string bankname, string
accountnumber)
         {
             this.Id = id;
             this.LastName = lastname;
             this.Name = name;
             this.BankName = bankname;
             this.AccountNumber = accountnumber;
         }

         /// <summary>
```

```
        /// ToString override for formatting.
        /// </summary>
        /// <returns>Formatted string.</returns>
        public override string ToString()
        {
            return string.Format($"| {Id,20} | {LastName,-15} | {Name,-15} | {BankName,-
15} | {AccountNumber,-20} | ");
        }


    }
}
```

Payout.cs

```
using Lab2_v2.Bonuses;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab2_v2.Payouts
{
	/// <summary>
	/// Class for Payout object.
	/// </summary>
	public class Payout
	{
		public string LastName { get; set; }
		public string Name { get; set; }
		public BonusList BonusCoef { get; set; }
		public BonusList BonusAmounts { get; set; }
		public double Sum { get; set; }

		/// <summary>
		/// Constructor for object.
		/// </summary>
		/// <param name="lastname">Last name of a person.</param>
		/// <param name="name">Name of a person.</param>
		/// <param name="bonusCoef">Linked list for all coeficients.</param>
		/// <param name="bonusAmounts">Linked list for bonus amounts.</param>
		public Payout(string lastname, string name, BonusList bonusCoef, BonusList
bonusAmounts)
		{
			this.LastName = lastname;
			this.Name = name;
			this.BonusCoef = bonusCoef;
			this.BonusAmounts = bonusAmounts;
			this.Sum = 0;

		}

		/// <summary>
		/// Compares node data.
		/// </summary>
		/// <param name="other">Other node data.</param>
		/// <returns>True if they are the same false otherwise.</returns>
		public bool Compare(Payout other)
		{
			if (other.Name == Name && other.LastName == LastName)
			{
				return true;
			}
			return false;
```

```
            }
            /// <summary>
            /// Calculates the sum of bonuses.
            /// </summary>
            public void SumCalc()
            {
                  double sum = 0;
                  for (BonusAmounts.Start(); BonusAmounts.Exists(); BonusAmounts.Next())
                  {
                        sum += BonusAmounts.Get();
                  }
                  this.Sum = sum;
            }
            /// <summary>
            /// Compares two node data.
            /// </summary>
            /// <param name="other">Other node data.</param>
            /// <returns></returns>
        public int CompareTo(Payout other)
        {
            int lastNameComp = string.Compare(this.LastName, other.LastName,
StringComparison.OrdinalIgnoreCase);
                  if (lastNameComp != 0)
                  {
               return lastNameComp;
            }

            int nameComp = string.Compare(this.Name, other.Name,
StringComparison.OrdinalIgnoreCase);
                  if (nameComp != 0)
                  {
                        return nameComp;
                  }

            return this.Sum.CompareTo(other.Sum);
        }

    }
```

}

BonusList.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using  System.Web;
using System.Xml.Linq;


namespace Lab2_v2.Bonuses
{
    /// <summary>
    /// Linked list for bonuses.
    /// </summary>
       public sealed class BonusList
       {
         /// <summary>
         /// Node class for linked list elements.
         /// </summary>
         public class BonusNode
         {
             public double Data { get; set; }
             public BonusNode Link { get; set; }
```

```
    public BonusNode(double bonus, BonusNode next)
    {
        this.Data = bonus;
        this.Link = next;
    }
}

BonusNode Head;
BonusNode Tail;
BonusNode Current;
int Count = 0;


/// <summary>
/// Linked list constructor.
/// </summary>
public BonusList()
{
    this.Tail = null;
    this.Head = null;
    Current = null;
}

/// <summary>
/// Adds nodes to the end of a linked list.
/// </summary>
/// <param name="value">Value to add.</param>
public void Add(double value)
{
    BonusNode newNode = new BonusNode(value, null);

    if (Head == null)
    {
        Head = newNode;
        return;
    }

    BonusNode current = Head;
    while (current.Link != null)
    {
        current = current.Link;
    }

    current.Link = newNode;
    Count++;
}

/// <summary>
/// Sets pointer to head.
/// </summary>

public void Start()
{
    Current = Head;
}

/// <summary>
/// Moves pointer by 1 node.
/// </summary>

public void Next()
{
    Current = Current.Link;
```

```
        }

        /// <summary>
        /// Checks if the node exists.
        /// </summary>
        /// <returns>True if data exits false if not.</returns>

        public bool Exists()
        {
            return Current != null;
        }


        /// <summary>
        /// Gets data from current node.
        /// </summary>
        /// <returns></returns>

        public double Get()
        {
            return Current.Data;
        }
    }
                                                }
```

InputList.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;


namespace Lab2_v2.Einput
{
    /// <summary>
    /// Linked list for all worker contributions.
    /// </summary>
    public sealed class InputList
      {
        /// <summary>
        /// Node class for linked list elements.
        /// </summary>
        public class InputNode
             {
            public Input Data { get; set; }
            public InputNode Link { get; set; }
            public InputNode(Input input, InputNode next)
            {
                this.Data = input;
                this.Link = next;
            }
        }

        InputNode Head;
        InputNode Tail;
        InputNode Current;

        /// <summary>
        /// Linked list constructor.
        /// </summary>

        public InputList()
```

```
{
    Head = null;
    Tail = null;
    Current = null;
}

/// <summary>
/// Adds nodes to the end of a linked list.
/// </summary>
/// <param name="value">Value to add.</param>
public void Add(Input value)
{
    InputNode newNode = new InputNode(value, null);

    if (Head == null)
    {
        Head = newNode;
        return;
    }

    InputNode current = Head;
    while (current.Link != null)
    {
        current = current.Link;
    }

    current.Link = newNode;
}

/// <summary>
/// Sets pointer to head.
/// </summary>

public void Start()
{
    Current = Head;
}

/// <summary>
/// Moves pointer by 1 node.
/// </summary>

public void Next()
{
    Current = Current.Link;
}

/// <summary>
/// Checks if the node exists.
/// </summary>
/// <returns>True if data exits false if not.</returns>

public bool Exists()
{
    return Current != null;
}

/// <summary>
/// Gets data from current node.
/// </summary>
/// <returns></returns>

public Input Get()
```

```
        {
            return Current.Data;
        }


    }
```

}

EmployeeList.cs

```
using Lab2_v2.Einput;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;



namespace Lab2_v2.Employees
{
    /// <summary>
    /// Linked list for all employees.
    /// </summary>
       public sealed class EmployeeList
       {
         /// <summary>
         /// Node class for linked list elements.
         /// </summary>
         public class EmployeeNode
         {
             public Employee Data { get; set; }
             public EmployeeNode Link { get; set; }
             public EmployeeNode(Employee employee, EmployeeNode next)
             {
                 this.Data = employee;
                 this.Link = next;
             }
         }

         EmployeeNode Head;
         EmployeeNode Tail;
         EmployeeNode Current;

         /// <summary>
         /// Linked list constructor.
         /// </summary>

         public EmployeeList()
         {
             Head = null;
             Tail = null;
             Current = null;
         }

         /// <summary>
         /// Adds nodes to the end of a linked list.
         /// </summary>
         /// <param name="value">Value to add.</param>

         public void Add(Employee value)
         {
             EmployeeNode newNode = new EmployeeNode(value, null);
```

```
            if (Head == null)
            {
                Head = newNode;
                return;
            }

            EmployeeNode current = Head;
            while (current.Link != null)
            {
                current = current.Link;
            }

            current.Link = newNode;
        }
        /// <summary>
        /// Sets pointer to head.
        /// </summary>
        public void Start()
        {
            Current = Head;
        }
        /// <summary>
        /// Moves pointer by 1 node.
        /// </summary>
        public void Next()
        {
            Current = Current.Link;
        }
        /// <summary>
        /// Checks if the node exists.
        /// </summary>
        /// <returns>True if data exits false if not.</returns>
        public bool Exists()
        {
            return Current != null;
        }
        /// <summary>
        /// Gets data from current node.
        /// </summary>
        /// <returns></returns>
        public Employee Get()
        {
            return Current.Data;
        }

    }
                                                }
```

PayoutList.cs

```
using Lab2_v2.Employees;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using static Lab2_v2.Employees.EmployeeList;


namespace Lab2_v2.Payouts
{/// <summary>
 /// Linked list for all payouts.
 /// </summary>
    public sealed class PayoutList
      {
```

```
/// <summary>
/// Node class for linked list elements.
/// </summary>
public class PayoutNode
     {
            public Payout Data { get; set; }
            public PayoutNode Link { get; set; }

            public PayoutNode(Payout data, PayoutNode next)
            {
                  this.Data = data;
                  this.Link = next;
            }
     }
PayoutNode Head;
PayoutNode Tail;
PayoutNode Current;

/// <summary>
/// Linked list constructor.
/// </summary>
public PayoutList()
{
    Head = null;
    Tail = null;
    Current = null;
}
/// <summary>
/// Adds nodes to the end of a linked list.
/// </summary>
/// <param name="value">Value to add.</param>
public void Add(Payout value)
{
    PayoutNode newNode = new PayoutNode(value, null);

    if (Head == null)
    {
        Head = newNode;
        return;
    }

    PayoutNode current = Head;
    while (current.Link != null)
    {
        current = current.Link;
    }

    current.Link = newNode;
}
/// <summary>
/// Sets pointer to head.
/// </summary>
public void Start()
{
    Current = Head;
}
/// <summary>
/// Moves pointer by 1 node.
/// </summary>
public void Next()
{
    Current = Current.Link;
}
```

```
/// <summary>
/// Checks if the node exists.
/// </summary>
/// <returns>True if data exits false if not.</returns>
public bool Exists()
{
    return Current != null;
}
/// <summary>
/// Gets data from current node.
/// </summary>
/// <returns></returns>
public Payout Get()
{
    return Current.Data;
}
/// <summary>
/// Checks if list contains specific data.
/// </summary>
/// <param name="payout">Data to be compared to.</param>
/// <returns>True if found else false.</returns>
public bool Contains(Payout payout)
{
    for (Start(); Exists(); Next())
    {
        if (Get().Compare(payout))
        {
            return true;
        }
    }
    return false;
}
/// <summary>
/// Selection sort for data of nodes.
/// </summary>
public void Sort()
{
    for (Start(); Exists(); Next())
    {
        PayoutNode outer = Current;
        PayoutNode min = Current;
        PayoutNode inner = outer.Link;
        while (inner != null)
        {
            if (inner.Data.CompareTo(min.Data) < 0)
            {
                min = inner;
            }

            inner = inner.Link;
        }
        if (min != outer)
        {
            Payout temp = outer.Data;
            outer.Data = min.Data;
            min.Data = temp;
        }
    }
}
```

```
    }
                                                    }
```

TaskUtils.cs

```
using Lab2_v2.Bonuses;
using Lab2_v2.Einput;
using Lab2_v2.Payouts;
using Lab2_v2.Employees;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.SessionState;

namespace Lab2_v2.Utils
{
       public static class TaskUtils
       {
         /// <summary>
         /// Creates a list of new payout objects.
         /// </summary>
         /// <param name="inputs">Input coeficients.</param>
         /// <param name="employees">Employees linked list.</param>
         /// <returns>Linked list of payout objects.</returns>
              private static PayoutList AddToList(InputList inputs, EmployeeList employees)
              {
                     PayoutList payoutList = new PayoutList();
                     for (employees.Start(); employees.Exists(); employees.Next())
                     {
                            Employee employee = employees.Get();
                            for (inputs.Start(); inputs.Exists(); inputs.Next())
                            {
                                   Input input = inputs.Get();
                                   if (input.Id == employee.Id)
                                   {
                                          Payout p = new Payout(employee.LastName,
employee.Name, input.Bonuses, new BonusList());
                                          if (!payoutList.Contains(p))
                                          {
                                    payoutList.Add(p);
                                }
                             }
                                   }

                            }
                            return payoutList;
              }

         /// <summary>
         /// Calculates bonus for each worker and each theme.
         /// </summary>
         /// <param name="inputs">Input coeficients.</param>
         /// <param name="employees">Employees linked list.</param>
         /// <param name="bonuses">Bonus linked list.</param>
         /// <returns>Linked list with payout objects.</returns>
         public static PayoutList CalculateBonus(InputList inputs, EmployeeList employees,
BonusList bonuses)
         {
             PayoutList payouts = AddToList(inputs, employees);
             payouts.Start();
             while (payouts.Exists())
```

```
            {
                Payout payout = payouts.Get();



                bonuses.Start();
                payout.BonusCoef.Start();
                int themeNumber = 0;

                while (payout.BonusCoef.Exists())
                {
                    double totalCoef = AllCoef(inputs, themeNumber);
                    double bonusAmount = 0;

                    if (totalCoef > 0)
                    {
                        bonusAmount = (payout.BonusCoef.Get() / totalCoef) *
bonuses.Get();
                    }

                    payout.BonusAmounts.Add(bonusAmount);

                    bonuses.Next();
                    payout.BonusCoef.Next();
                    themeNumber++;
                }
                payout.SumCalc();
                payouts.Next();
            }

            return payouts;
        }
        /// <summary>
        /// Calculates sum of coeficiens for each theme.
        /// </summary>
        /// <param name="inputs">Input coeficients.</param>
        /// <param name="themeNumber">Theme number.</param>
        /// <returns></returns>
        private static double AllCoef(InputList inputs, int themeNumber)
        {
            double coefs = 0;

            for (inputs.Start(); inputs.Exists(); inputs.Next())
            {
                var bonusList = inputs.Get().Bonuses;
                bonusList.Start();
                for (int i = 0; i < themeNumber; i++)
                {
                    if (!bonusList.Exists()) break;
                    bonusList.Next();
                }

                if (bonusList.Exists())
                {
                    coefs += bonusList.Get();
                }
            }

            return coefs;
        }
        /// <summary>
        /// Picks out objects that have sum smaller than average sum.
        /// </summary>
```

```
        /// <param name="payouts">Payout linked list.</param>
        /// <returns>Payout linked list with objects that have smaller sums than
average.</returns>
        public static PayoutList LessThanAvarage(PayoutList payouts)
        {
            double avg = CalculateAveragePayout(payouts);
            PayoutList lessthanaverage = new PayoutList();
            for (payouts.Start(); payouts.Exists(); payouts.Next())
            {
                Payout p = payouts.Get();
                if (p.Sum<avg)
                {
                    lessthanaverage.Add(p);
                }
            }
            return lessthanaverage;
        }
        /// <summary>
        /// Calculates average of a bonus.
        /// </summary>
        /// <param name="payouts">Whole payout linked list.</param>
        /// <returns>Avergae of bonus sums.</returns>
        private static double CalculateAveragePayout(PayoutList payouts)
        {
            double sum = 0;
            int count = 0;
            for (payouts.Start(); payouts.Exists(); payouts.Next())
            {
                Payout p = payouts.Get();
                for (p.BonusAmounts.Start(); p.BonusAmounts.Exists();
p.BonusAmounts.Next())
                {
                    sum += p.BonusAmounts.Get();

                }
                count++;
            }
            return sum/count;
        }

    }
}
```

InOutUtils.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.IO;
using System.Runtime.Remoting.Messaging;
using Lab2_v2;
using Lab2_v2.Einput;
using Lab2_v2.Bonuses;
using Lab2_v2.Employees;
using System.Web.UI.WebControls;
using Lab2_v2.Payouts;

namespace Lab2.Classes.Utils
{
     /// <summary>
     /// Class that handles all inputs and outputs.
     /// </summary>
```

```
    static class InOutUtils
    {
        /// <summary>
        /// Writes all calcualted data to file.
        /// </summary>
        /// <param name="fileName">Name of a file.</param>
        /// <param name="payouts">Linked list.</param>
        /// <param name="header">Header to write.</param>
        public static void WriteToFileCalculations(string fileName, PayoutList payouts,
string header)
        {
            using (StreamWriter writer = File.AppendText(fileName))
            {
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 279));
                writer.WriteLine($"| {"Pavardė",20} | {"Vardas",15} | {"Tema1",-15} |
{"Tema2",-15} |" +
                    $" {"Tema3",-15} | {"Tema4",-15} | {"Premija už tema1", -30} |
{"Premija už tema2",-30} |" +
                    $" {"Premija už tema3",-30} | {"Premija už tema4",-30} | {"Suma",-
30}");
                writer.WriteLine(new string('-', 279));
                for (payouts.Start(); payouts.Exists(); payouts.Next())
                {
                    string line = ($"| {payouts.Get().LastName,20} | {payouts.Get().Name,
15} |");
                    for (payouts.Get().BonusCoef.Start();
payouts.Get().BonusCoef.Exists(); payouts.Get().BonusCoef.Next())
                    {
                        line += ($" {payouts.Get().BonusCoef.Get().ToString(),-15} |");
                    }
                    for (payouts.Get().BonusAmounts.Start();
payouts.Get().BonusAmounts.Exists(); payouts.Get().BonusAmounts.Next())
                    {
                        line += ($" {payouts.Get().BonusAmounts.Get().ToString(),-30} |");
                    }
                    line += ($" {payouts.Get().Sum,-30} |");
                    writer.WriteLine(line);
                }
                writer.WriteLine(new string('-', 279));
            }
        }
        /// <summary>
        /// Writes all input data to file.
        /// </summary>
        /// <param name="fileName">Name of a file.</param>
        /// <param name="inputList">Linked list.</param>
        /// <param name="header">Header to write.</param>
        public static void WriteToFileInput(string fileName, InputList inputList, string
header)
        {
            using (StreamWriter writer = File.AppendText(fileName))
            {
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 96));
                writer.WriteLine($"| {"Asmens kodas",20} | {"Tema1",-15} | {"Tema2",-15} |
{"Tema3",-15} | {"Tema4",-15} | ");
                writer.WriteLine(new string('-', 96));
                for (inputList.Start(); inputList.Exists(); inputList.Next())
                {
                    string line = ($"| {inputList.Get().Id,20} |");
                    for (inputList.Get().Bonuses.Start();
inputList.Get().Bonuses.Exists(); inputList.Get().Bonuses.Next())
```

```
                    {
                        line += ($" {inputList.Get().Bonuses.Get(), -15} |");
                    }
                    writer.WriteLine(line);
                }
                writer.WriteLine(new string('-', 96));
            }
        }
        /// <summary>
        /// Writes all worker data to file.
        /// </summary>
        /// <param name="fileName">Name of a file.</param>
        /// <param name="employeeList">Linked list.</param>
        /// <param name="header">Header to write.</param>
        public static void WriteToFileWorkers(string fileName, EmployeeList employeeList,
string header)
        {
            using (StreamWriter writer = File.AppendText(fileName))
            {
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 100));
                writer.WriteLine($"| {"Asmens kodas", 20} | {"Pavardė", -15} | {"Vardas",
-15} | {"Bankas", -15} | {"Banko sąskaita", -20} | ");
                writer.WriteLine(new string('-', 100));
                for (employeeList.Start(); employeeList.Exists(); employeeList.Next())
                {
                    writer.WriteLine(employeeList.Get().ToString());
                }
                writer.WriteLine(new string('-', 100));
            }
        }
        /// <summary>
        /// Writes all bonus data to file.
        /// </summary>
        /// <param name="fileName">Name of a file.</param>
        /// <param name="bonuses">Linked list.</param>
        /// <param name="header">Header to write.</param>
        public static void WriteToFileBonus(string fileName, BonusList bonuses, string
header)
        {
            using (StreamWriter writer = File.AppendText(fileName))
            {
                int count = 1;
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 37));

                for (bonuses.Start(); bonuses.Exists(); bonuses.Next())
                {
                    double current = bonuses.Get();
                    writer.WriteLine($"| {"Tema" + count, -15} | {current.ToString(), -15}
|");

                    count++;
                }
                writer.WriteLine(new string('-', 37));
                writer.WriteLine();
            }
        }



        /// <summary>
        /// Reads all workers.
        /// </summary>
```

```
        /// <param name="fileContetnt">File content.</param>
        /// <returns>Worker linked list.</returns>

        public static EmployeeList ReadWorkers(Stream fileContetnt)
        {
            EmployeeList list = new EmployeeList();
            using (StreamReader reader = new StreamReader(fileContetnt))
            {
                string line;
                while ((line = reader.ReadLine()) != null)
                {
                    string[] Values = line.Split(';');
                    int ID = int.Parse(Values[0]);
                    string LastName = Values[1];
                    string Name = Values[2];
                    string BankName = Values[3];
                    string BankAccountNum = Values[4];
                    Employee w = new Employee(ID, LastName, Name, BankName,
BankAccountNum);
                    list.Add(w);
                }
            }

            return list;
        }

        /// <summary>
        /// Reads all contributions.
        /// </summary>
        /// <param name="FileContent">Content of a file.</param>
        /// <returns>ContList linked list.</returns>
        public static InputList ReadContributions(Stream FileContent)
        {
            InputList list = new InputList();
            using (StreamReader reader = new StreamReader(FileContent))
            {

                reader.ReadLine();
                string line;
                while ((line = reader.ReadLine()) != null)
                {
                    string[] Values = line.Split(';');
                    int ID = int.Parse(Values[0]);
                    BonusList contributionValuesList = new BonusList();
                    foreach (var amount in Values.Skip(1))
                    {
                        double value = Double.Parse(amount);
                        contributionValuesList.Add(value);
                    }
                    Input c = new Input(ID, contributionValuesList);
                    list.Add(c);
                }
            }
            return list;
        }
        /// <summary>
        /// Reads all bonuses from first line of a file.
        /// </summary>
        /// <param name="FileContent">File content.</param>
        /// <returns>Linked list of all bonuses.</returns>

        public static BonusList ReadBonusAmounts(Stream FileContent)
        {
```

```
BonusList list = new BonusList();
```

```
            using (StreamReader reader = new StreamReader(FileContent))
            {
                string bonuses = reader.ReadLine();
                string[] Values = bonuses.Split(';');
                foreach (var value in Values)
                {
                    double bonus = Double.Parse(value);
                    list.Add(bonus);
                }
            }
            return list;
        }
    }
```

}

Forma.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Forma.aspx.cs"
Inherits="Lab2.Forma" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
    <link rel="stylesheet" type="text/css" href="styles.css" />
</head>
<body>
    <form id="form1" runat="server">

        <h1>Premijos</h1>
            <div>
                <br />
                Įkelti darbuotojų sąrašą:<br />
                <asp:FileUpload ID="FileUpload1" runat="server" />
                <br />
                <br />
                Įkelti premijų sąrašą:<br />
                <asp:FileUpload ID="FileUpload2" runat="server" />
                <br />
                <br />
                <asp:Button ID="Upload1" runat="server" Text="Įkelti failus ir
susikaičiuoti" OnClick="Upload1_Click" />
                <br />
                <br />
                <asp:Table ID="PayoutsPerTheme" runat="server"></asp:Table>
                <br />
                <asp:Table ID="DataFile1" runat="server"></asp:Table>
                <br />
                <asp:Table ID="DataFile2" runat="server"></asp:Table>
                <br />
                Premijų skaičiavimo rezultatai:<br />
                <asp:Table ID="Results" runat="server">
                </asp:Table>
                <br />
                Darbuotojai uždirbę mažiau nei premijų vidurkis:<br />
                <asp:Table ID="LessAvg" runat="server">
                </asp:Table>
                Išrinkimas pagal temą:<br />
                <br />
                Pasirinkite temą: <br />
                <asp:TextBox ID="ThemeSelection" runat="server"></asp:TextBox>
                <br />
                <asp:Table ID="PickOut" runat="server">
```

```
                </asp:Table>
                <br />
                <br />
                <asp:Button ID="Pick" runat="server" OnClick="Pick_Click" Text="Išrinkti"
/>
            </div>
    </form>

</body>
</html>
```

Forma.aspx.cs

```
using Lab2.Classes.Utils;
using System;
using System.Collections.Generic;
using System.IO;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using Lab2_v2.Employees;
using Lab2_v2.Bonuses;
using Lab2_v2;
using Lab2_v2.Einput;
using Lab2_v2.Utils;
using Lab2_v2.Payouts;


namespace Lab2
{

    public partial class Forma : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            SetTableHeaders();
            LoadSessionData();
        }

        private void SetTableHeaders()
        {
            string[] headers;

            headers = new string[] { "Tema 1", "Tema 2", "Tema 3", "Tema 4" };
            AddHeader(headers, PayoutsPerTheme);

            headers = new string[] { "Asm. k.", "Pavardė", "Vardas", "Bankas", "Banko
sąskaita" };
            AddHeader(headers, DataFile1);

            headers = new string[] { "Asm. k.", "Indėlis į tema1", "Indėlis į tema2",
"Indėlis į tema3", "Indėlis į tema4" };
            AddHeader(headers, DataFile2);

            headers = new string[] { "Pavardė", "Vardas", "Indėlis į tema1",
                "Indėlis į tema2", "Indėlis į tema3", "Indėlis į tema4", "Bonusas už tema
1", "Bonusas už tema 2",
                "Bonusas už tema 3", "Bonusas už tema 4", "Bonusų suma" };
            AddHeader(headers, Results);
            AddHeader(headers, LessAvg);
        }

        private void LoadSessionData()
        {
            if (Session["Bonuses"] is BonusList bonuses)
```

```
{
    AddDataToThemeTable(bonuses);
```

```
            }

            if (Session["Workers"] is EmployeeList workers)
            {
                AddDataFromFile1(workers);
            }

            if (Session["Contributions"] is InputList contributions)
            {
                AddDataFromFile2(contributions);
            }

            if (Session["Payouts"] is PayoutList payouts)
            {
                AddDataToResultsTable(payouts, Results);
            }

            if (Session["Less"] is PayoutList LessThanAverage)
            {
                AddDataToResultsTable(LessThanAverage, LessAvg);
            }
        }

        protected void Upload1_Click(object sender, EventArgs e)
        {
            BonusList bonuses = new BonusList();
            EmployeeList workers = new EmployeeList();
            InputList contributions = new InputList();

            if (FileUpload1.HasFile)
            {
                workers = InOutUtils.ReadWorkers(FileUpload1.FileContent);
                Session["Workers"] = workers;
            }

            if (FileUpload2.HasFile)
            {
                using (MemoryStream memStream = new MemoryStream())
                {
                    FileUpload2.FileContent.CopyTo(memStream);
                    memStream.Position = 0;

                    using (MemoryStream firstReadStream = new
MemoryStream(memStream.ToArray()))
                    {
                        contributions = InOutUtils.ReadContributions(firstReadStream);
                        Session["Contributions"] = contributions;
                    }

                    memStream.Position = 0;

                    using (MemoryStream secondReadStream = new
MemoryStream(memStream.ToArray()))
                    {
                        bonuses = InOutUtils.ReadBonusAmounts(secondReadStream);
                        Session["Bonuses"] = bonuses;
                    }
                }
            }

            AddDataToThemeTable(bonuses);
            AddDataFromFile1(workers);
            AddDataFromFile2(contributions);
```

```
            PayoutList payouts = TaskUtils.CalculateBonus(contributions, workers,
bonuses);
            payouts.Sort();
            Session["Payouts"] = payouts;
            AddDataToResultsTable(payouts, Results);

            PayoutList LessThanAverage = TaskUtils.LessThanAvarage(payouts);
            LessThanAverage.Sort();
            Session["Less"] = LessThanAverage;

            AddDataToResultsTable(LessThanAverage, LessAvg);
            string basepath = AppDomain.CurrentDomain.BaseDirectory;
            string filePath = Path.Combine(basepath, "ExternalData.txt");
            if(File.Exists(filePath)) { File.Delete(filePath); }
            InOutUtils.WriteToFileBonus(filePath, bonuses, "Premijų sumos: ");
            InOutUtils.WriteToFileWorkers(filePath, workers, "Darbuotojai: ");
            InOutUtils.WriteToFileInput(filePath, contributions, "Indėliai: ");
            InOutUtils.WriteToFileCalculations(filePath, payouts, "Išmokėjimai: ");
            InOutUtils.WriteToFileCalculations(filePath, LessThanAverage, "Darbuotojai
uždirbę mažiau nei premijų vidurkis: ");
        }

        protected void Pick_Click(object sender, EventArgs e)
        {

        }
    }
}
```

FormaMethods.aspx.cs

```
using Lab2.Classes;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using Lab2_v2.Einput;
using Lab2_v2.Bonuses;
using Lab2_v2.Employees;
using Lab2.Classes.Utils;
using Lab2_v2;
using Lab2_v2.Payouts;



namespace Lab2
{
    public partial class Forma : System.Web.UI.Page
    {
        /// <summary>
        /// Sets all headers for tables.
        /// </summary>
        /// <param name="headers">Headers in array</param>
        /// <param name="table">Table to set to.</param>

        public void AddHeader(string[] headers, Table table)
        {
            TableRow headerRow = new TableRow();
            foreach (string header in headers)
            {
```

```
            TableCell cell = new TableCell();
            cell.Text = header;
            headerRow.Cells.Add(cell);
        }
        table.Rows.Add(headerRow);
    }


    /// <summary>
    /// Adds all data to table.
    /// </summary>
    /// <param name="bonuses">Linked list from which data is added from.</param>

    public void AddDataToThemeTable(BonusList bonuses)
    {
        TableRow row = new TableRow();
        for (bonuses.Start(); bonuses.Exists(); bonuses.Next())
        {
            double curr = bonuses.Get();
            TableCell cell = new TableCell();
            cell.Text = curr.ToString();
            row.Cells.Add(cell);
        }
        PayoutsPerTheme.Rows.Add(row);
    }

    /// <summary>
    /// Adds all data to table.
    /// </summary>
    /// <param name="workers">Linked list from which data is added from.</param>

    public void AddDataFromFile1(EmployeeList workers)
    {
        for (workers.Start(); workers.Exists(); workers.Next())
        {
            TableRow row = new TableRow();
            Employee curr = workers.Get();

            TableCell idCell = new TableCell();
            idCell.Text = curr.Id.ToString();
            row.Cells.Add(idCell);

            TableCell lastNameCell = new TableCell();
            lastNameCell.Text = curr.LastName;
            row.Cells.Add(lastNameCell);

            TableCell firstNameCell = new TableCell();
            firstNameCell.Text = curr.Name;
            row.Cells.Add(firstNameCell);

            TableCell bankNameCell = new TableCell();
            bankNameCell.Text = curr.BankName;
            row.Cells.Add(bankNameCell);

            TableCell bankAccountCell = new TableCell();
            bankAccountCell.Text = curr.AccountNumber;
            row.Cells.Add(bankAccountCell);
            DataFile1.Rows.Add(row);
        }
    }

    /// <summary>
    /// Adds all data to table.
```

```
        /// </summary>
        /// <param name="conts">Linked list from which data is added from.</param>

        public void AddDataFromFile2(InputList conts)
        {
            for (conts.Start(); conts.Exists(); conts.Next())
            {
                TableRow row = new TableRow();

                Input curr = conts.Get();

                TableCell idCell = new TableCell();
                idCell.Text = curr.Id.ToString();
                row.Cells.Add(idCell);

                for (curr.Bonuses.Start(); curr.Bonuses.Exists(); curr.Bonuses.Next())
                {
                    TableCell cell = new TableCell();
                    cell.Text = curr.Bonuses.Get().ToString();
                    row.Cells.Add(cell);
                }
                DataFile2.Rows.Add(row);
            }
        }

        /// <summary>
        /// Adds data to Results table.
        /// </summary>
        /// <param name="payoutlist">Linked list from which data is added from.</param>

        public void AddDataToResultsTable(PayoutList payoutlist, Table table)
        {
            for (payoutlist.Start(); payoutlist.Exists(); payoutlist.Next())
            {
                TableRow row = new TableRow();
                Payout curr = payoutlist.Get();

                TableCell lastNameCell = new TableCell();
                lastNameCell.Text = curr.LastName;
                row.Cells.Add(lastNameCell);

                TableCell firstNameCell = new TableCell();
                firstNameCell.Text = curr.Name;
                row.Cells.Add(firstNameCell);

                for (curr.BonusCoef.Start(); curr.BonusCoef.Exists();
curr.BonusCoef.Next())
                {
                    TableCell tableCell = new TableCell();
                    tableCell.Text = curr.BonusCoef.Get().ToString();
                    row.Cells.Add(tableCell);
                }
                for (curr.BonusAmounts.Start(); curr.BonusAmounts.Exists();
curr.BonusAmounts.Next())
                {
                    TableCell tableCell = new TableCell();
                    tableCell.Text = curr.BonusAmounts.Get().ToString();
                    row.Cells.Add(tableCell);
                }
                TableCell SumCell = new TableCell();
                SumCell.Text = curr.Sum.ToString();
                row.Cells.Add(SumCell);
                table.Rows.Add(row);
```

```
            }

        }
```

Styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f8f9fa;
    margin: 20px;
    padding: 0;
    text-align: center;
    background-color: aqua;
}

table {
    width: 80%;
    margin: 20px auto;
    border-collapse: collapse;
    background-color: #ffffff;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    border-radius: 8px;
    overflow: hidden;
}

th, td {
    border: 2px solid black;
    padding: 10px;
    text-align: center;
}

th {
    background-color: #007bff;
    color: white;
    font-weight: bold;
}



#DataFile1, #DataFile2, #PayoutsPerTheme, #Results, #PickOut, #lessAvg {
    margin: 20px auto;
    padding: 10px;
    border: 2px solid black;
    border-radius: 8px;
    background-color: #ffffff;
    width: 90%;
}

button {
    background-color: #007bff;
    color: white;
    border: none;
    padding: 10px 20px;
    margin: 10px;
    font-size: 16px;
    border-radius: 5px;
    cursor: pointer;
}

    button:hover {
        background-color: #0056b3;
    }
```

2.7. Pradiniai duomenys ir rezultatai

Duomenys nr. 1

U6a.txt

23232323;Ralius;Balius;Ukio_Bankas;LT123

56566565;Domas;Romas;SUB;LT456

89998988;Bukis;Rukis;Swud;LT789

58585858;Dukis;Akis;Bek;LT420

U6b.txt

4000;6000;8000;2000

23232323;0.37;0.37;0.5;0.37

56566565;0.2;0.2;0.2;0.2

89998988;0.43;0.43;0.3;0.43

58585858;0.20;0.43;0.23;0.43

Duomenys tikrinantys esminį programos veikimą.

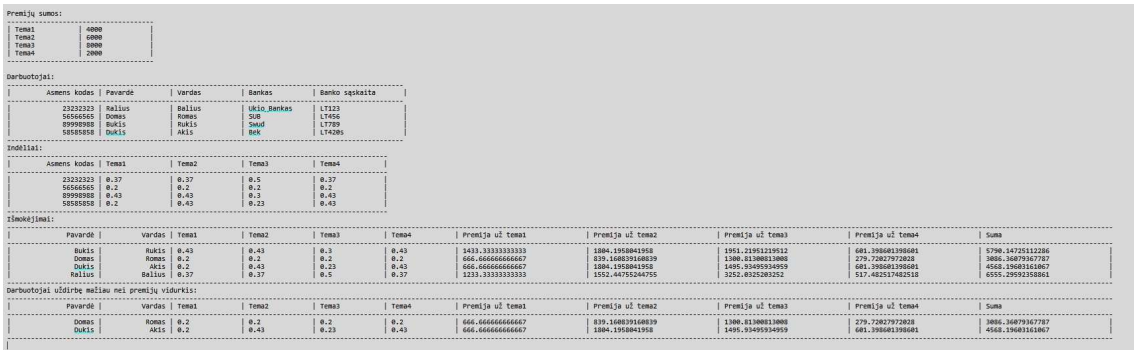
Premijų skaičiavimo rezultatai:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Bukis	Rukis	0.43	0.43	0.3	0.43	1433.33333333333	1804.1958041958	1951.21951219512	601.398601398601	5790.14725112286
Domas	Romas	0.2	0.2	0.2	0.2	666.666666666667	839.160839160839	1300.81300813008	279.72027972028	3086.36079367787
Dukis	Akis	0.2	0.43	0.23	0.43	666.666666666667	1804.1958041958	1495.93495934959	601.398601398601	4568.19603161067
Ralius	Balius	0.37	0.37	0.5	0.37	1233.33333333333	1552.44755244755	3252.0325203252	517.482517482518	6555.29592358861

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Domas	Romas	0.2	0.2	0.2	0.2	666.666666666667	839.160839160839	1300.81300813008	279.72027972028	3086.36079367787
Dukis	Akis	0.2	0.43	0.23	0.43	666.666666666667	1804.1958041958	1495.93495934959	601.398601398601	4568.19603161067

Pav. 7 Rezultatai



Premijų sumos:

Tema1	4000
Tema2	6000
Tema3	8000
Tema4	2000

Darbuotojai:

Asmens kodas	Pavardė	Vardas	Bankas	Banko sąskaita
23232323	Ralius	Balius	Ukio_Bankas	LT123
56566565	Domas	Romas	SUB	LT456
89998988	Bukis	Rukis	Swud	LT789
58585858	Dukis	Akis	Bek	LT420s

Indėliai:

Asmens kodas	Tema1	Tema2	Tema3	Tema4
23232323	0.37	0.37	0.5	0.37
56566565	0.2	0.2	0.2	0.2
89998988	0.43	0.43	0.3	0.43
58585858	0.2	0.43	0.23	0.43

Išmokėjimai:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Bukis	Rukis	0.43	0.43	0.3	0.43	1433.3333333333333	1804.1958041958	1951.219512195512	601.398601398601	5790.14725112286
Domas	Romas	0.2	0.2	0.2	0.2	666.6666666666667	839.160839160839	1300.8130081300808	279.72027972028	3086.36079367787
Dukis	Akis	0.2	0.43	0.23	0.43	666.6666666666667	1804.1958041958	1495.9349593495959	601.398601398601	4568.19603161067
Ralius	Balius	0.37	0.37	0.5	0.37	1233.3333333333333	1552.447552447755	3252.032520325203	517.482517482518	6555.2959235861

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Domas	Romas	0.2	0.2	0.2	0.2	666.6666666666667	839.160839160839	1300.8130081300808	279.72027972028	3086.36079367787
Dukis	Akis	0.2	0.43	0.23	0.43	666.6666666666667	1804.1958041958	1495.9349593495959	601.398601398601	4568.19603161067

Pav. 8 Rezultatai faile

Duomenys nr. 2

U6a.txt

23232323;Ralius;Balius;Ukio_Bankas;LT123

56566565;Domas;Romas;SUB;LT456

89998988;Bukis;Rukis;Swud;LT789

58585858;Dukis;Akis;Bek;LT420

U6b.txt

4000;6000;8000;2000

23232323;1;2;3;4

56566565;6;6;7;8

89998988;9;10;11;12

58585858;3;4;5;8

Duomenys su skirtingais koeficientais.

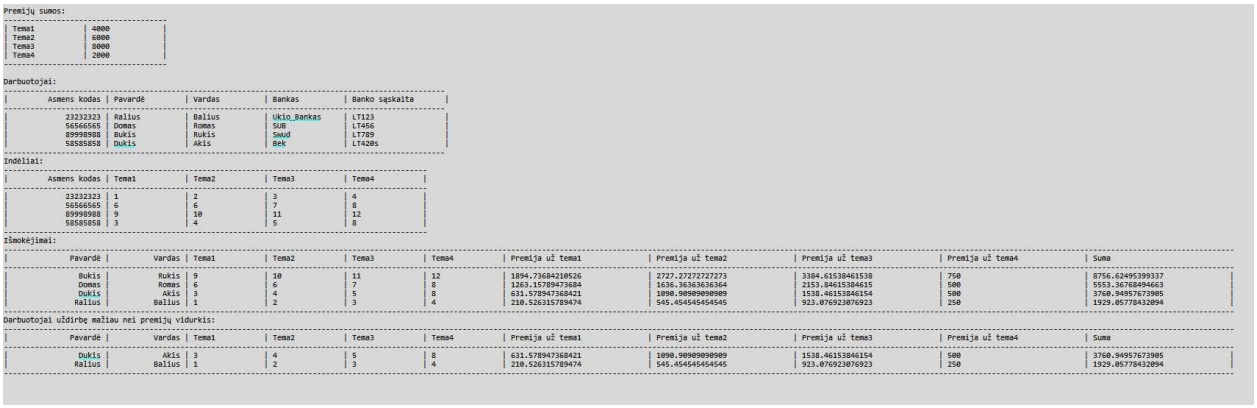
Premijų skaičiavimo rezultatai:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Bukis	Rukis	9	10	11	12	1894.73684210526	2727.27272727273	3384.61538461538	750	8756.62495399337
Domas	Romas	6	6	7	8	1263.15789473684	1636.36363636364	2153.84615384615	500	5553.36768494663
Dukis	Akis	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673905
Ralius	Balius	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Dukis	Akis	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673905
Ralius	Balius	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094

Pav. 9 Rezultatai



Premijų sumos:

Tema1	4000
Tema2	6000
Tema3	8000
Tema4	2000

Darbuotojai:

Asmens kodas	Pavardė	Vardas	Bankas	Banko sąskaita
23232323	Ralius	Balius	Ukio_Bankas	LT123
56566565	Domas	Romas	SUB	LT456
89998988	Bukis	Rukis	Swud	LT789
58585858	Dukis	Akis	Bek	LT420s

Indėliai:

Asmens kodas	Tema1	Tema2	Tema3	Tema4
23232323	1	2	3	4
56566565	6	6	7	8
89998988	9	10	11	12
58585858	3	4	5	8

Išmokėjimai:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Bukis	Rukis	9	10	11	12	1894.73684210526	2727.27272727273	3384.61538461538	750	8756.62495399337
Domas	Romas	6	6	7	8	1263.15789473684	1636.36363636364	2153.84615384615	500	5553.36768494663
Dukis	Akis	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673985
Ralius	Balius	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Dukis	Akis	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673985
Ralius	Balius	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094

Pav. 10 Rezultatai faile

Duomenys nr. 3

U6a.txt

23232323;A;C;Ukio_Bankas;LT123

56566565;A;A;SUB;LT456

89998988;B;A;Swud;LT789

58585858;A;B;Bek;LT420

U6b.txt

4000;6000;8000;2000

23232323;1;2;3;4

56566565;6;6;7;8

89998988;9;10;11;12

58585858;3;4;5;8

Duomenys tikrina rikiavimą.

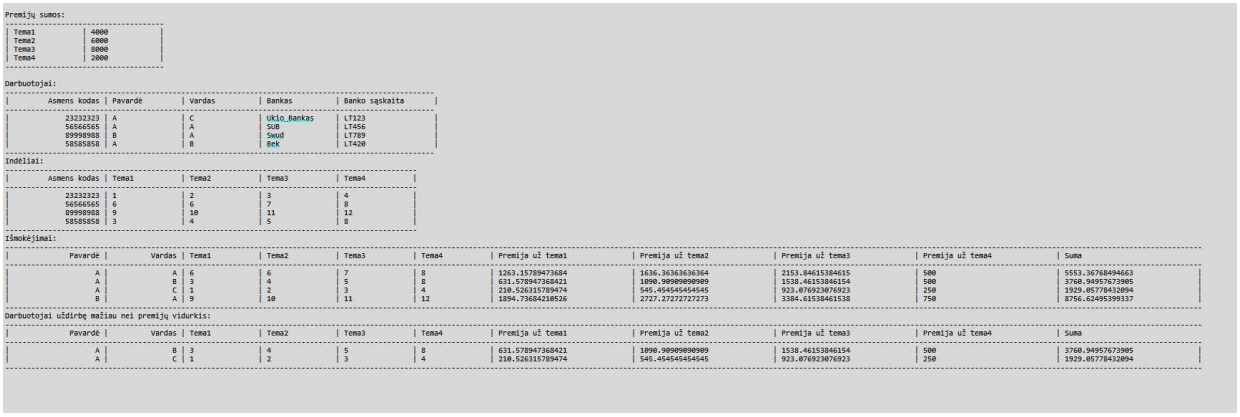
Premijų skaičiavimo rezultatai:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
A	A	6	6	7	8	1263.15789473684	1636.36363636364	2153.84615384615	500	5553.36768494663
A	B	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673905
A	C	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094
B	A	9	10	11	12	1894.73684210526	2727.27272727273	3384.61538461538	750	8756.62495399337

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
A	B	3	4	5	8	631.578947368421	1090.90909090909	1538.46153846154	500	3760.94957673905
A	C	1	2	3	4	210.526315789474	545.454545454545	923.076923076923	250	1929.05778432094

Pav. 11 Rezultatai



Pav. 12 Rezultatai faile

2.8. Dėstytojo pastabos

- Vidiniai linked list klasės metodai negali nadoti sąsajos metodų;
- Duomenų klasėje esantys vienkrypčiai sąrašai turi būti private;
- Sum turi būti skaičiuojamas automatiškai;
- SumCalc() metodas neturi grąžinti reikšmės;

3. Bendrinės klasės ir testavimas (L3)

3.1. Darbo užduotis

LD_6. Premijos. Moksliniai darbuotojai atliko darbus 4 skirtingose temose. Visos temos gavo premijas. Remiantis darbuotojų indėliais, suskaičiuokite kiekvienam darbuotojui priklausančios premijos dydį pagal kiekvieną temą atskirai ir bendrą premijų sumą. Darbuotojo indėlis rodo, kokia premijos dalis jam priklauso. Indėlis gali būti išreikštas bet kokiu skaičiumi. Bet tas pats matas naudojamas visiems darbuotojams. Neišdalinkite pinigų daugiau, negu turite, ir turite išdalinti visus pinigus. Sudarykite sąrašą darbuotojų (pavardė, vardas, darbuotojo indėliai, darbuotojo premijų dydžiai pagal temas, bendra premijų sumą), kurie uždirbo mažiau už vidurkį.

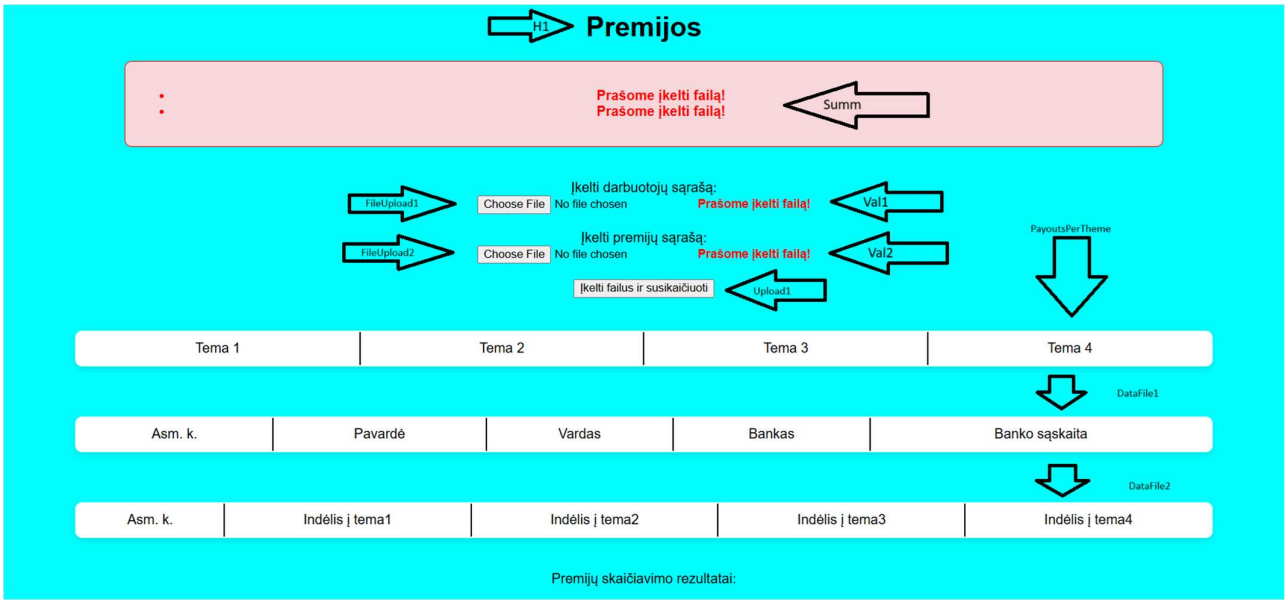
Duomenys:

• Tekstiniame faile U6a.txt duota informacija apie darbuotojus: darbuotojų asmens kodai, pavardės, vardai, banko pavadinimas, sąskaitos numeris.

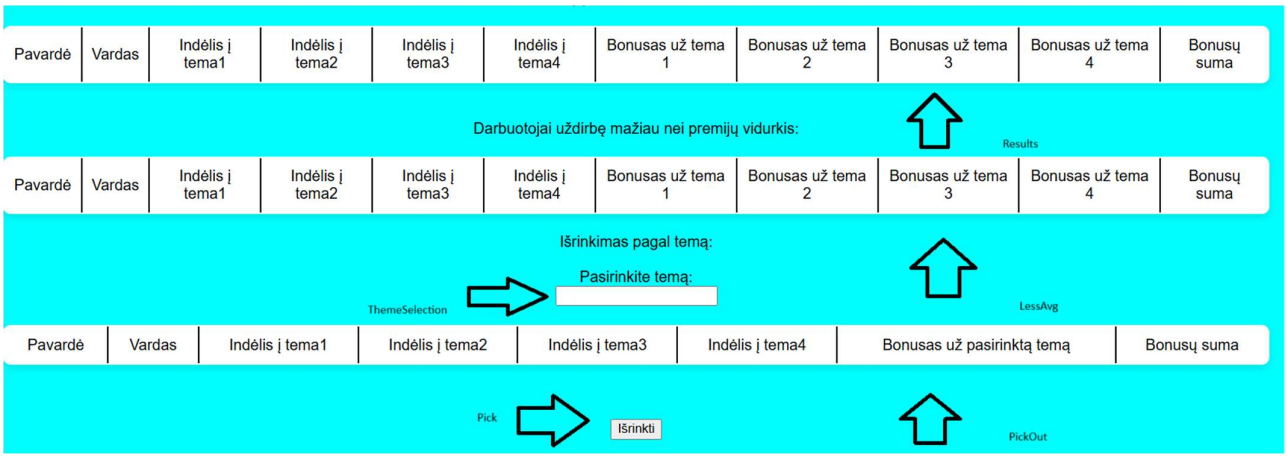
• Tekstiniame faile U6b.txt duota informacija apie indėlius į darbą: pirmoje eilutėje - premijų dydžiai, tolesnėse eilutėse - darbuotojų asmens kodai, darbuotojų indėliai, kurie išreikšti naudingumo koeficientu, į eilinę temą. Informacija apie darbuotoją užima vieną eilutę.

Spausdinamas sąrašas turi būti surikiuotas pagal darbuotojų pavardes, vardus abėcėlės tvarka ir bendrą premijų sumą didėjimo tvarka. Sudarykite nurodytos temos (įvedama klaviatūra) darbuotojų uždarbių sąrašą (pavardė, vardas, darbuotojo indėliai, darbuotojo premijų dydžiai pagal temas, bendra premijų sumą).

3.2. Grafinės naudotojo sąsajos schema



Pav. 13 GUI schema

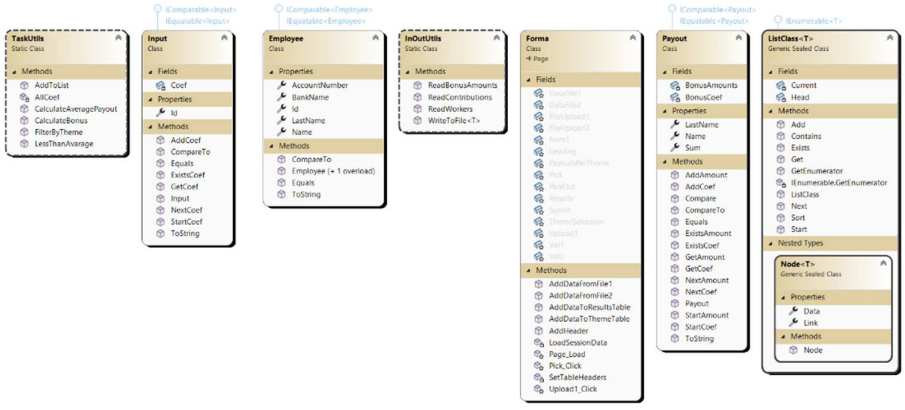


Pav.14 GUI schema

3.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė
H1	Text	Premijos
Summ	CssClass	Summ
Val1	ControlToValidate	FileUpload1
	ErrorMessage	Prašome įkelti failą!
	CssClass	Val1
Val2	ControlToValidate	FileUpload2
	ErrorMessage	Prašome įkelti failą!
	CssClass	Val2
Upload1	Text	Įkelti ir suskaičiuoti

3.4. Klasių diagrama



Pav.15 Klasių diagrama

3.5. Programos naudotojo vadovas

Atisdarius programą reikia įkelti failus, kurie turi informaciją apie darbuotojus ir failą, kuris turi informaciją apie darbuotojų indėlius bei premijų dydžius. Įkėlus failus spaudžiamas mygtukas „Įkelti ir suskaičiuoti“. Programa automatiškai paskirsto premijas, pagal darbuotojų indėlius. Dar kartą įkėlus failus ir įvedus norimą temą, yra galimybė išfiltruoti pagal pasirinktą temą.

3.6. Programos tekstas

Employee.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab3
{
    /// <summary>
    /// Employee object class.
    /// </summary>
    public class Employee : IComparable<Employee>, IEquatable<Employee>
    {
        public int Id { get; set; }
        public string LastName { get; set; }
        public string Name { get; set; }
        public string BankName { get; set; }
        public string AccountNumber { get; set; }

        /// <summary>
        /// Default constructor for object.
        /// </summary>
        public Employee() { }
        /// <summary>
        /// Constructor for object.
        /// </summary>
        /// <param name="id">Id of a person.</param>
        /// <param name="lastname">Last name of a person.</param>
        /// <param name="name">Name of a person.</param>
        /// <param name="bankname">Bank name of a person.</param>
        /// <param name="accountnumber">Account number of person.</param>

        public Employee(int id, string lastname, string name, string bankname, string
accountnumber)
        {
            this.Id = id;
            this.LastName = lastname;
            this.Name = name;
            this.BankName = bankname;
            this.AccountNumber = accountnumber;
        }

        /// <summary>
        /// ToString override for formatting.
        /// </summary>
        /// <returns>Formatted string.</returns>
```

```
        public override string ToString()
        {
            return string.Format($" {Id,20} | {LastName,-15} | {Name,-15} | {BankName,-15} |
{AccountNumber,-20} ");
        }

        /// <summary>
        /// Compares two Employee objects by Id.
        /// </summary>
        /// <param name="other">other Employee object.</param>
        /// <returns>-1 ,0 or 1 depending on comparison the values</returns>
        public int CompareTo(Employee other)
        {
            return this.Id.CompareTo(other.Id);
        }

        /// <summary>
        /// Compares two Employee objects by Id.
        /// </summary>
        /// <param name="other">other Employee object</param>
        /// <returns>true or false depending on the values</returns>

        public bool Equals(Employee other)
        {
            return this.Id.Equals(other.Id);
        }
    }
                                                }
```

Input.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab3
{
    /// <summary>
    /// Input object class.
    /// </summary>
    public class Input : IComparable<Input>, IEquatable<Input>
    {
        public int Id { get; set; }
        private ListClass<double> Coef;

        /// <summary>
        /// Constructor for object.
        /// </summary>
        /// <param name="id">ID of a person.</param>
        /// <param name="bonuses">Bonus linked list for storing bonus amounts.</param>
        public Input(int id, ListClass<double> bonuses)
        {
            this.Id = id;
            this.Coef = bonuses;
        }

        /// <summary>
        /// Starts the linked list for coeficients.
        /// </summary>

        public void StartCoef()
        {
```

```
    Coef.Start();
}

/// <summary>
/// Checks if the linked list for coeficients exists.
/// </summary>
/// <returns>True or false depening on existance of a value.</returns>

public bool ExistsCoef()
{
    return Coef.Exists();
}

/// <summary>
/// Moves to the next coeficient in the linked list.
/// </summary>

public void NextCoef()
{
    Coef.Next();
}

/// <summary>
/// Gets the current coeficient from the linked list.
/// </summary>
/// <returns>Gets node data.</returns>

public double GetCoef()
{
    return Coef.Get();
}

/// <summary>
/// Adds a coeficient to the linked list.
/// </summary>
/// <param name="value">Value to add.</param>

public void AddCoef(double value)
{
    Coef.Add(value);
}

/// <summary>
/// Compares two Input objects by Id.
/// </summary>
/// <param name="other">Other Input object.</param>
/// <returns></returns>

public int CompareTo(Input other)
{
    return this.Id.CompareTo(other.Id);
}

/// <summary>
/// Compares two Input objects by Id.
/// </summary>
/// <param name="other">Other Input object.</param>
/// <returns>True or false depending on values.</returns>
public bool Equals(Input other)
{
    return this.Id.Equals(other.Id);
}

/// <summary>
```

```
        /// ToString override for formatting.
        /// </summary>
        /// <returns>Foramted string</returns>

        public override string ToString()
        {
            Coef.Start();
            string firstCoef = Coef.Get().ToString();
            Coef.Next();
            string secondCoef = Coef.Get().ToString();
            Coef.Next();
            string thirdCoef = Coef.Get().ToString();
            Coef.Next();
            string fourthCoef = Coef.Get().ToString();

            return string.Format($"{this.Id,20} | {firstCoef,-15} | {secondCoef,-15} |
{thirdCoef,-15} | {fourthCoef,-15} ");
        }
    }
}
```

Payout.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace Lab3
{
    public class Payout : IComparable<Payout>, IEquatable<Payout>
    {
        /// <summary>
        /// Class for Payout object.
        /// </summary>

        public string LastName { get; set; }
        public string Name { get; set; }
        private ListClass<double> BonusCoef;
        private ListClass<double> BonusAmounts;
        public double Sum { get; set; }

        /// <summary>
        /// Constructor for object.
        /// </summary>
        /// <param name="lastname">Last name of a person.</param>
        /// <param name="name">Name of a person.</param>
        /// <param name="bonusCoef">Linked list for all coeficients.</param>
        /// <param name="bonusAmounts">Linked list for bonus amounts.</param>

        public Payout(string lastname, string name)
        {
            this.LastName = lastname;
            this.Name = name;
            this.BonusCoef = new ListClass<double>();
            this.BonusAmounts = new ListClass<double>();
            this.Sum = 0;
        }
        /// <summary>
        /// Adds a coeficient to the linked list.
        /// </summary>
        public void StartCoef()
        {
```

```
    BonusCoef.Start();
}
/// <summary>
/// Checks if the linked list for coeficients exists.
/// </summary>
/// <returns></returns>
public bool ExistsCoef()
{
    return BonusCoef.Exists();
}
/// <summary>
/// Moves to the next coeficient in the linked list.
/// </summary>
public void NextCoef()
{
    BonusCoef.Next();
}
/// <summary>
/// Gets the current coeficient from the linked list.
/// </summary>
/// <returns></returns>
public double GetCoef()
{
    return BonusCoef.Get();
}
/// <summary>
/// Adds a coeficient to the linked list.
/// </summary>
/// <param name="value"></param>
public void AddCoef(double value)
{
    BonusCoef.Add(value);
}
/// <summary>
/// Adds a bonus amount to the linked list.
/// </summary>
public void StartAmount()
{
    BonusAmounts.Start();
}
/// <summary>
/// Checks if the linked list for bonus amounts exists.
/// </summary>
/// <returns></returns>
public bool ExistsAmount()
{
    return BonusAmounts.Exists();
}
/// <summary>
/// Moves to the next bonus amount in the linked list.
/// </summary>
public void NextAmount()
{
    BonusAmounts.Next();
}
/// <summary>
/// Gets the current bonus amount from the linked list.
/// </summary>
/// <returns></returns>
public double GetAmount()
{
    return BonusAmounts.Get();
}
/// <summary>
```

```
        /// Adds a bonus amount to the linked list.
        /// </summary>
        /// <param name="value"></param>
        public void AddAmount(double value)
        {
            Sum += value;
            BonusAmounts.Add(value);
        }

        /// <summary>
        /// Compares node data.
        /// </summary>
        /// <param name="other">Other node data.</param>
        /// <returns>True if they are the same false otherwise.</returns>
        public bool Compare(Payout other)
        {
            if (other.Name == Name && other.LastName == LastName)
            {
                return true;
            }
            return false;
        }


        /// <summary>
        /// Compares two node data.
        /// </summary>
        /// <param name="other">Other node data.</param>
        /// <returns></returns>
        public int CompareTo(Payout other)
        {
            int lastNameComp = string.Compare(this.LastName, other.LastName,
StringComparison.OrdinalIgnoreCase);
            if (lastNameComp != 0)
            {
                return lastNameComp;
            }

            int nameComp = string.Compare(this.Name, other.Name,
StringComparison.OrdinalIgnoreCase);
            if (nameComp != 0)
            {
                return nameComp;
            }

            return this.Sum.CompareTo(other.Sum);
        }
        /// <summary>
        /// Checks if two node data are equal.
        /// </summary>
        /// <param name="other">Other Payout node.</param>
        /// <returns></returns>
        public bool Equals(Payout other)
        {
            return this.Name.Equals(other.Name) && this.LastName.Equals(other.LastName);
        }
        /// <summary>
        /// ToString override for formatting.
        /// </summary>
        /// <returns>Formated string.</returns>
        public override string ToString()
        {
            BonusCoef.Start();
            string firstCoef = BonusCoef.Get().ToString();
```

```
            BonusCoef.Next();
            string secondCoef = BonusCoef.Get().ToString();
            BonusCoef.Next();
            string thirdCoef = BonusCoef.Get().ToString();
            BonusCoef.Next();
            string fourthCoef = BonusCoef.Get().ToString();

            BonusAmounts.Start();
            string firstam = BonusAmounts.Get().ToString();
            BonusAmounts.Next();
            string secondam = BonusAmounts.Get().ToString();
            BonusAmounts.Next();
            string thirdam = BonusAmounts.Get().ToString();
            BonusAmounts.Next();
            string fourtham = BonusAmounts.Get().ToString();



            return string.Format($"{LastName,20} | {Name,15} | {firstCoef,-15} |
{secondCoef,-15} |" +
                    $" {thirdCoef,-15} | {fourthCoef,-15} | {firstam,-30} | {secondam,-30}
|" +
                    $" {thirdam,-30} | {fourtham,-30} | {Sum,-30}");
        }
    }
                                            }
```

Payout.cs

```
using System;
using System.Collections;
using System.Collections.Generic;

namespace Lab3
{
    /// <summary>
    /// Class for linked list.
    /// </summary>
    /// <typeparam name="T">Generic object type.</typeparam>
    public sealed class ListClass<T> : IEnumerable<T> where T : IComparable<T>,
IEquatable<T>
    {
        /// <summary>
        /// Node class for linked list.
        /// </summary>
        private sealed class Node<T>
        {
            public T Data { get; set; }
            public Node<T> Link { get; set; }

            /// <summary>
            /// Constructor for node.
            /// </summary>
            /// <param name="data">Data to add.</param>
            /// <param name="next">Pointer to next node.</param>

            public Node(T data, Node<T> next)
            {
                this.Data = data;
                this.Link = next;
            }
        }
```

```
Node<T> Head;
Node<T> Current;

/// <summary>
/// Constructor for linked list.
/// </summary>
public ListClass()
{
    Head = null;
    Current = null;
}
/// <summary>
/// Adds a new node to the end of the linked list.
/// </summary>
/// <param name="value"></param>
public void Add(T value)
{
    Node<T> newNode = new Node<T>(value, null);

    if (Head == null)
    {
        Head = newNode;
        return;
    }

    Node<T> current = Head;
    while (current.Link != null)
    {
        current = current.Link;
    }

    current.Link = newNode;
}
/// <summary>
/// Starts the linked list.
/// </summary>
public void Start()
{
    Current = Head;
}
/// <summary>
/// Moves to the next node in the linked list.
/// </summary>
public void Next()
{
    Current = Current.Link;
}
/// <summary>
/// Checks if the current node exists.
/// </summary>
/// <returns>True if exists, else false.</returns>
public bool Exists()
{
    return Current != null;
}
/// <summary>
/// Gets the current node data.
/// </summary>
/// <returns>Data of a node.</returns>
public T Get()
{
    return Current.Data;
}
/// <summary>
```

```
        /// Adds a new node to the start of the linked list.
        /// </summary>
        /// <param name="node">Node to check.</param>
        /// <returns>True if contains, else false.</returns>
        public bool Contains(T node)
        {
            Node<T> current = Head;
            while (current != null)
            {
                if (current.Data.Equals(node))
                {
                    return true;
                }
                current = current.Link;
            }
            return false;
        }
        /// <summary>
        /// Sorts the linked list using selection sort.
        /// </summary>
        public void Sort()
        {
            for (Node<T> outer = Head; outer != null; outer = outer.Link)
            {
                Node<T> min = outer;
                for (Node<T> inner = outer.Link; inner != null; inner = inner.Link)
                {
                    if (inner.Data.CompareTo(min.Data) < 0)
                    {
                        min = inner;
                    }
                }

                if (min != outer)
                {
                    T temp = outer.Data;
                    outer.Data = min.Data;
                    min.Data = temp;
                }
            }
        }
        /// <summary>
        /// Gets the count of nodes in the linked list.
        /// </summary>
        /// <returns></returns>
        public IEnumerator<T> GetEnumerator()
        {
            Node<T> current = Head;
            while (current != null)
            {
                yield return current.Data;
                current = current.Link;
            }
        }
        /// <summary>
        /// Gets the count of nodes in the linked list.
        /// </summary>
        /// <returns></returns>
        IEnumerator IEnumerable.GetEnumerator()
        {
            return GetEnumerator();
        }
    }
}
```

InOutUtils.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.Remoting.Messaging;
using System.Web;

namespace Lab3
{
    /// <summary>
    /// Class that handles all inputs and outputs.
    /// </summary>
    static class InOutUtils
    {
        /// <summary>
        /// Generic method that writes to a file.
        /// </summary>
        /// <typeparam name="T">Generic object type from the project.</typeparam>
        /// <param name="fileName">File to write to.</param>
        /// <param name="List">Generic list to write from.</param>
        /// <param name="Header">Header line for the table.</param>
        /// <param name="TableHeader">Table header.</param>
        /// <param name="ws">Spaces for '-' symbol.</param>
        public static void WriteToFile<T>(string fileName, IEnumerable<T> List, string
Header, string TableHeader, int ws) where T : IEquatable<T>, IComparable<T>
        {
            using (StreamWriter writer = File.AppendText(fileName))
            {
                writer.WriteLine(Header);
                writer.WriteLine(new string('-', ws));
                writer.WriteLine(TableHeader);
                writer.WriteLine(new string('-', ws));
                foreach (T obj in List)
                {
                    writer.WriteLine($"| {obj.ToString(),-10} |");
                }
                writer.WriteLine(new string('-', ws));
                writer.WriteLine();
            }

        }

        /// <summary>
        /// Reads all workers.
        /// </summary>
        /// <param name="fileContetnt">File content.</param>
        /// <returns>Worker linked list.</returns>

        public static ListClass<Employee> ReadWorkers(Stream fileContetnt)
        {
            ListClass<Employee> list = new ListClass<Employee>();
            using (StreamReader reader = new StreamReader(fileContetnt))
            {
                string line;
                while ((line = reader.ReadLine()) != null)
                {
                    string[] Values = line.Split(';');
                    int ID = int.Parse(Values[0]);
                    string LastName = Values[1];
                    string Name = Values[2];
                    string BankName = Values[3];
                    string BankAccountNum = Values[4];
```

```
                    Employee w = new Employee(ID, LastName, Name, BankName, BankAccountNum);
                    list.Add(w);
                }
            }
            return list;
        }

        /// <summary>
        /// Reads all contributions.
        /// </summary>
        /// <param name="FileContent">Content of a file.</param>
        /// <returns>ContList linked list.</returns>
        public static ListClass<Input> ReadContributions(Stream FileContent)
        {
            ListClass<Input> list = new ListClass<Input>();
            using (StreamReader reader = new StreamReader(FileContent))
            {

                reader.ReadLine();
                string line;
                while ((line = reader.ReadLine()) != null)
                {
                    string[] Values = line.Split(';');
                    int ID = int.Parse(Values[0]);
                    ListClass<double> contributionValuesList = new ListClass<double>();
                    foreach (var amount in Values.Skip(1))
                    {
                        double value = Double.Parse(amount);
                        contributionValuesList.Add(value);
                    }
                    Input c = new Input(ID, contributionValuesList);
                    list.Add(c);
                }
            }
            return list;
        }
        /// <summary>
        /// Reads all bonuses from first line of a file.
        /// </summary>
        /// <param name="FileContent">File content.</param>
        /// <returns>Linked list of all bonuses.</returns>

        public static ListClass<double> ReadBonusAmounts(Stream FileContent)
        {
            ListClass<double> list = new ListClass<double>();
            using (StreamReader reader = new StreamReader(FileContent))
            {
                string bonuses = reader.ReadLine();
                string[] Values = bonuses.Split(';');
                foreach (var value in Values)
                {
                    double bonus = double.Parse(value);
                    list.Add(bonus);
                }
            }
            return list;
        }
    }
}
```

TaskUtils.cs

```
using System;
using System.Collections.Generic;
```

```
using System.Linq;
using System.Text.RegularExpressions;
using System.Web;

namespace Lab3
{
    public static class TaskUtils
    {
        /// <summary>
        /// Creates a list of new payout objects.
        /// </summary>
        /// <param name="inputs">Input coeficients.</param>
        /// <param name="employees">Employees linked list.</param>
        /// <returns>Linked list of payout objects.</returns>
             public static ListClass<Payout> AddToList(ListClass<Input> inputs,
ListClass<Employee> employees)
        {
            ListClass<Payout> payoutList = new ListClass<Payout>();
            for (employees.Start(); employees.Exists(); employees.Next())
            {
                Employee employee = employees.Get();
                for (inputs.Start(); inputs.Exists(); inputs.Next())
                {
                    Input input = inputs.Get();
                    if (input.Id == employee.Id)
                    {
                        Payout p = new Payout(employee.LastName, employee.Name);
                        for (input.StartCoef(); input.ExistsCoef(); input.NextCoef())
                        {
                            p.AddCoef(input.GetCoef());
                        }
                        if (!payoutList.Contains(p))
                        {
                            payoutList.Add(p);
                        }
                    }
                }

            }
            return payoutList;
        }

        /// <summary>
        /// Calculates bonus for each worker and each theme.
        /// </summary>
        /// <param name="inputs">Input coeficients.</param>
        /// <param name="employees">Employees linked list.</param>
        /// <param name="bonuses">Bonus linked list.</param>
        /// <returns>Linked list with payout objects.</returns>
        public static ListClass<Payout> CalculateBonus(ListClass<Payout> payouts,
ListClass<double> bonuses, ListClass<Input> inputs)
        {
            payouts.Start();
            while (payouts.Exists())
            {
                Payout payout = payouts.Get();
                bonuses.Start();
                payout.StartCoef();
                int themeNumber = 0;

                while (bonuses.Exists())
                {
                    double totalCoef = AllCoef(inputs, themeNumber);
                    double bonusAmount = 0;
```

```
                    if (totalCoef > 0)
                    {
                        bonusAmount = (payout.GetCoef() / totalCoef) * bonuses.Get();
                    }

                    payout.AddAmount(bonusAmount);
                    bonuses.Next();
                    payout.NextCoef();
                    themeNumber++;
                }


                payouts.Next();
            }


            return payouts;
        }

        /// <summary>
        /// Calculates sum of coeficients for each theme.
        /// </summary>
        /// <param name="inputs">Input coeficients.</param>
        /// <param name="themeNumber">Theme number.</param>
        /// <returns></returns>
        private static double AllCoef(ListClass<Input> inputs, int themeNumber)
        {
            double coefs = 0;

            for (inputs.Start(); inputs.Exists(); inputs.Next())
            {
                var bonusList = inputs.Get();
                bonusList.StartCoef();
                for (int i = 0; i < themeNumber; i++)
                {
                    if (!bonusList.ExistsCoef()) break;
                    bonusList.NextCoef();
                }

                if (bonusList.ExistsCoef())
                {
                    coefs += bonusList.GetCoef();
                }
            }

            return coefs;
        }
        /// <summary>
        /// Picks out objects that have sum smaller than average sum.
        /// </summary>
        /// <param name="payouts">Payout linked list.</param>
        /// <returns>Payout linked list with objects that have smaller sums than
average.</returns>
        public static ListClass<Payout> LessThanAvarage(ListClass<Payout> payouts, double
avg)
        {
            ListClass<Payout> lessthanaverage = new ListClass<Payout>();
            for (payouts.Start(); payouts.Exists(); payouts.Next())
            {
                Payout p = payouts.Get();
                if (p.Sum < avg)
                {
                    lessthanaverage.Add(p);
```

```
                }
            }
            return lessthanaverage;
        }

        /// <summary>
        /// Calculates average of a bonus.
        /// </summary>
        /// <param name="payouts">Whole payout linked list.</param>
        /// <returns>Avergae of bonus sums.</returns>
        public static double CalculateAveragePayout(ListClass<Payout> payouts)
        {
            double sum = 0;
            int count = 0;
            for (payouts.Start(); payouts.Exists(); payouts.Next())
            {
                Payout p = payouts.Get();
                sum += p.Sum;
                count++;
            }
            return sum / count;
        }
        /// <summary>
        /// Filters payout objects by theme.
        /// </summary>
        /// <param name="Theme">Theme selected.</param>
        /// <param name="payouts">Linked list with all objects.</param>
        /// <returns>Payout linkedlist with only amount of selected theme.</returns>
        public static ListClass<Payout> FilterByTheme(string Theme, ListClass<Payout>
payouts)
        {
            ListClass<Payout> filtered = new ListClass<Payout>();


            string[] values = Theme.Split(' ');
            int value = int.Parse(values[1]) - 1;

            for (payouts.Start(); payouts.Exists(); payouts.Next())
            {
                Payout p = new Payout(payouts.Get().LastName, payouts.Get().Name);
                payouts.Get().StartAmount();
                for (int i = 0; i < value; i++)
                {
                    payouts.Get().NextAmount();
                }
                p.AddAmount(payouts.Get().GetAmount());
                p.Sum = payouts.Get().Sum;
                for (payouts.Get().StartCoef(); payouts.Get().ExistsCoef();
payouts.Get().NextCoef())
                {
                    p.AddCoef(payouts.Get().GetCoef());
                }
                filtered.Add(p);
            }
            return filtered;
        }




    }
                                            }
```

Forma.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Form.aspx.cs"
Inherits="Lab3.Forma" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
    <link rel="stylesheet" type="text/css" href="styles.css" />
</head>
<body>
    <form id="form1" runat="server">

        <h1>Premijos</h1>
            <div>
                <asp:ValidationSummary ID="Summ" runat="server" CssClass="Summ" />
                <br />
                Įkelti darbuotojų sąrašą:<br />
                <asp:FileUpload ID="FileUpload1" runat="server" />
                <asp:RequiredFieldValidator ID="Val1" runat="server"
ControlToValidate="FileUpload1" ErrorMessage="Prašome įkelti failą!"
CssClass="val1"></asp:RequiredFieldValidator>
                <br />
                <br />
                Įkelti premijų sąrašą:<br />
                <asp:FileUpload ID="FileUpload2" runat="server" />
                <asp:RequiredFieldValidator ID="Val2" runat="server"
ControlToValidate="FileUpload2" ErrorMessage="Prašome įkelti failą!"
CssClass="val2"></asp:RequiredFieldValidator>
                <br />
                <br />
                <asp:Button ID="Upload1" runat="server" Text="Įkelti failus ir
susikaičiuoti" OnClick="Upload1_Click" />
                <br />
                <br />
                <asp:Table ID="PayoutsPerTheme" runat="server"></asp:Table>
                <br />
                <asp:Table ID="DataFile1" runat="server"></asp:Table>
                <br />
                <asp:Table ID="DataFile2" runat="server"></asp:Table>
                <br />
                Premijų skaičiavimo rezultatai:<br />
                <asp:Table ID="Results" runat="server">
                </asp:Table>
                <br />
                Darbuotojai uždirbę mažiau nei premijų vidurkis:<br />
                <asp:Table ID="LessAvg" runat="server">
                </asp:Table>
                Išrinkimas pagal temą:<br />
                <br />
                Pasirinkite temą: <br />
                <asp:TextBox ID="ThemeSelection" runat="server"></asp:TextBox>
                <br />
                <asp:Table ID="PickOut" runat="server">
                </asp:Table>
                <br />
                <br />
                <asp:Button ID="Pick" runat="server" OnClick="Pick_Click" Text="Išrinkti" />
            </div>
    </form>
</body>
</html>
```

Form.aspx.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Web;
using System.Web.Hosting;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Lab3
{
       public partial class Forma : System.Web.UI.Page
       {
         protected void Page_Load(object sender, EventArgs e)
         {
             SetTableHeaders();
             LoadSessionData();
         }

         private void SetTableHeaders()
         {
             string[] headers;

             headers = new string[] { "Tema 1", "Tema 2", "Tema 3", "Tema 4" };
             AddHeader(headers, PayoutsPerTheme);

             headers = new string[] { "Asm. k.", "Pavardė", "Vardas", "Bankas", "Banko
sąskaita" };
             AddHeader(headers, DataFile1);

             headers = new string[] { "Asm. k.", "Indėlis į tema1", "Indėlis į tema2",
"Indėlis į tema3", "Indėlis į tema4" };
             AddHeader(headers, DataFile2);

             headers = new string[] { "Pavardė", "Vardas", "Indėlis į tema1",
                 "Indėlis į tema2", "Indėlis į tema3", "Indėlis į tema4", "Bonusas už tema
1", "Bonusas už tema 2",
                 "Bonusas už tema 3", "Bonusas už tema 4", "Bonusų suma" };
             AddHeader(headers, Results);
             AddHeader(headers, LessAvg);

             headers = new string[] { "Pavardė", "Vardas", "Indėlis į tema1",
                 "Indėlis į tema2", "Indėlis į tema3", "Indėlis į tema4", "Bonusas už
pasirinktą temą", "Bonusų suma" };
             AddHeader(headers, PickOut);

         }

         private void LoadSessionData()
         {
             if (Session["Bonuses"] is ListClass<double> bonuses)
             {
                 AddDataToThemeTable(bonuses);
             }

             if (Session["Workers"] is ListClass<Employee> workers)
             {
                 AddDataFromFile1(workers);
             }

             if (Session["Contributions"] is ListClass<Input> contributions)
             {
```

```
                AddDataFromFile2(contributions);
            }

            if (Session["Payouts"] is ListClass<Payout> payouts)
            {
                AddDataToResultsTable(payouts, Results);
            }

            if (Session["Less"] is ListClass<Payout> LessThanAverage)
            {
                AddDataToResultsTable(LessThanAverage, LessAvg);
            }
        }

        protected void Upload1_Click(object sender, EventArgs e)
        {
            ListClass<double>  bonuses = new ListClass<double>();
            ListClass<Employee> workers = new ListClass<Employee>();
            ListClass<Input> contributions = new ListClass<Input>();

            if (FileUpload1.HasFile)
            {
                workers = InOutUtils.ReadWorkers(FileUpload1.FileContent);
                Session["Workers"] = workers;
            }

            if (FileUpload2.HasFile)
            {
                using (MemoryStream memStream = new MemoryStream())
                {
                    FileUpload2.FileContent.CopyTo(memStream);
                    memStream.Position = 0;

                    using (MemoryStream firstReadStream = new
MemoryStream(memStream.ToArray()))
                    {
                        contributions = InOutUtils.ReadContributions(firstReadStream);
                        Session["Contributions"] = contributions;
                    }

                    memStream.Position = 0;

                    using (MemoryStream secondReadStream = new
MemoryStream(memStream.ToArray()))
                    {
                        bonuses = InOutUtils.ReadBonusAmounts(secondReadStream);
                        Session["Bonuses"] = bonuses;
                    }
                }
            }

            AddDataToThemeTable(bonuses);
            AddDataFromFile1(workers);
            AddDataFromFile2(contributions);
            ListClass<Payout> payoutsList = TaskUtils.AddToList(contributions, workers);
            ListClass<Payout> payouts = TaskUtils.CalculateBonus(payoutsList, bonuses,
contributions);
            payouts.Sort();
            Session["Payouts"] = payouts;
            AddDataToResultsTable(payouts, Results);
            double avg = TaskUtils.CalculateAveragePayout(payouts);
            ListClass<Payout> LessThanAverage = TaskUtils.LessThanAvarage(payouts, avg);
            LessThanAverage.Sort();
            Session["Less"] = LessThanAverage;
```

```
            AddDataToResultsTable(LessThanAverage, LessAvg);


            string basepath = AppDomain.CurrentDomain.BaseDirectory;
            string filePath = Path.Combine(basepath, "ExternalData.txt");
            if (File.Exists(filePath)) { File.Delete(filePath); }
            InOutUtils.WriteToFile(filePath, bonuses, "Premijų sumos: ", $"| {"Sumos" ,-10}
|", 15);
            InOutUtils.WriteToFile(filePath, workers, "Darbuotojai: ", $"| {"Asmens
kodas",20} |" +
                $" {"Pavardė",-15} | {"Vardas",-15} | {"Bankas",-15} | {"Banko sąskaita",-
20} | ", 100);
            InOutUtils.WriteToFile(filePath, contributions, "Indėliai: ", $"| {"Asmens
kodas",20} | {"Tema1",-15} |" +
                $" {"Tema2",-15} | {"Tema3",-15} | {"Tema4",-15} | ", 96);
            InOutUtils.WriteToFile(filePath, payouts, "Išmokėjimai: ", $"| {"Pavardė",20} |
{"Vardas",15} | {"Tema1",-15} | {"Tema2",-15} |" +
                    $" {"Tema3",-15} | {"Tema4",-15} | {"Premija už tema1",-30} | {"Premija
už tema2",-30} |" +
                    $" {"Premija už tema3",-30} | {"Premija už tema4",-30} | {"Suma",-30} |"
, 279);

            InOutUtils.WriteToFile(filePath, LessThanAverage, "Darbuotojai uždirbę mažiau
nei premijų vidurkis: ", $"| {"Pavardė",20} | {"Vardas",15} | {"Tema1",-15} | {"Tema2",-15}
|" +
                    $" {"Tema3",-15} | {"Tema4",-15} | {"Premija už tema1",-30} | {"Premija
už tema2",-30} |" +
                    $" {"Premija už tema3",-30} | {"Premija už tema4",-30} | {"Suma",-30}
|", 279);


        }

        protected void Pick_Click(object sender, EventArgs e)
        {
            string ParsedTheme = ThemeSelection.Text;

            if (ParsedTheme == "Tema 1" || ParsedTheme == "Tema 2" || ParsedTheme == "Tema
3" || ParsedTheme == "Tema 4")
            {
                ListClass<Payout> filteredByTheme = TaskUtils.FilterByTheme(ParsedTheme,
(ListClass<Payout>)Session["Payouts"]);
                AddDataToResultsTable(filteredByTheme, PickOut);
            }


        }
    }
                                            }
```

FormaMethods.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI.WebControls;

namespace Lab3
{
    public partial class Forma : System.Web.UI.Page
    {
        /// <summary>
```

```
/// Sets all headers for tables.
/// </summary>
/// <param name="headers">Headers in array</param>
/// <param name="table">Table to set to.</param>

public void AddHeader(string[] headers, Table table)
{
    TableRow headerRow = new TableRow();
    foreach (string header in headers)
    {
        TableCell cell = new TableCell();
        cell.Text = header;
        headerRow.Cells.Add(cell);
    }
    table.Rows.Add(headerRow);
}


/// <summary>
/// Adds bonus data to the table.
/// </summary>
/// <param name="bonuses">Linked list from which data is added from.</param>

public void AddDataToThemeTable(ListClass<double> bonuses)
{
    TableRow row = new TableRow();
    for (bonuses.Start(); bonuses.Exists(); bonuses.Next())
    {
        double curr = bonuses.Get();
        TableCell cell = new TableCell();
        cell.Text = curr.ToString();
        row.Cells.Add(cell);
    }
    PayoutsPerTheme.Rows.Add(row);
}

/// <summary>
/// Adds employee data to the table.
/// </summary>
/// <param name="workers">Linked list from which data is added from.</param>

public void AddDataFromFile1(ListClass<Employee> workers)
{
    for (workers.Start(); workers.Exists(); workers.Next())
    {
        TableRow row = new TableRow();
        Employee curr = workers.Get();

        TableCell idCell = new TableCell();
        idCell.Text = curr.Id.ToString();
        row.Cells.Add(idCell);

        TableCell lastNameCell = new TableCell();
        lastNameCell.Text = curr.LastName;
        row.Cells.Add(lastNameCell);

        TableCell firstNameCell = new TableCell();
        firstNameCell.Text = curr.Name;
        row.Cells.Add(firstNameCell);

        TableCell bankNameCell = new TableCell();
        bankNameCell.Text = curr.BankName;
        row.Cells.Add(bankNameCell);
```

```
            TableCell bankAccountCell = new TableCell();
            bankAccountCell.Text = curr.AccountNumber;
            row.Cells.Add(bankAccountCell);
            DataFile1.Rows.Add(row);
        }
    }

    /// <summary>
    /// Adds input data to the table.
    /// </summary>
    /// <param name="conts">Linked list from which data is added from.</param>

    public void AddDataFromFile2(ListClass<Input> conts)
    {
        for (conts.Start(); conts.Exists(); conts.Next())
        {
            TableRow row = new TableRow();

            Input curr = conts.Get();

            TableCell idCell = new TableCell();
            idCell.Text = curr.Id.ToString();
            row.Cells.Add(idCell);

            for (curr.StartCoef(); curr.ExistsCoef(); curr.NextCoef())
            {
                TableCell cell = new TableCell();
                cell.Text = curr.GetCoef().ToString();
                row.Cells.Add(cell);
            }
            DataFile2.Rows.Add(row);
        }
    }

    /// <summary>
    /// Adds data to Results table.
    /// </summary>
    /// <param name="payoutlist">Linked list from which data is added from.</param>

    public void AddDataToResultsTable(ListClass<Payout> payoutlist, Table table)
    {

        for (payoutlist.Start(); payoutlist.Exists(); payoutlist.Next())
        {
            TableRow row = new TableRow();
            Payout curr = payoutlist.Get();

            TableCell lastNameCell = new TableCell();
            lastNameCell.Text = curr.LastName;
            row.Cells.Add(lastNameCell);

            TableCell firstNameCell = new TableCell();
            firstNameCell.Text = curr.Name;
            row.Cells.Add(firstNameCell);

            for (curr.StartCoef(); curr.ExistsCoef(); curr.NextCoef())
            {
                TableCell tableCell = new TableCell();
                tableCell.Text = curr.GetCoef().ToString();
                row.Cells.Add(tableCell);
            }
            for (curr.StartAmount(); curr.ExistsAmount(); curr.NextAmount())
            {
                TableCell tableCell = new TableCell();
```

```
                    tableCell.Text = curr.GetAmount().ToString();
                    row.Cells.Add(tableCell);
                }
                TableCell SumCell = new TableCell();
                SumCell.Text = curr.Sum.ToString();
                row.Cells.Add(SumCell);
                table.Rows.Add(row);
            }

        }
    }
                                            }
```

UnitTest1.cs

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Lab3;
using System.Linq;

namespace Lab3Tests
{
    [TestClass]
    public class ListClassTestsEmployee
    {
        private Employee CreateEmployee(int id, string name)
        {
            return new Employee(id, name, "TestLastName", "TestBank", "LT123456789");
        }

        [TestMethod]
        public void Add_SingleEmployee_ShouldContainEmployee()
        {
            var list = new ListClass<Employee>();
            var employee = CreateEmployee(1, "Jonas");
            list.Add(employee);

            list.Start();
            Assert.IsTrue(list.Exists());
            Assert.AreEqual(employee, list.Get());
        }

        [TestMethod]
        public void Add_MultipleEmployees_ShouldContainEmployeesInOrder()
        {
            var list = new ListClass<Employee>();
            var employee1 = CreateEmployee(1, "Jonas");
            var employee2 = CreateEmployee(2, "Pranas");
            var employee3 = CreateEmployee(3, "Bomba");

            list.Add(employee1);
            list.Add(employee2);
            list.Add(employee3);

            list.Start();
            Assert.AreEqual(employee1, list.Get());
            list.Next();
            Assert.AreEqual(employee2, list.Get());
            list.Next();
            Assert.AreEqual(employee3, list.Get());
        }

        [TestMethod]
```

```
public void Start_ShouldPointToFirstEmployee()
{
    var list = new ListClass<Employee>();
    var employee = CreateEmployee(1, "Romas");
    list.Add(employee);

    list.Start();
    Assert.AreEqual(employee, list.Get());
}

[TestMethod]
public void Next_ShouldMoveToNextEmployee()
{
    var list = new ListClass<Employee>();
    var employee1 = CreateEmployee(1, "Tomas");
    var employee2 = CreateEmployee(2, "Andrius");

    list.Add(employee1);
    list.Add(employee2);

    list.Start();
    list.Next();
    Assert.AreEqual(employee2, list.Get());
}

[TestMethod]
public void Exists_ShouldReturnTrue_WhenEmployeeExists()
{
    var list = new ListClass<Employee>();
    var employee = CreateEmployee(1, "Jonas");
    list.Add(employee);

    list.Start();
    Assert.IsTrue(list.Exists());
}

[TestMethod]
public void Exists_ShouldReturnFalse_WhenListIsEmpty()
{
    var list = new ListClass<Employee>();

    list.Start();
    Assert.IsFalse(list.Exists());
}

[TestMethod]
public void Get_ShouldReturnCurrentEmployee()
{
    var list = new ListClass<Employee>();
    var employee = CreateEmployee(1, "Jonas");
    list.Add(employee);

    list.Start();
    var value = list.Get();
    Assert.AreEqual(employee, value);
}

[TestMethod]
public void Contains_ShouldReturnTrue_WhenEmployeeExists()
{
    var list = new ListClass<Employee>();
    var employee1 = CreateEmployee(1, "Jonas");
    var employee2 = CreateEmployee(2, "Pranas");
```

```
            list.Add(employee1);
            list.Add(employee2);

            Assert.IsTrue(list.Contains(employee2));
        }

        [TestMethod]
        public void Contains_ShouldReturnFalse_WhenEmployeeDoesNotExist()
        {
            var list = new ListClass<Employee>();
            var employee1 = CreateEmployee(1, "Jonas");
            var employee2 = CreateEmployee(2, "Pranas");

            list.Add(employee1);

            Assert.IsFalse(list.Contains(employee2));
        }

        [TestMethod]
        public void Sort_ShouldSortEmployeesByIdAscending()
        {
            var list = new ListClass<Employee>();
            var employee1 = CreateEmployee(3, "Bananas");
            var employee2 = CreateEmployee(1, "Jonas");
            var employee3 = CreateEmployee(2, "Pranas");

            list.Add(employee1);
            list.Add(employee2);
            list.Add(employee3);

            list.Sort();
            list.Start();

            Assert.AreEqual(employee2, list.Get());
            list.Next();
            Assert.AreEqual(employee3, list.Get());
            list.Next();
            Assert.AreEqual(employee1, list.Get());
        }

        [TestMethod]
        public void Enumerator_ShouldReturnAllEmployees()
        {
            var list = new ListClass<Employee>();
            var employee1 = CreateEmployee(1, "Jonas");
            var employee2 = CreateEmployee(2, "Pranas");
            var employee3 = CreateEmployee(3, "Bananas");

            list.Add(employee1);
            list.Add(employee2);
            list.Add(employee3);

            var elements = list.ToList();
            CollectionAssert.AreEqual(new[] { employee1, employee2, employee3 }, elements);
        }
    }
                                            }
```

UnitTest2.cs

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Lab3;
using System.Linq;
```

```
namespace Lab3Tests
{
    [TestClass]
    public class ListClassTestsInt
    {
        [TestMethod]
        public void Add_SingleInt_ShouldContainInt()
        {
            var list = new ListClass<int>();
            list.Add(1);

            list.Start();
            Assert.IsTrue(list.Exists());
            Assert.AreEqual(1, list.Get());
        }

        [TestMethod]
        public void Add_MultipleInts_ShouldContainIntsInOrder()
        {
            var list = new ListClass<int>();
            list.Add(1);
            list.Add(2);
            list.Add(3);

            list.Start();
            Assert.AreEqual(1, list.Get());
            list.Next();
            Assert.AreEqual(2, list.Get());
            list.Next();
            Assert.AreEqual(3, list.Get());
        }

        [TestMethod]
        public void Start_ShouldPointToFirstInt()
        {
            var list = new ListClass<int>();
            list.Add(5);

            list.Start();
            Assert.AreEqual(5, list.Get());
        }

        [TestMethod]
        public void Next_ShouldMoveToNextInt()
        {
            var list = new ListClass<int>();
            list.Add(5);
            list.Add(10);

            list.Start();
            list.Next();
            Assert.AreEqual(10, list.Get());
        }

        [TestMethod]
        public void Exists_ShouldReturnTrue_WhenIntExists()
        {
            var list = new ListClass<int>();
            list.Add(1);

            list.Start();
            Assert.IsTrue(list.Exists());
        }
```

```
[TestMethod]
public void Exists_ShouldReturnFalse_WhenListIsEmpty()
{
    var list = new ListClass<int>();

    list.Start();
    Assert.IsFalse(list.Exists());
}

[TestMethod]
public void Get_ShouldReturnCurrentInt()
{
    var list = new ListClass<int>();
    list.Add(99);

    list.Start();
    var value = list.Get();
    Assert.AreEqual(99, value);
}

[TestMethod]
public void Contains_ShouldReturnTrue_WhenIntExists()
{
    var list = new ListClass<int>();
    list.Add(1);
    list.Add(2);

    Assert.IsTrue(list.Contains(2));
}

[TestMethod]
public void Contains_ShouldReturnFalse_WhenIntDoesNotExist()
{
    var list = new ListClass<int>();
    list.Add(1);

    Assert.IsFalse(list.Contains(2));
}

[TestMethod]
public void Sort_ShouldSortIntsByAscending()
{
    var list = new ListClass<int>();
    list.Add(30);
    list.Add(10);
    list.Add(20);

    list.Sort();
    list.Start();

    Assert.AreEqual(10, list.Get());
    list.Next();
    Assert.AreEqual(20, list.Get());
    list.Next();
    Assert.AreEqual(30, list.Get());
}

[TestMethod]
public void Enumerator_ShouldReturnAllInts()
{
    var list = new ListClass<int>();
    list.Add(1);
    list.Add(2);
```

```
            list.Add(3);

            var elements = list.ToList();
            CollectionAssert.AreEqual(new[] { 1, 2, 3 }, elements);
        }
    }
                                                }
```

Styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f8f9fa;
    margin: 20px;
    padding: 0;
    text-align: center;
    background-color: aqua;
}

table {
    width: 80%;
    margin: 20px auto;
    border-collapse: collapse;
    background-color: #ffffff;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    border-radius: 8px;
    overflow: hidden;
}

th, td {
    border: 2px solid black;
    padding: 10px;
    text-align: center;
}

th {
    background-color: #007bff;
    color: white;
    font-weight: bold;
}


#DataFile1, #DataFile2, #PayoutsPerTheme, #Results, #PickOut, #LessAvg {
    margin: 20px auto;
    padding: 10px;
    border: 2px solid black;
    border-radius: 8px;
    background-color: #ffffff;
    width: 90%;
}

button {
    background-color: #007bff;
    color: white;
    border: none;
    padding: 10px 20px;
    margin: 10px;
    font-size: 16px;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #0056b3;
}
```

```css
.Summ {
    color: red;
    font-weight: bold;
    text-align: center;
    background-color: #f8d7da;
    padding: 15px;
    border-radius: 8px;
    margin: 20px auto;
    width: 80%;
    border: 1px solid red;
}

.val1, .val2 {
    color: red;
    font-weight: bold;
    font-size: 14px;
}
```

3.7. Pradiniai duomenys ir rezultatai

Duomenys Nr. 1

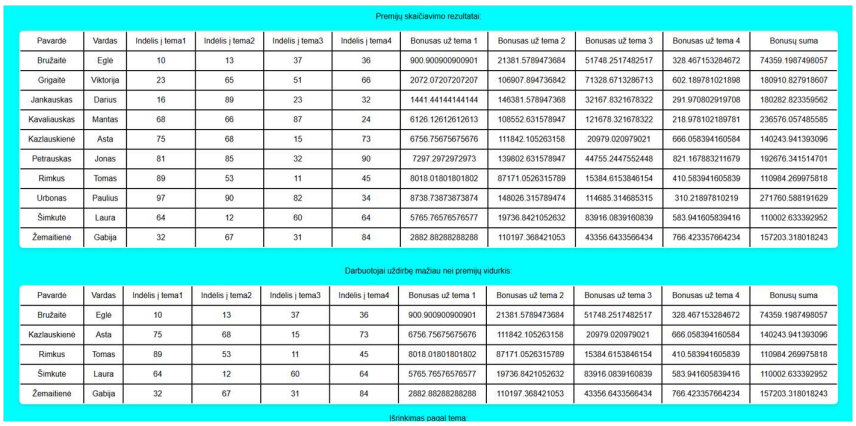
U6a.txt

```
 1;Petrauskas;Jonas;Swedbank;LT257044012345678901
 10;Kazlauskienė;Asta;SEB;LT137044000123456789
 3;Jankauskas;Darius;Luminor;LT427044012345678902
 8;Bružaitė;Eglė;Šiaulių bankas;LT737044012345678903
 5;Urbonas;Paulius;Citadele;LT217044012345678904
 6;Žemaitienė;Gabija;Swedbank;LT837044012345678905
 11;Kavaliauskas;Mantas;SEB;LT147044012345678906
 7;Šimkutė;Laura;Luminor;LT357044012345678907
 9;Rimkus;Tomas;Šiaulių bankas;LT567044012345678908
14;Grigaitė;Viktorija;Citadele;LT677044012345678909
```

U6b.txt

```
 50000;1000000;600000;5000
 1;81;85;32;90
 10;75;68;15;73
 3;16;89;23;32
 14;23;65;51;66
 5;97;90;82;34
 6;32;67;31;84
 11;68;66;87;24
 7;64;12;60;64
 9;89;53;11;45
8;10;13;37;36
```

Rezultatai



Premijų skaičiavimo rezultatai:

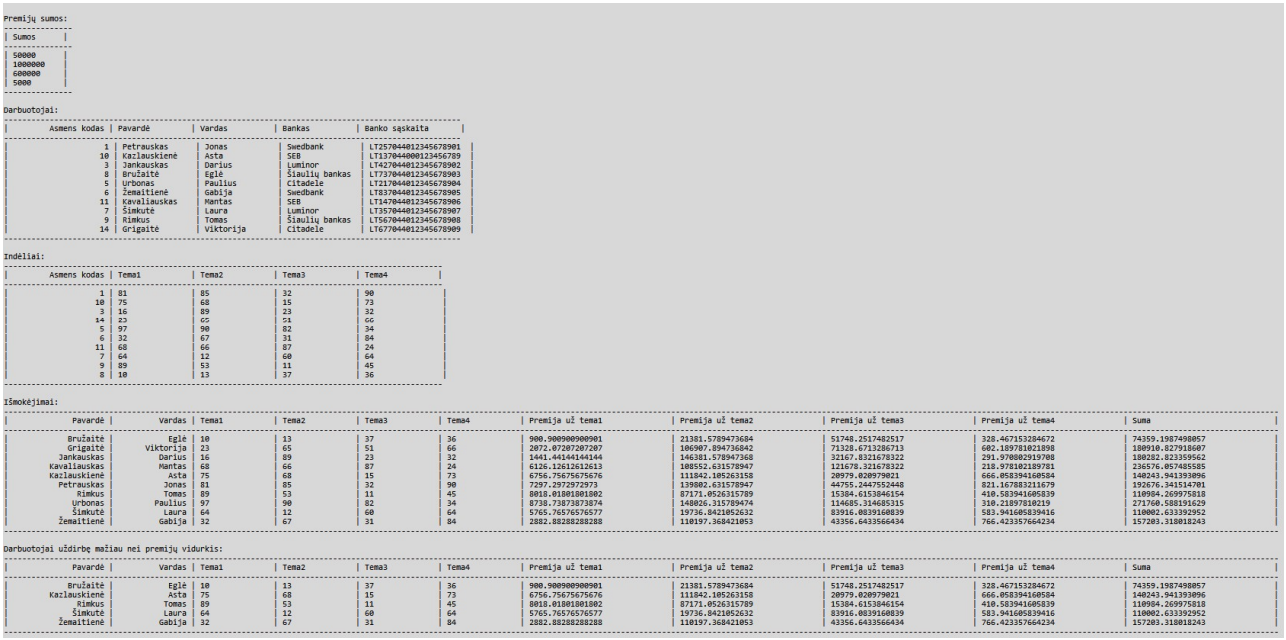
Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Bružaitė	Egle	10	13	37	36	900.900900900901	21381.5789473684	51748.2517482517	328.467153284672	74359.1987498057
Grigaitė	Viktorija	23	65	51	66	2072.07207207207	106907.894736842	71328.6713286713	602.189781021898	180910.827918607
Jankauskas	Darius	16	89	23	32	1441.44144144144	146381.578947368	32167.8321678322	291.970802919708	180282.823359562
Kavaliauskas	Mantas	68	66	87	24	6126.12612612613	108552.631578947	121678.321678322	218.978102189781	236576.057485585
Kazlauskienė	Asta	75	68	15	73	6756.75675675676	111842.105263158	20979.020979021	666.058394160584	140243.941393096
Petrauskas	Jonas	81	85	32	90	7297.2972972973	139802.631578947	44755.2447552448	821.167883211679	192676.341514701
Rimkus	Tomas	89	53	11	45	8018.01801801802	87171.0526315789	15384.6153846154	410.583941605839	110984.269975818
Urbonas	Paulius	97	90	82	34	8738.73873873874	148026.315789474	114685.314685315	310.21897810219	271760.588191629
Šimkutė	Laura	64	12	60	64	5765.76576576577	19736.8421052632	83916.0839160839	583.941605839416	110002.633392952
Žemaitienė	Gabija	32	67	31	84	2882.88288288288	110197.368421053	43356.6433566434	766.423357664234	157203.318018243

Darbuotojai uždirbę mažiau nei premijų vidurkis

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
Bružaitė	Egle	10	13	37	36	900.900900900901	21381.5789473684	51748.2517482517	328.467153284672	74359.1987498057
Kazlauskienė	Asta	75	68	15	73	6756.75675675676	111842.105263158	20979.020979021	666.058394160584	140243.941393096
Rimkus	Tomas	89	53	11	45	8018.01801801802	87171.0526315789	15384.6153846154	410.583941605839	110984.269975818
Šimkutė	Laura	64	12	60	64	5765.76576576577	19736.8421052632	83916.0839160839	583.941605839416	110002.633392952
Žemaitienė	Gabija	32	67	31	84	2882.88288288288	110197.368421053	43356.6433566434	766.423357664234	157203.318018243

Išrinkimas pagal tema

Pav.16 Rezultatai programoje



Premijų sumos:

Sumos
50000
1000000
600000
5000

Darbuotojai:

Asmens kodas	Pavardė	Vardas	Bankas	Banko sąskaita
1	Petrauskas	Jonas	Swedbank	LT257044012345678901
10	Kazlauskienė	Asta	SEB	LT137044000123456789
3	Jankauskas	Darius	Luminor	LT427044012345678902
8	Bružaitė	Eglė	Šiaulių bankas	LT737044012345678903
5	Urbonas	Paulius	Citadele	LT217044012345678904
6	Žemaitienė	Gabija	Swedbank	LT837044012345678905
11	Kavaliauskas	Mantas	SEB	LT147044012345678906
7	Šimkutė	Laura	Luminor	LT357044012345678907
9	Rimkus	Tomas	Šiaulių bankas	LT567044012345678908
14	Grigaitė	Viktorija	Citadele	LT677044012345678909

Indėliai:

Asmens kodas	Tema1	Tema2	Tema3	Tema4
1	81	85	32	90
10	75	68	15	73
3	16	89	23	32
14	23	GG	51	GG
5	97	90	82	34
6	32	67	31	84
11	68	66	87	24
7	64	12	60	64
9	89	53	11	45
8	10	13	37	36

Išmokėjimai:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Bružaitė	Eglė	10	13	37	36	900.900900900901	21381.5789473684	51748.2517482517	328.467153284672	74359.1987498057
Grigaitė	Viktorija	23	65	51	66	2072.07207207207	106907.894736842	71328.6713286713	602.189781021898	180910.827918607
Jankauskas	Darius	16	89	23	32	1441.44144144144	146381.578947368	32167.8321678322	291.970802919708	180282.823359562
Kavaliauskas	Mantas	68	66	87	24	6126.12612612613	108552.631578947	121678.321678322	218.978102189781	236576.057485585
Kazlauskienė	Asta	75	68	15	73	6756.75675675676	111842.105263158	20979.020979021	666.058394160584	140243.941393096
Petrauskas	Jonas	81	85	32	90	7297.2972972973	139802.631578947	44755.2447552448	821.167883211679	192676.341514701
Rimkus	Tomas	89	53	11	45	8018.01801801802	87171.0526315789	15384.6153846154	410.583941605839	110984.269975818
Urbonas	Paulius	97	90	82	34	8738.73873873874	148026.315789474	114685.314685315	310.21897810219	271760.588191629
Šimkutė	Laura	64	12	60	64	5765.76576576577	19736.8421052632	83916.0839160839	583.941605839416	110002.633392952
Žemaitienė	Gabija	32	67	31	84	2882.88288288288	110197.368421053	43356.6433566434	766.423357664234	157203.318018243

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Tema1	Tema2	Tema3	Tema4	Premija už tema1	Premija už tema2	Premija už tema3	Premija už tema4	Suma
Bružaitė	Eglė	10	13	37	36	900.900900900901	21381.5789473684	51748.2517482517	328.467153284672	74359.1987498057
Kazlauskienė	Asta	75	68	15	73	6756.75675675676	111842.105263158	20979.020979021	666.058394160584	140243.941393096
Rimkus	Tomas	89	53	11	45	8018.01801801802	87171.0526315789	15384.6153846154	410.583941605839	110984.269975818
Šimkutė	Laura	64	12	60	64	5765.76576576577	19736.8421052632	83916.0839160839	583.941605839416	110002.633392952
Žemaitienė	Gabija	32	67	31	84	2882.88288288288	110197.368421053	43356.6433566434	766.423357664234	157203.318018243

Pav.17 Rezultatai faile

Duomenys Nr. 2

U6a.txt

```
12;A;B;Swedbank;LT257044012345678901
34;A;C;SEB;LT137044000123456789
56;C;D;Luminor;LT427044012345678902
78;A;A;Šiaulių bankas;LT737044012345678903
```

U6b.txt

```
50000;1000000;600000;5000
12;1;1;1;1
34;2;2;2;2
56;3;3;3;3
78;4;4;4;4
```

Rezultatai

Premijų skaičiavimo rezultatai:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
A	A	4	4	4	4	20000	400000	240000	2000	662000
A	B	1	1	1	1	5000	100000	60000	500	165500
A	C	2	2	2	2	10000	200000	120000	1000	331000
C	D	3	3	3	3	15000	300000	180000	1500	496500

Darbuotojai uždirbę mažiau nei premijų vidurkis:

Pavardė	Vardas	Indėlis į tema1	Indėlis į tema2	Indėlis į tema3	Indėlis į tema4	Bonusas už tema 1	Bonusas už tema 2	Bonusas už tema 3	Bonusas už tema 4	Bonusų suma
A	B	1	1	1	1	5000	100000	60000	500	165500
A	C	2	2	2	2	10000	200000	120000	1000	331000

Pav.18 Rezultatai programoje

```
Premijų sumos:
-------------
| Sumos     |
-------------
| 50000     |
| 1000000   |
| 600000    |
| 5000      |
-------------

Darbuotojai:
-------------------------------------------------------------------------------------
|        Asmens kodas | Pavardė         | Vardas          | Bankas          | Banko sąskaita          |
-------------------------------------------------------------------------------------
|                  12 | A               | B               | Swedbank        | LT257044012345678901    |
|                  34 | A               | C               | SEB             | LT137044000123456789    |
|                  56 | C               | D               | Luminor         | LT427044012345678902    |
|                  78 | A               | A               | Šiaulių bankas  | LT737044012345678903    |
-------------------------------------------------------------------------------------

Indėliai:
-------------------------------------------------------------------------
|        Asmens kodas | Tema1           | Tema2           | Tema3           | Tema4           |
-------------------------------------------------------------------------
|                  12 | 1               | 1               | 1               | 1               |
|                  34 | 2               | 2               | 2               | 2               |
|                  56 | 3               | 3               | 3               | 3               |
|                  78 | 4               | 4               | 4               | 4               |
-------------------------------------------------------------------------

Išmokėjimai:
-------------------------------------------------------------------------------------------------------------------------------------------------------
|            Pavardė |           Vardas | Tema1           | Tema2           | Tema3           | Tema4           | Premija už tema1        | Premija už tema2        | Premija už tema3        | Premija už tema4        | Suma                    |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|                  A |                A | 4               | 4               | 4               | 4               | 20000                   | 400000                  | 240000                  | 2000                    | 662000                  |
|                  A |                B | 1               | 1               | 1               | 1               | 5000                    | 100000                  | 60000                   | 500                     | 165500                  |
|                  A |                C | 2               | 2               | 2               | 2               | 10000                   | 200000                  | 120000                  | 1000                    | 331000                  |
|                  C |                D | 3               | 3               | 3               | 3               | 15000                   | 300000                  | 180000                  | 1500                    | 496500                  |
-------------------------------------------------------------------------------------------------------------------------------------------------------

Darbuotojai uždirbę mažiau nei premijų vidurkis:
-------------------------------------------------------------------------------------------------------------------------------------------------------
|            Pavardė |           Vardas | Tema1           | Tema2           | Tema3           | Tema4           | Premija už tema1        | Premija už tema2        | Premija už tema3        | Premija už tema4        | Suma                    |
-------------------------------------------------------------------------------------------------------------------------------------------------------
|                  A |                B | 1               | 1               | 1               | 1               | 5000                    | 100000                  | 60000                   | 500                     | 165500                  |
|                  A |                C | 2               | 2               | 2               | 2               | 10000                   | 200000                  | 120000                  | 1000                    | 331000                  |
-------------------------------------------------------------------------------------------------------------------------------------------------------
```

Pav.19 Rezultatai faile

3.8. Dėstytojo pastabos

- Nėra išvedimo pagal temas į failą.
- SumCalc () metodas yra netinkamai paskelbtas

4. Polimorfizmas ir išimčių valdymas (L4)

4.1. Darbo užduotis

U4_6. Nekilnojamojo turto agentūra.
Turite ir kitų nekilnojamojo turto agentūrų (>=3) duomenis. Pirmoje eilutėje yra pavadinimas, antroje – adresas, trečioje – telefonas. Nekilnojamojo turto agentūra parduoda butus ir nuosavus namus. Sukurkite abstrakčiąją klasę „RealEstate“ (savybės – miestas, mikrorajonas, gatvė, namo numeris, tipas, pastatymo metai, plotas, kambarių skaičius), kurią paveldės klasės “Flat” (savybė - aukštas) ir “House” (savybė – šildymo būdas).
• Raskite, kurioje gatvėje daugiausiai parduodamų nekilnojamo turto objektų (namų ir butų), ekrane atspausdinkite gatvės pavadinimą ir parduodamų objektų kiekį.
• Raskite seniausią nekilnojamojo turto objektą, ekrane atspausdinkite visą jo informaciją.
• Raskite, kurių namų ar butų skelbimai yra paskelbti daugiau nei vienoje agentūroje (savininkas tikriausiai labai skuba parduoti, ir bus linkęs nuleisti kainą). Išrikiuokite juos pagal gatvės pavadinimą ir namo numerį. Į failą „Kartojasi.cvs“ įrašykite informaciją apie šiuos objektus.
• Sudarykite ir surikiuokite didelių NT objektų sąrašą. Namas yra didelis, jei jo plotas didesnis nei 200 kv.m. Namus rikiuokite pagal plotą ir šildymo būdą. Butas yra didelis, jei jo plotas didesnis nei 90 kv.m. Butus rikiuokite pagal plotą ir aukštą. Rezultatus įrašykite į failą „Dideli.csv“.

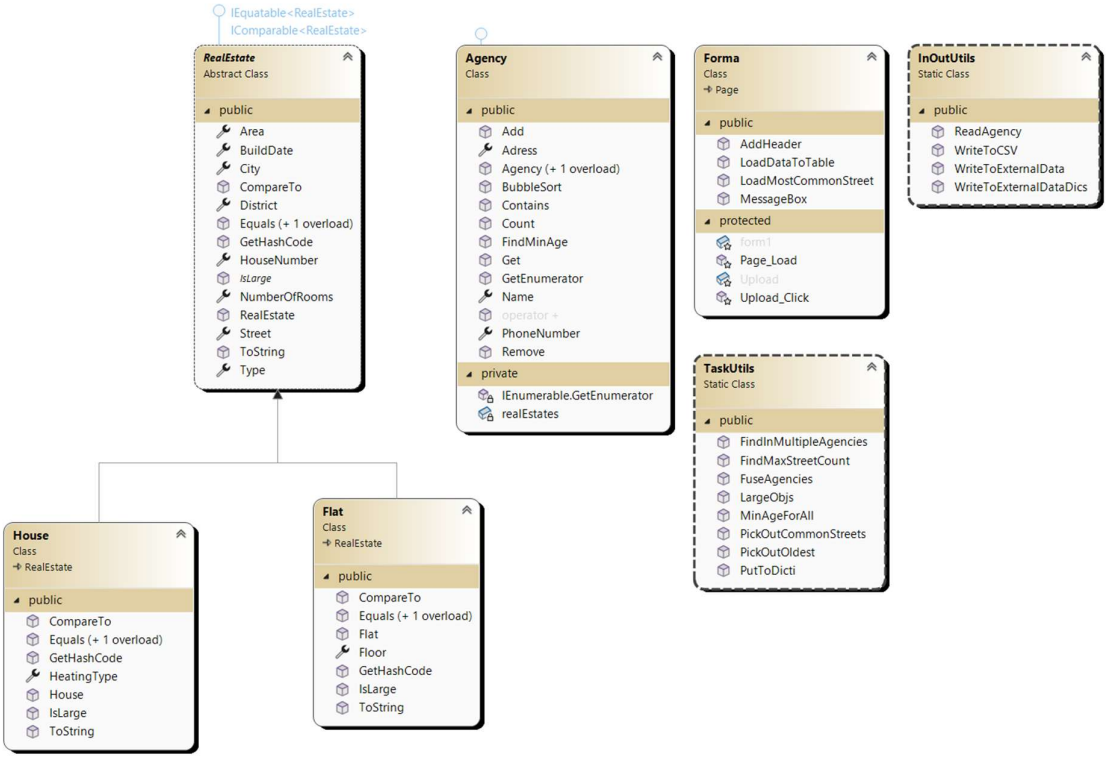
4.2. Grafinės naudotojo sąsajos schema

Pav.20 GUI schema

4.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė
Button1	Text	„Skaičiuoti“
	Width	80px

4.4. Klasių diagrama



Pav.21 Klasių diagrama

4.5. Programos naudotojo vadovas

Į programos AgencyData katalogą įdedami failai, kuriuose pirma eilutė yra agentūros pavadinimas, antra adresas, trečia telefono numeris. Paleidus programą duomenys automatiškai nuskaitomi iš katalogo paspaudus mygtuką „Skaičiuoti“. Kataloge AgencyData galima rasti ir sugeneruotus .csv failus. Projekto kataloge galima rasti visą apdorotą informaciją faile ExternalData.txt.

4.6. Programos tekstas

RealEstate.cs

```
using System;
using System.Collections.Generic;
using System.Diagnostics.PerformanceData;
using System.Drawing;
using System.Linq;
using System.Runtime.InteropServices;
using System.Web;

namespace L4
{
    /// <summary>
    /// Represents a real estate with basic details.
    /// </summary>
    public abstract class RealEstate : IEquatable<RealEstate>, IComparable<RealEstate>
    {
        /// <summary>
```

```
        /// The city of the real estate.
        /// </summary>
        public string City { get; set; }

        /// <summary>
        /// The district of the real estate.
        /// </summary>
        public string District { get; set; }

        /// <summary>
        /// The street of the real estate.
        /// </summary>
        public string Street { get; set; }

        /// <summary>
        /// The house number of the real estate.
        /// </summary>
        public int HouseNumber { get; set; }

        /// <summary>
        /// The type of the real estate (e.g., apartment, house).
        /// </summary>
        public string Type { get; set; }

        /// <summary>
        /// The year the real estate was built.
        /// </summary>
        public int BuildDate { get; set; }

        /// <summary>
        /// The size of the real estate in square meters.
        /// </summary>
        public double Area { get; set; }

        /// <summary>
        /// The number of rooms in the real estate.
        /// </summary>
        public int NumberOfRooms { get; set; }

        /// <summary>
        /// Creates a new real estate with the given details.
        /// </summary>
        public RealEstate(string city, string district, string street, int houseNumber,
string type, int buildDate, double area, int numberOfRooms)
        {
            City = city;
            District = district;
            Street = street;
            HouseNumber = houseNumber;
            Type = type;
            BuildDate = buildDate;
            Area = area;
            NumberOfRooms = numberOfRooms;
        }

        /// <summary>
        /// Checks if this real estate is the same as another based on location and size.
        /// </summary>
        public bool Equals(RealEstate other)
        {
            if (other == null) return false;
            if (GetType() != other.GetType()) return false;
```

```
            return Street == other.Street && HouseNumber == other.HouseNumber
                && City == other.City && District == other.District && Area == other.Area;
        }

        /// <summary>
        /// Checks if this object is the same as another object.
        /// </summary>
        public override bool Equals(object obj)
        {
            return Equals(obj as RealEstate);
        }

        /// <summary>
        /// Returns a unique code for the real estate based on its street and house number.
        /// </summary>
        public override int GetHashCode()
        {
            return Street.GetHashCode() ^ HouseNumber.GetHashCode();
        }

        /// <summary>
        /// Compares this real estate to another based on street and house number.
        /// </summary>
        public int CompareTo(RealEstate other)
        {
            if (Street.CompareTo(other.Street) == 0)
            {
                return HouseNumber.CompareTo(other.HouseNumber);
            }
            return Street.CompareTo(other.Street);
        }

        /// <summary>
        /// Returns a string with all the details of the real estate.
        /// </summary>
        public override string ToString()
        {
            return
$"{City},{District},{Street},{HouseNumber},{Type},{BuildDate},{Area},{NumberOfRooms}";
        }

        /// <summary>
        /// Checks if the real estate is large based on specific rules.
        /// </summary>
        public abstract bool IsLarge();
    }
                                                    }
```

House.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace L4
{
    /// <summary>
    /// Represents a house, which is a type of property.
    /// </summary>
```

```
    public class House : RealEstate
    {
        /// <summary>
        /// The heating system used in the house.
        /// </summary>
        public string HeatingType { get; set; }

        /// <summary>
        /// Creates a new house with the given details.
        /// </summary>
        /// <param name="city">The city where the house is located.</param>
        /// <param name="district">The district where the house is located.</param>
        /// <param name="street">The street where the house is located.</param>
        /// <param name="houseNumber">The house number.</param>
        /// <param name="type">The type of the house.</param>
        /// <param name="buildDate">The year the house was built.</param>
        /// <param name="area">The size of the house in square meters.</param>
        /// <param name="numberOfRooms">The number of rooms in the house.</param>
        /// <param name="HeatingType">The heating system used in the house.</param>
        public House(string city, string district, string street, int houseNumber, string
type, int buildDate, double area, int numberOfRooms, string HeatingType) :
            base(city, district, street, houseNumber, type, buildDate, area, numberOfRooms)
        {
            this.HeatingType = HeatingType;
        }

        /// <summary>
        /// Checks if another object is the same as this house.
        /// </summary>
        /// <param name="other">The object to compare with.</param>
        /// <returns>True if they are the same, false otherwise.</returns>
        public override bool Equals(object other)
        {
            return this.Equals(other as House);
        }

        /// <summary>
        /// Checks if another house is the same as this one.
        /// </summary>
        /// <param name="house">The house to compare with.</param>
        /// <returns>True if they are the same, false otherwise.</returns>
        public bool Equals(House house)
        {
            if (house == null)
            {
                return false;
            }
            return base.Equals(house);
        }

        /// <summary>
        /// Compares this house to another house by size and heating system.
        /// </summary>
        /// <param name="house">The house to compare with.</param>
        /// <returns>A negative number if this house is smaller, positive if larger, or 0 if
they are the same size.</returns>
        public int CompareTo(House house)
        {
            if (this.Area.CompareTo(house.Area) == 0)
            {
                return this.HeatingType.CompareTo(house.HeatingType);
            }
```

```
            return this.Area.CompareTo(house.Area);
        }

        /// <summary>
        /// Gets a unique code for this house.
        /// </summary>
        /// <returns>A hash code for the house.</returns>
        public override int GetHashCode()
        {
            return base.GetHashCode();
        }

        /// <summary>
        /// Checks if the house is big (over 200 square meters).
        /// </summary>
        /// <returns>True if the house is big, false otherwise.</returns>
        public override bool IsLarge()
        {
            return this.Area > 200;
        }

        /// <summary>
        /// Gets a text description of the house.
        /// </summary>
        /// <returns>A string with details about the house.</returns>
        public override string ToString()
        {
            return base.ToString() + $",{this.HeatingType},{"-"}";
        }
    }
```

}

Flat.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Web;
using System.Web.Routing;

namespace L4
{
    /// <summary>
    /// A Flat is a type of RealEstate.
    /// </summary>
    public class Flat : RealEstate
    {
        /// <summary>
        /// The floor number of the flat.
        /// </summary>
        public int Floor { get; set; }

        /// <summary>
        /// Creates a new Flat with the given details.
        /// </summary>
        /// <param name="city">The city of the flat.</param>
        /// <param name="district">The district of the flat.</param>
        /// <param name="street">The street of the flat.</param>
        /// <param name="houseNumber">The house number of the flat.</param>
        /// <param name="type">The type of the flat.</param>
        /// <param name="buildDate">The year the flat was built.</param>
```

```
        /// <param name="area">The size of the flat in square meters.</param>
        /// <param name="numberOfRooms">The number of rooms in the flat.</param>
        /// <param name="Floor">The floor number of the flat.</param>
        public Flat(string city, string district, string street, int houseNumber, string
type, int buildDate, double area, int numberOfRooms, int Floor) :
            base(city, district, street, houseNumber, type, buildDate, area, numberOfRooms)
        {
            this.Floor = Floor;
        }

        /// <summary>
        /// Checks if another object is the same as this Flat.
        /// </summary>
        /// <param name="other">The object to compare with.</param>
        /// <returns>True if they are the same, otherwise false.</returns>
        public override bool Equals(object other)
        {
            return this.Equals(other as Flat);
        }

        /// <summary>
        /// Checks if another Flat is the same as this Flat.
        /// </summary>
        /// <param name="flat">The Flat to compare with.</param>
        /// <returns>True if they are the same, otherwise false.</returns>
        public bool Equals(Flat flat)
        {
            if (flat == null)
            {
                return false;
            }
            return base.Equals(flat);
        }

        /// <summary>
        /// Compares this Flat with another Flat
        /// </summary>
        /// <param name="flat">The Flat to compare with.</param>
        /// <returns>
        /// A negative number if this Flat is smaller, 0 if they are the same, or a positive
number if this Flat is larger.
        /// </returns>
        public int CompareTo(Flat flat)
        {
            if (this.Area.CompareTo(flat.Area) == 0)
            {
                return this.Floor.CompareTo(flat.Floor);
            }
            return this.Area.CompareTo(flat.Area);
        }

        /// <summary>
        /// Gets a unique number for this Flat.
        /// </summary>
        /// <returns>A unique number for this Flat.</returns>
        public override int GetHashCode()
        {
            return base.GetHashCode();
        }

        /// <summary>
        /// Checks if the flat is large.
```

```
        /// </summary>
        /// <returns>True if the flat is large otherwise false</returns>
        public override bool IsLarge()
        {
            return this.Area > 90;
        }

        /// <summary>
        /// Gets a text description of the Flat
        /// </summary>
        /// <returns>A text description of the Flat.</returns>
        public override string ToString()
        {
            return base.ToString() + $",{"-"},{this.Floor}";
        }
    }
```

}

Agency.cs

```
using System;
using System.CodeDom;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Web;
using System.Xml;

namespace L4
{
    /// <summary>
    /// A real estate agency.
    /// </summary>
    public class Agency : IEnumerable<RealEstate>
    {
        public string Name { get; private set; }
        public string Adress { get; private set; }
        public int PhoneNumber { get; private set; }

        private List<RealEstate> realEstates;

        /// <summary>
        /// Creates an agency with name, address, and phone number.
        /// </summary>
        /// <param name="name">Agency name.</param>
        /// <param name="adress">Agency address.</param>
        /// <param name="phoneNumber">Agency phone number.</param>
        public Agency(string name, string adress, int phoneNumber)
        {
            this.Name = name;
            this.Adress = adress;
            this.PhoneNumber = phoneNumber;
            this.realEstates = new List<RealEstate>();
        }
        /// <summary>
        /// Creates an empty agency.
        /// </summary>
        public Agency()
        {
            this.realEstates = new List<RealEstate>();
        }
```

```
/// <summary>
/// Adds a real estate to the agency.
/// </summary>
/// <param name="re">Real estate to add.</param>
public void Add(RealEstate re)
{
    realEstates.Add(re);
}
/// <summary>
/// Removes a real estate from the agency.
/// </summary>
/// <param name="re">Real estate to remove.</param>
public void Remove(RealEstate re)
{
    realEstates.Remove(re);
}
/// <summary>
/// Gets a real estate by its position in the list.
/// </summary>
/// <param name="index">Position in the list.</param>
/// <returns>The real estate at the position.</returns>
public RealEstate Get(int index)
{
    return realEstates[index];
}
/// <summary>
/// Counts the real estates in the agency.
/// </summary>
/// <returns>Number of real estates.</returns>
public int Count()
{
    return realEstates.Count;
}

/// <summary>
/// Checks if the agency has a specific real estate.
/// </summary>
/// <param name="estate">Real estate to check.</param>
/// <returns>True if found, otherwise false.</returns>
public bool Contains(RealEstate estate)
{
    foreach (RealEstate curr in realEstates)
    {
        if (curr.Equals(estate))
        {
            return true;
        }
    }
    return false;
}

/// <summary>
/// Finds the oldest real estate in the agency.
/// </summary>
/// <returns>Year of the oldest real estate.</returns>
public int FindMinAge()
{
    int minAge = 9999;
    foreach (RealEstate estate in realEstates)
    {
        if (estate.BuildDate < minAge)
        {
```

```
                minAge = estate.BuildDate;
            }
        }
        return minAge;
    }
    /// <summary>
    /// Combines two agencies into one.
    /// </summary>
    /// <param name="a1">First agency.</param>
    /// <param name="a2">Second agency.</param>
    /// <returns>New agency with all real estates.</returns>
    public static Agency operator +(Agency a1, Agency a2)
    {
        Agency newAgency = new Agency();
        foreach (RealEstate estate in a1.realEstates)
        {
            newAgency.Add(estate);
        }
        foreach (RealEstate estate in a2.realEstates)
        {
            newAgency.Add(estate);
        }
        return newAgency;
    }
    /// <summary>
    /// Sorts the real estates in the agency.
    /// </summary>
    public void BubbleSort()
    {
        int n = realEstates.Count();
        bool swapped;

        for (int i = 0; i < n - 1; i++)
        {
            swapped = false;

            for (int j = 0; j < n - i - 1; j++)
            {
                if (realEstates[i].CompareTo(realEstates[j]) > 0)
                {
                    RealEstate temp = realEstates[j];
                    realEstates[j] = realEstates[j + 1];
                    realEstates[j + 1] = temp;

                    swapped = true;
                }
            }
            if (!swapped)
                break;
        }
    }

    /// <summary>
    /// Allows looping through the real estates.
    /// </summary>
    /// <returns>Enumerator for real estates.</returns>
    public IEnumerator<RealEstate> GetEnumerator()
    {
        foreach (var realEstate in realEstates)
        {
            yield return realEstate;
        }
```

```
        }
        /// <summary>
        /// Allows looping through the real estates.
        /// </summary>
        /// <returns>Enumerator for real estates.</returns>
        IEnumerator IEnumerable.GetEnumerator()
        {
            return GetEnumerator();
        }

    }
}
```

TaskUtils.cs

```
using System.Collections.Generic;

namespace L4
{
    public static class TaskUtils
    {
        /// <summary>
        /// Finds all large real estates from the given agencies.
        /// </summary>
        /// <param name="agencies">Array of agencies to search in.</param>
        /// <returns>Agency containing only large real estates.</returns>
        public static Agency LargeObjs(Agency[] agencies)
        {
            Agency bigOnes = new Agency();
            foreach (Agency agency in agencies)
            {
                foreach (RealEstate estate in agency)
                {
                    if (estate.IsLarge())
                    {
                        bigOnes.Add(estate);
                    }
                }
            }
            return bigOnes;
        }

        /// <summary>
        /// Combines all agencies into one.
        /// </summary>
        /// <param name="agencies">Array of agencies to combine.</param>
        /// <returns>A single agency containing all real estates.</returns>
        public static Agency FuseAgencies(Agency[] agencies)
        {
            Agency Fused = new Agency();
            foreach (Agency agency in agencies)
            {
                Fused += agency;
            }
            return Fused;
        }

        /// <summary>
        /// Finds real estates that appear in multiple agencies.
        /// </summary>
        /// <param name="fused">An agency containing all real estates.</param>
        /// <returns>Agency with real estates found in multiple agencies.</returns>
        public static Agency FindInMultipleAgencies(Agency fused)
```

```
{
    Agency result = new Agency();

    for (int i = 0; i < fused.Count(); i++)
    {
        RealEstate current = fused.Get(i);
        int count = 0;

        for (int j = 0; j < fused.Count(); j++)
        {
            if (current.Equals(fused.Get(j)))
            {
                count++;
                if (count > 1) break;
            }
        }

        if (count > 1 && !result.Contains(current))
        {
            result.Add(current);
        }
    }

    return result;
}

/// <summary>
/// Picks out real estates built in a specific year.
/// </summary>
/// <param name="agencies">Array of agencies to search in.</param>
/// <param name="oldestValue">Year to filter by.</param>
/// <returns>Agency with real estates built in the given year.</returns>
public static Agency PickOutOldest(Agency[] agencies, int oldestValue)
{
    Agency oldestRealEstates = new Agency();
    foreach (Agency a in agencies)
    {
        foreach (RealEstate estate in a)
        {
            if (estate.BuildDate == oldestValue)
            {
                oldestRealEstates.Add(estate);
            }
        }
    }
    return oldestRealEstates;
}

/// <summary>
/// Finds the minimum age of real estates across all agencies.
/// </summary>
/// <param name="agencies">Array of agencies to search in.</param>
/// <returns>The minimum age of real estates.</returns>
public static int MinAgeForAll(Agency[] agencies)
{
    int minAge = 9999;
    foreach (Agency agency in agencies)
    {
        if (minAge > agency.FindMinAge())
        {
            minAge = agency.FindMinAge();
        }
```

```
            }
            return minAge;
        }

        /// <summary>
        /// Finds the maximum count of real estates on a single street.
        /// </summary>
        /// <param name="streetCounts">Dictionary with street names and their
counts.</param>
        /// <returns>The maximum count of real estates on a street.</returns>
        public static int FindMaxStreetCount(Dictionary<string, int> streetCounts)
        {
            int max = 0;
            foreach (var pair in streetCounts)
            {
                if (pair.Value > max)
                {
                    max = pair.Value;
                }
            }
            return max;
        }

        /// <summary>
        /// Creates a dictionary of street names and their real estate counts.
        /// </summary>
        /// <param name="agencies">Array of agencies to process.</param>
        /// <returns>Dictionary with street names and counts.</returns>
        public static Dictionary<string, int> PutToDicti(Agency[] agencies)
        {
            Dictionary<string, int> result = new Dictionary<string, int>();
            foreach (Agency agency in agencies)
            {
                foreach (RealEstate estate in agency)
                {
                    if (result.ContainsKey(estate.Street))
                    {
                        result[estate.Street]++;
                    }
                    else
                    {
                        result.Add(estate.Street, 1);
                    }
                }
            }
            return result;
        }

        /// <summary>
        /// Finds streets with the maximum number of real estates.
        /// </summary>
        /// <param name="values">Dictionary of street names and counts.</param>
        /// <param name="MaxValue">The maximum count to filter by.</param>
        /// <returns>Dictionary with streets having the maximum count.</returns>
        public static Dictionary<string, int> PickOutCommonStreets(Dictionary<string, int>
values, int MaxValue)
        {
            Dictionary<string, int> result = new Dictionary<string, int>();
            foreach (var pair in values)
            {
                if (pair.Value == MaxValue)
                {
```

```
                    result.Add(pair.Key, pair.Value);
                }
            }
            return result;
        }
    }
                                                    }
```

InOutUtils.cs

```
using Microsoft.SqlServer.Server;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Web;
using System.Xml;
using System.Xml.Linq;

namespace L4
{
    public static class InOutUtils
    {
        /// <summary>
        /// Reads data about an agency from a file and creates an Agency object.
        /// </summary>
        /// <param name="fileName">The file to read data from.</param>
        /// <returns>An Agency object with the data from the file.</returns>
        public static Agency ReadAgency(string fileName)
        {
            Agency agency;

            using (StreamReader reader = new StreamReader(fileName))
            {
                string name = reader.ReadLine();
                string address = reader.ReadLine();
                int phoneNumber;

                phoneNumber = int.Parse(reader.ReadLine());

                agency = new Agency(name, address, phoneNumber);

                string line;
                int lineNumber = 4;

                while ((line = reader.ReadLine()) != null)
                {
                    lineNumber++;
                    string[] values = line.Split(';');
                    try
                    {
                        string city = values[0];
                        string district = values[1];
                        string street = values[2];
                        int houseNumber = int.Parse(values[3]);
                        string type = values[4];
                        int buildYear = int.Parse(values[5]);
                        double area = double.Parse(values[6]);
                        int numberOfRooms = int.Parse(values[7]);
```

```
                        string diff = values[8];

                        RealEstate re;

                        if (int.TryParse(diff, out int floor))
                        {
                            re = new Flat(city, district, street, houseNumber, type,
buildYear, area, numberOfRooms, floor);
                        }
                        else
                        {
                            re = new House(city, district, street, houseNumber, type,
buildYear, area, numberOfRooms, diff);
                        }

                        agency.Add(re);
                    }
                    catch (Exception)
                    {
                        HttpContext.Current.Response.Write(
                         $"<script>alert('[{fileName}, line {lineNumber}: {line.Replace("'",
"\\'")}');</script>");
                        continue;
                    }
                }
            }

            return agency;
        }

        /// <summary>
        /// Saves agency data to a CSV file.
        /// </summary>
        /// <param name="a">The agency to save.</param>
        /// <param name="fileName">The file to save data to.</param>
        /// <param name="header">The header text for the CSV file.</param>
        public static void WriteToCSV(Agency a, string fileName, string header)
        {
            using (StreamWriter writer = new StreamWriter(fileName, false, Encoding.UTF8))
            {
                writer.WriteLine(header);
                writer.WriteLine($"{"City"},{"District"},{"Street"}," +
                    $"{"House Number"},{"Type"},{"Build Year"},{"Area"}" +
                    $",{"Number of Rooms"},{"Heating Type"},{"Floor"}");
                foreach (RealEstate estate in a)
                {
                    writer.WriteLine(estate.ToString());
                }
            }
        }

        /// <summary>
        /// Saves a dictionary of the most common streets to a file.
        /// </summary>
        /// <param name="fileName">The file to save data to.</param>
        /// <param name="mostCommonStreets">The dictionary of streets and their
counts.</param>
        /// <param name="header">The header text for the file.</param>
        public static void WriteToExternalDataDics(string fileName, Dictionary<string, int>
mostCommonStreets, string header)
        {
            using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
```

```
            {
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 37));
                writer.WriteLine($"| {"Street",15} | {"Count",15} |");
                writer.WriteLine(new string('-', 37));
                foreach (var pair in mostCommonStreets)
                {
                    writer.WriteLine($"| {pair.Key,15} | {pair.Value,15} |");
                }
                writer.WriteLine(new string('-', 37));
            }
        }

        /// <summary>
        /// Saves detailed agency data to a file.
        /// </summary>
        /// <param name="fileName">The file to save data to.</param>
        /// <param name="agency">The agency to save.</param>
        /// <param name="header">The header text for the file.</param>
        public static void WriteToExternalData(string fileName, Agency agency, string
header)
        {
            using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
            {
                writer.WriteLine(header);
                writer.WriteLine($"{"Agency Name: "}{agency.Name}");
                writer.WriteLine($"{"Agency Address: "}{agency.Adress}");
                writer.WriteLine($"{"Agency Phone Number: "}{agency.PhoneNumber}");
                writer.WriteLine(new string('-', 191));
                writer.WriteLine($"| {"City",15} | {"District",15} | {"Street",15} |" +
                    $" {"House Number",-15} | {"Type",-15} | {"Build Year",15} |" +
                    $" {"Area",15} | {"Number of Rooms",-20} | {"Heating Type",20} |
{"Floor",-15} |");
                writer.WriteLine(new string('-', 191));
                foreach (RealEstate estate in agency)
                {
                    writer.WriteLine($"| {estate.City,15} | {estate.District,15} |" +
                        $" {estate.Street,15} | {estate.HouseNumber,-15} | {estate.Type,15}
|" +
                        $" {estate.BuildDate,-15} | {estate.Area,-15} |
{estate.NumberOfRooms,-20} |" +
                        $" {(estate is House ? ((House)estate).HeatingType : "-"),20} |" +
                        $" {(estate is Flat ? ((Flat)estate).Floor.ToString() : "-"),-15}
|");
                }
                writer.WriteLine(new string('-', 191));
            }
        }
    }


}
```

Styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    color: #333;
    margin: 0;
    padding: 20px;
}
```

```css
form {
    background-color: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    text-align: center;
}

h1 {
    text-align: center;
    color: #2c3e50;
    margin-bottom: 30px;
}

form {
    width: 95%;
    max-width: 1400px;
    margin: auto;
    background-color: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 0 15px rgba(0, 0, 0, 0.15);
    text-align: center;
    box-sizing: border-box;
    overflow-x: auto;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
    margin-bottom: 30px;
}

table, th, td {
    border: 1px solid #ccc;
}

th, td {
    padding: 8px;
    text-align: center;
}

caption {
    font-weight: bold;
    margin-bottom: 10px;
}
```

FormaMethods.cs

```csharp
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;

namespace L4
{
    public partial class Forma : System.Web.UI.Page
    {
        /// <summary>
```

```
        /// Loads data about an agency and its real estates into a table.
        /// </summary>
        /// <param name="agency">The agency containing real estate data.</param>
        /// <param name="form">The form where the table will be added.</param>
        /// <param name="header">The header text for the table.</param>
        public void LoadDataToTable(Agency agency, HtmlForm form, string header)
        {
            try
            {
                Table table = new Table();
                TableRow headerRow = new TableRow();
                headerRow.Cells.Add(new TableCell() { Text = string.Format(header),
ColumnSpan = 10 });
                table.Rows.Add(headerRow);
                if (agency.Name != string.Empty && agency.Adress != string.Empty &&
agency.PhoneNumber != 0)
                {
                    TableRow hRow0 = new TableRow();
                    hRow0.Cells.Add(new TableCell() { Text = $"Pavadinimas: {agency.Name}",
ColumnSpan = 10 });
                    TableRow hRow1 = new TableRow();
                    hRow1.Cells.Add(new TableCell() { Text = $"Adresas: {agency.Adress}",
ColumnSpan = 10 });
                    TableRow hRow2 = new TableRow();
                    hRow2.Cells.Add(new TableCell() { Text = $"Telefono numeris:
{agency.PhoneNumber}", ColumnSpan = 10 });
                    table.Rows.Add(hRow0);
                    table.Rows.Add(hRow1);
                    table.Rows.Add(hRow2);
                }
                AddHeader(new string[] {
                    "Miestas", "Rajonas", "Gatvė", "Namo numeris", "Tipas",
                    "Pastatymo data", "Plotas", "Kambarių skaičius", "Aukštas", "Šildymo
tipas" }, table);
                foreach (RealEstate re in agency)
                {
                    TableRow row = new TableRow();
                    row.Cells.Add(new TableCell() { Text = re.City });
                    row.Cells.Add(new TableCell() { Text = re.District });
                    row.Cells.Add(new TableCell() { Text = re.Street });
                    row.Cells.Add(new TableCell() { Text = re.HouseNumber.ToString() });
                    row.Cells.Add(new TableCell() { Text = re.Type });
                    row.Cells.Add(new TableCell() { Text = re.BuildDate.ToString() });
                    row.Cells.Add(new TableCell() { Text = re.Area.ToString() });
                    row.Cells.Add(new TableCell() { Text = re.NumberOfRooms.ToString() });
                    if (re is Flat flat)
                    {
                        row.Cells.Add(new TableCell() { Text = flat.Floor.ToString() });
                        row.Cells.Add(new TableCell() { Text = "-" });
                    }
                    else if (re is House house)
                    {
                        row.Cells.Add(new TableCell() { Text = "-" });
                        row.Cells.Add(new TableCell() { Text = house.HeatingType });
                    }
                    table.Rows.Add(row);
                }
                form.Controls.Add(table);
            }
            catch (Exception ex)
            {
                throw new Exception("Klaida įkeliant duomenis į lentelę: " + ex.Message);
```

```
            }
        }

        /// <summary>
        /// Loads the most common streets and their counts into a table.
        /// </summary>
        /// <param name="MostCommonStreetEstates">Dictionary of street names and their
counts.</param>
        /// <param name="form">The form where the table will be added.</param>
        public void LoadMostCommonStreet(Dictionary<string, int> MostCommonStreetEstates,
HtmlForm form)
        {
            Table table = new Table();
            TableRow hRow0 = new TableRow();
            hRow0.Cells.Add(new TableCell() { Text = "Dažniausiai pasikartojančios gatvės",
ColumnSpan = 10 });
            table.Rows.Add(hRow0);
            AddHeader(new string[] { "Gatvės pavadinimas", "Kiekis" }, table);
            foreach (var pair in MostCommonStreetEstates)
            {
                TableRow row = new TableRow();
                row.Cells.Add(new TableCell() { Text = pair.Key });
                row.Cells.Add(new TableCell() { Text = pair.Value.ToString() });
                table.Rows.Add(row);
            }
            form.Controls.Add(table);
        }

        /// <summary>
        /// Adds a header row to a table.
        /// </summary>
        /// <param name="headers">Array of header names.</param>
        /// <param name="table">The table where the header will be added.</param>
        public void AddHeader(string[] headers, Table table)
        {
            TableRow headerRow = new TableRow();
            foreach (string header in headers)
            {
                TableCell cell = new TableCell();
                cell.Text = header;
                headerRow.Cells.Add(cell);
            }
            table.Rows.Add(headerRow);
        }

        /// <summary>
        /// Displays a message box with a given message.
        /// </summary>
        /// <param name="page">The page where the message box will be shown.</param>
        /// <param name="strMsg">The message to display.</param>
        public static void MessageBox(System.Web.UI.Page page, string strMsg)
        {
            ScriptManager.RegisterClientScriptBlock(page, page.GetType(), "alertMessage",
"alert('" + strMsg + "')", true);
        }
    }
```

}

Forma.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Forma.aspx.cs" Inherits="L4.Forma"
%>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <link href="Styles.css" rel="stylesheet" type="text/css" />
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <h1>Nekilnojamojo turto agentūra</h1>
            <asp:Button ID="Upload" runat="server" Text="Skaičiuoti" Width="80px"
OnClick="Upload_Click" />
            <br />
        </div>
    </form>
</body>
</html>
```

Forma.aspx.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace L4
{
    public partial class Forma : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            try
            {
                if (File.Exists(Server.MapPath("~/ExternalData.txt")))
                {
                    File.Delete(Server.MapPath("~/ExternalData.txt"));
                }
            }
            catch (Exception ex)
            {
                MessageBox(this, $"Klaida ištrinant failą: {ex.Message}");
            }
        }

        protected void Upload_Click(object sender, EventArgs e)
        {
            string[] filePaths;
            string folderPath = "~/AgencyData";
            int count = 0;

            try
            {
                filePaths = Directory.GetFiles(Server.MapPath(folderPath), "*.txt");
            }
            catch (Exception ex)
```

```
            {
                MessageBox(this, $"Nepavyko gauti failų iš aplanko: {folderPath}. Klaida:
{ex.Message}");
                return;
            }

            Agency[] agencies = new Agency[filePaths.Length];
            Agency data = null;

            foreach (string path in filePaths)
            {
                try
                {
                    data = InOutUtils.ReadAgency(path);
                }
                catch (Exception ex)
                {
                    MessageBox(this, $"Klaida skaitant failą {Path.GetFileName(path)}:
{ex.Message}");
                    continue;
                }

                try
                {
                    LoadDataToTable(data, form1, $"Duomenys iš failo:
{Path.GetFileName(path)}");
                }
                catch (Exception ex)
                {
                    MessageBox(this, $"Klaida įkeliant duomenis į lentelę: {ex.Message}");
                }

                try
                {
                    agencies[count] = data;
                    count++;
                }
                catch (Exception ex)
                {
                    MessageBox(this, $"Klaida pridedant agentūrą į masyvą: {ex.Message}");
                    continue;
                }

                try
                {
                    InOutUtils.WriteToExternalData(Server.MapPath("~/ExternalData.txt"),
data, "Nuskaityti duomenys:");
                }
                catch (Exception ex)
                {
                    MessageBox(this, $"Nepavyko įrašyti duomenų iš failo
{Path.GetFileName(path)} į išorinį failą. Klaida: {ex.Message}");
                    continue;
                }
            }

            try
            {
                Dictionary<string, int> mostCommonStreetEstates =
TaskUtils.PutToDicti(agencies);
                int MaxCount = TaskUtils.FindMaxStreetCount(mostCommonStreetEstates);
```

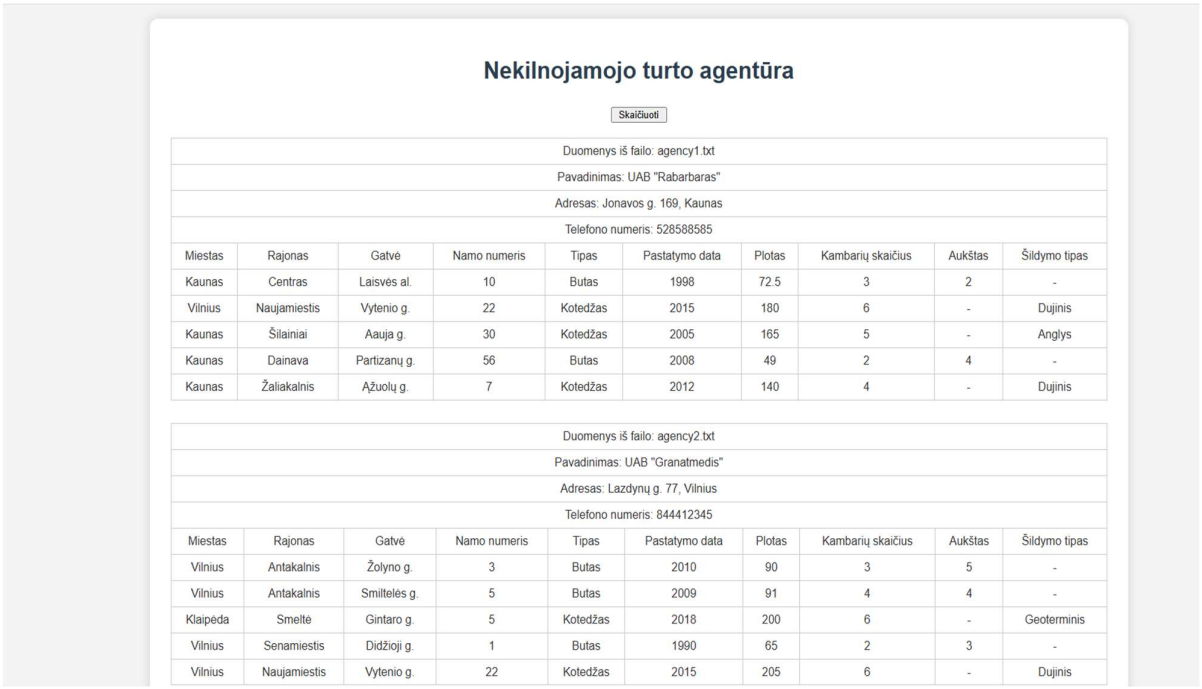
```
                mostCommonStreetEstates =
TaskUtils.PickOutCommonStreets(mostCommonStreetEstates, MaxCount);
                LoadMostCommonStreet(mostCommonStreetEstates, form1);
                InOutUtils.WriteToExternalDataDics(Server.MapPath("~/ExternalData.txt"),
mostCommonStreetEstates, "Dažniausiai pasikartojančios gatvės: ");
            }
            catch (Exception ex)
            {
                MessageBox(this, $"Klaida apdorojant dažniausiai pasikartojančias gatves:
{ex.Message}");
            }

            try
            {
                int minAge = TaskUtils.MinAgeForAll(agencies);
                Agency oldestEstates = TaskUtils.PickOutOldest(agencies, minAge);
                LoadDataToTable(oldestEstates, form1, "Seniausi objektai");
                InOutUtils.WriteToExternalData(Server.MapPath("~/ExternalData.txt"),
oldestEstates, "Seniausi objektai: ");
            }
            catch (Exception ex)
            {
                MessageBox(this, $"Klaida apdorojant seniausius objektus: {ex.Message}");
            }

            try
            {
                Agency FusedAgencies = TaskUtils.FuseAgencies(agencies);
                Agency InAllAgencies = TaskUtils.FindInMultipleAgencies(FusedAgencies);
                InAllAgencies.BubbleSort();
                LoadDataToTable(InAllAgencies, form1, "Objektai kurie yra daugiau nei
vienoje agentūroje");
                InOutUtils.WriteToExternalData(Server.MapPath("~/ExternalData.txt"),
InAllAgencies, "Objektai kurie yra daugiau nei vienoje agentūroje: ");
                InOutUtils.WriteToCSV(InAllAgencies,
Server.MapPath("~/AgencyData/Kartojasi.csv"), "Objektai kurie yra daugiau nei vienoje
agentūroje");
            }
            catch (Exception ex)
            {
                MessageBox(this, $"Klaida apdorojant objektus, kurie yra daugiau nei vienoje
agentūroje: {ex.Message}");
            }

            try
            {
                Agency Large = TaskUtils.LargeObjs(agencies);
                Large.BubbleSort();
                LoadDataToTable(Large, form1, "Dideli objektai");
                InOutUtils.WriteToCSV(Large, Server.MapPath("~/AgencyData/Dideli.csv"),
"Dideli objektai");
                InOutUtils.WriteToExternalData(Server.MapPath("~/ExternalData.txt"), Large,
"Dideli objektai: ");
            }
            catch (Exception ex)
            {
                MessageBox(this, $"Klaida apdorojant didelius objektus: {ex.Message}");
            }
        }
    }
}
```

4.7. Pradiniai duomenys ir rezultatai

Tikrinamas esminis programos veikimas.
Duomenys nr. 1

Agency1.txt
UAB "Rabarbaras"
Jonavos g. 169, Kaunas
528588585
Kaunas;Centras;Laisvės al.;10;Butas;1998;72.5;3;2
Vilnius;Naujamiestis;Vytenio g.;22;Kotedžas;2015;180;6;Dujinis
Kaunas;Šilainiai;Aauja g.;30;Kotedžas;2005;165;5;Anglys
Kaunas;Dainava;Partizanų g.;56;Butas;2008;49;2;4
Kaunas;Žaliakalnis;Ąžuolų g.;7;Kotedžas;2012;140;4;Dujinis
Agency2.txt
UAB "Granatmedis"
Lazdynų g. 77, Vilnius
844412345
Vilnius;Antakalnis;Žolyno g.;3;Butas;2010;90;3;5
Vilnius;Antakalnis;Smiltelės g.;5;Butas;2009;91;4;4
Klaipėda;Smeltė;Gintaro g.;5;Kotedžas;2018;200;6;Geoterminis
Vilnius;Senamiestis;Didžioji g.;1;Butas;1990;65;2;3
Vilnius;Naujamiestis;Vytenio g.;22;Kotedžas;2015;205;6;Dujinis
Agency3.txt
UAB "Molibdenas"
Taikos pr. 101, Klaipėda
862233445
Klaipėda;Bandužiai;Smiltelės g.;18;Butas;2000;54;2;1
Klaipėda;Centras;H. Manto g.;22;Butas;1990;67;3;2
Klaipėda;Smeltė;Gintaro g.;5;Kotedžas;2018;200;6;Geoterminis
Klaipėda;Bandužiai;Smiltelės g.;20;Butas;2001;60;2;3

Rezultatai



Duomenys iš failo: agency1.txt									
Pavadinimas: UAB "Rabarbaras"									
Adresas: Jonavos g. 169, Kaunas									
Telefono numeris: 528588585									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Kaunas	Centras	Laisvės al.	10	Butas	1998	72.5	3	2	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	180	6	-	Dujinis
Kaunas	Šilainiai	Aauja g.	30	Kotedžas	2005	165	5	-	Anglys
Kaunas	Dainava	Partizanų g.	56	Butas	2008	49	2	4	-
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	-	Dujinis

Duomenys iš failo: agency2.txt									
Pavadinimas: UAB "Granatmedis"									
Adresas: Lazdynų g. 77, Vilnius									
Telefono numeris: 844412345									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Vilnius	Antakalnis	Žolyno g.	3	Butas	2010	90	3	5	-
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	4	-
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	3	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	-	Dujinis

Pav.22 Rezultatai sąsajoje

Duomenys iš failo: agency3.txt									
Pavadinimas: UAB "Molibdenas"									
Adresas: Taikos pr. 101, Klaipėda									
Telefono numeris: 862233445									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Klaipėda	Bandužiai	Smiltelės g.	18	Butas	2000	54	2	1	-
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	2	-
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis
Klaipėda	Bandužiai	Smiltelės g.	20	Butas	2001	60	2	3	-

Dažniausiai pasikartojančios gatvės	
Gatvės pavadinimas	Kiekis
Smiltelės g.	3

Seniausi objektai									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	3	-
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	2	-

Objektai kurie yra daugiau nei vienoje agentūroje									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis

Pav.23 Rezultatai sąsajoje

Dideli objektai									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	4	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	-	Dujinis

Pav.24 Rezultatai sąsajoje

Dideli objektai

City	District	Street	House Nur	Type	Build Year	Area	Number of	Heating Ty	Floor
Vilnius	Antakalnis	Smiltelės g	5	Butas	2009	91	4	-	4
Vilnius	Naujamies	Vytenio g.	22	Kotedžas	2015	205	6	Dujinis	-

Pav.25 Rezultatai „Dideli.csv“

Objektai kurie yra daugiau nei vienoje agentūroje

City	District	Street	House Nur	Type	Build Year	Area	Number of	Heating Ty	Floor
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	Geotermin	-

Pav.26 Rezultatai „Kartojasi.csv“

```
Nuskaityti duomenys:
Agency Name: UAB "Rabarbaras"
Agency Address: Jonavos g. 169, Kaunas
Agency Phone Number: 528588585
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Kaunas | Centras | Laisvės al. | 10 | Butas | 1998 | 72.5 | 3 | - | 2 |
| Vilnius | Naujamiestis | Vytenio g. | 22 | Kotedžas | 2015 | 180 | 6 | Dujinis | - |
| Kaunas | Šilainiai | Aauja g. | 30 | Kotedžas | 2005 | 165 | 5 | Anglys | - |
| Kaunas | Dainava | Partizanų g. | 56 | Butas | 2008 | 49 | 2 | - | 4 |
| Kaunas | Žaliakalnis | Ąžuolų g. | 7 | Kotedžas | 2012 | 140 | 4 | Dujinis | - |

Nuskaityti duomenys:
Agency Name: UAB "Granatmedis"
Agency Address: Lazdynų g. 77, Vilnius
Agency Phone Number: 844412345
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Vilnius | Antakalnis | Žolyno g. | 3 | Butas | 2010 | 90 | 3 | - | 5 |
| Vilnius | Antakalnis | Smiltelės g. | 5 | Butas | 2009 | 91 | 4 | - | 4 |
| Klaipėda | Smeltė | Gintaro g. | 5 | Kotedžas | 2018 | 200 | 6 | Geoterminis | - |
| Vilnius | Senamiestis | Didžioji g. | 1 | Butas | 1990 | 65 | 2 | - | 3 |
| Vilnius | Naujamiestis | Vytenio g. | 22 | Kotedžas | 2015 | 205 | 6 | Dujinis | - |

Nuskaityti duomenys:
Agency Name: UAB "Molibdenas"
Agency Address: Taikos pr. 101, Klaipėda
Agency Phone Number: 862233445
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Klaipėda | Bandužiai | Smiltelės g. | 18 | Butas | 2000 | 54 | 2 | - | 1 |
| Klaipėda | Centras | H. Manto g. | 22 | Butas | 1990 | 67 | 3 | - | 2 |
| Klaipėda | Smeltė | Gintaro g. | 5 | Kotedžas | 2018 | 200 | 6 | Geoterminis | - |
| Klaipėda | Bandužiai | Smiltelės g. | 20 | Butas | 2001 | 60 | 2 | - | 3 |

Dažniausiai pasikartojančios gatvės:
| Street | Count |
| Smiltelės g. | 3 |

Seniausi objektai:
Agency Name:
Agency Address:
Agency Phone Number: 0
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Vilnius | Senamiestis | Didžioji g. | 1 | Butas | 1990 | 65 | 2 | - | 3 |
| Klaipėda | Centras | H. Manto g. | 22 | Butas | 1990 | 67 | 3 | - | 2 |

Objektai kurie yra daugiau nei vienoje agentūroje:
Agency Name:
Agency Address:
Agency Phone Number: 0
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Klaipėda | Smeltė | Gintaro g. | 5 | Kotedžas | 2018 | 200 | 6 | Geoterminis | - |

Dideli objektai:
Agency Name:
Agency Address:
Agency Phone Number: 0
| City | District | Street | House Number | Type | Build Year | Area | Number of Rooms | Heating Type | Floor |
| Vilnius | Antakalnis | Smiltelės g. | 5 | Butas | 2009 | 91 | 4 | - | 4 |
| Vilnius | Naujamiestis | Vytenio g. | 22 | Kotedžas | 2015 | 205 | 6 | Dujinis | - |
```

Pav.27 Rezultatai „ExternalData.txt“

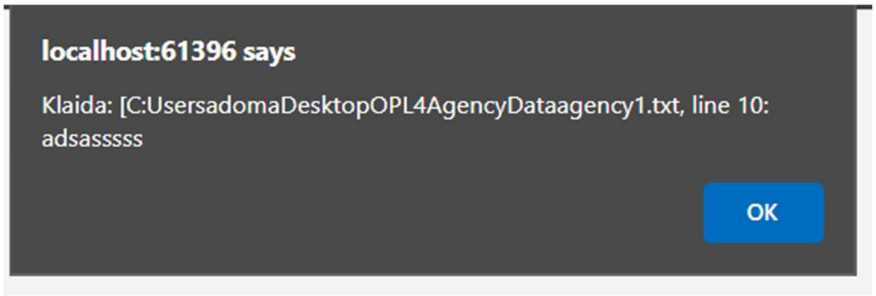
Tikrinamas išimčių valdymas. Pridėti keli dideli objektai.

Duomenys nr. 2

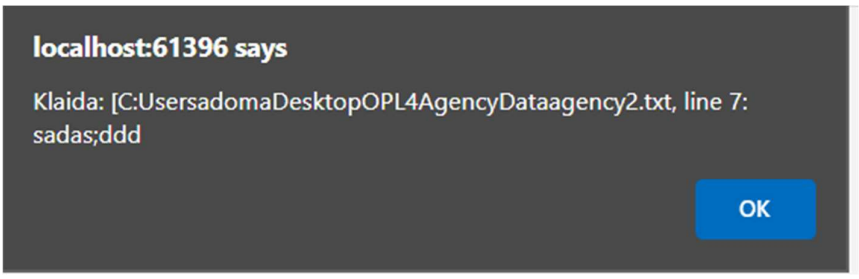
Agency1.txt

UAB "Rabarbaras"
Jonavos g. 169, Kaunas
528588585
Kaunas;Centras;Laisvės al.;10;Butas;1998;8000;3;2
Vilnius;Naujamiestis;Vytenio g.;22;Kotedžas;2015;180;6;Dujinis
Kaunas;Šilainiai;Aauja g.;30;Kotedžas;2005;165;5;Anglys
Kaunas;Dainava;Partizanų g.;56;Butas;2008;3000;2;4
Kaunas;Žaliakalnis;Ąžuolų g.;7;Kotedžas;2012;140;4;Dujinis
Adsasssss
Agency2.txt
UAB "Granatmedis"
Lazdynų g. 77, Vilnius
844412345
Vilnius;Antakalnis;Žolyno g.;3;Butas;2010;90;3;5
Vilnius;Antakalnis;Smiltelės g.;5;Butas;2009;91;4;4
sadas;ddd
Klaipėda;Smeltė;Gintaro g.;5;Kotedžas;2018;200;6;Geoterminis
Vilnius;Senamiestis;Didžioji g.;1;Butas;1990;65;2;3
Vilnius;Naujamiestis;Vytenio g.;22;Kotedžas;2015;205;6;Dujinis
Agency3.txt
UAB "Molibdenas"
Taikos pr. 101, Klaipėda
862233445
Klaipėda;Bandužiai;Smiltelės g.;18;Butas;2000;54;2;1
Klaipėda;Centras;H. Manto g.;22;Butas;1990;67;3;2
Klaipėda;Smeltė;Gintaro g.;5;Kotedžas;2018;200;6;Geoterminis
Klaipėda;Bandužiai;Smiltelės g.;20;Butas;2001;60;2;3

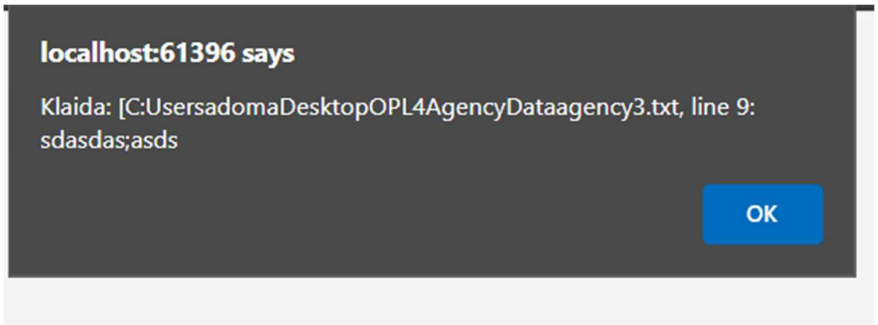
Rezultatai



Pav.28 Klaidos pranešimas



Pav.29 Klaidos pranešimas



Pav.30 Klaidos pranešimas

Programa toliau tesia darbą, praleidusi eilutes kurios netinka.

Nekilnojamojo turto agentūra

Skaičiuoti

Duomenys iš failo: agency1.txt									
Pavadinimas: UAB "Rabarbaras"									
Adresas: Jonavos g. 169, Kaunas									
Telefono numeris: 528588585									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Kaunas	Centras	Laisvės al.	10	Butas	1998	8000	3	2	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	180	6	-	Dujinis
Kaunas	Šilainiai	Aauja g.	30	Kotedžas	2005	165	5	-	Anglys
Kaunas	Dainava	Partizanų g.	56	Butas	2008	3000	2	4	-
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	-	Dujinis

Duomenys iš failo: agency2.txt									
Pavadinimas: UAB "Granatmedis"									
Adresas: Lazdynų g. 77, Vilnius									
Telefono numeris: 844412345									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Vilnius	Antakalnis	Žolyno g.	3	Butas	2010	90	3	5	-
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	4	-
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	3	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	-	Dujinis
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	-	Dujinis

Pav.31 Rezultatai sąsajoje

Duomenys iš failo: agency3.txt									
Pavadinimas: UAB "Molibdenas"									
Adresas: Taikos pr. 101, Klaipėda									
Telefono numeris: 862233445									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Klaipėda	Bandužiai	Smiltelės g.	18	Butas	2000	54	2	1	-
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	2	-
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis
Klaipėda	Bandužiai	Smiltelės g.	20	Butas	2001	60	2	3	-

Dažniausiai pasikartojančios gatvės	
Gatvės pavadinimas	Kiekis
Smiltelės g.	3

Seniausi objektai									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	3	-
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	2	-

Objektai kurie yra daugiau nei vienoje agentūroje									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	-	Dujinis
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	-	Geoterminis

Pav.32 Rezultatai sąsajoje

Dideli objektai									
Miestas	Rajonas	Gatvė	Namo numeris	Tipas	Pastatymo data	Plotas	Kambarių skaičius	Aukštas	Šildymo tipas
Kaunas	Centras	Laisvės al.	10	Butas	1998	8000	3	2	-
Kaunas	Dainava	Partizanų g.	56	Butas	2008	3000	2	4	-
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	4	-
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	-	Dujinis

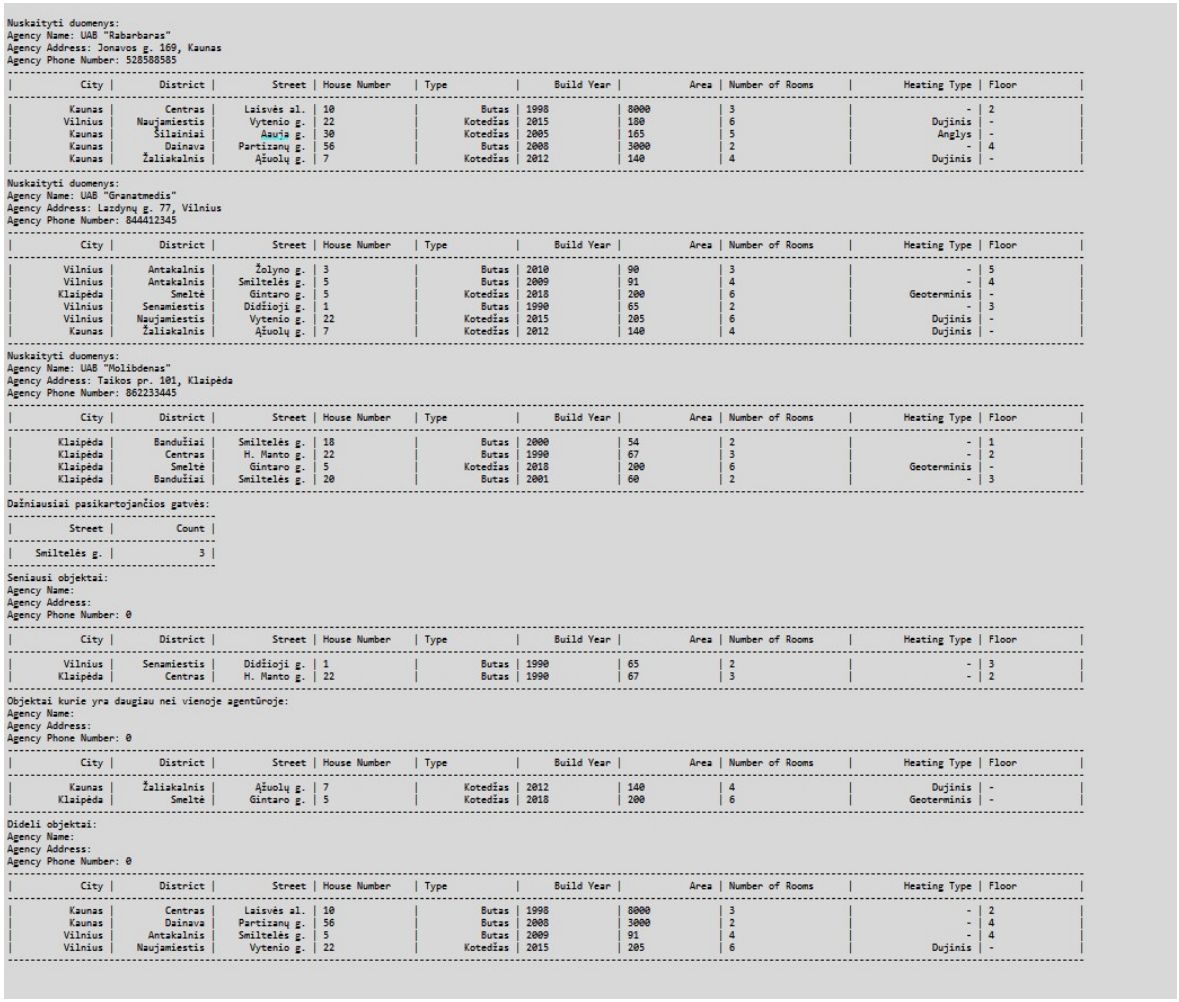
Pav.33 Rezultatai sąsajoje

Dideli objektai									
City	District	Street	House Nur	Type	Build Year	Area	Number of	Heating Ty	Floor
Kaunas	Centras	Laisvės al.	10	Butas	1998	8000	3	-	2
Kaunas	Dainava	Partizanų g	56	Butas	2008	3000	2	-	4
Vilnius	Antakalnis	Smiltelės g	5	Butas	2009	91	4	-	4
Vilnius	Naujamies	Vytenio g.	22	Kotedžas	2015	205	6	Dujinis	-

Pav.34 Rezultatai faile „Dideli.csv“

Objektai kurie yra daugiau nei vienoje agentūroje									
City	District	Street	House Nur	Type	Build Year	Area	Number of	Heating Ty	Floor
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	Geotermin	-

Pav.35 Rezultatai faile „Kartojasi.csv“



```
Nuskaityti duomenys:
Agency Name: UAB "Rabarbaras"
Agency Address: Jonavos g. 169, Kaunas
Agency Phone Number: 528588585
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Kaunas	Centras	Laisvės al.	10	Butas	1998	8000	3	-	2
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	180	6	Dujinis	-
Kaunas	Šilainiai	Aauja g.	30	Kotedžas	2005	165	5	Anglys	-
Kaunas	Dainava	Partizanų g.	56	Butas	2008	3000	2	-	4
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	Dujinis	-

```
Nuskaityti duomenys:
Agency Name: UAB "Granatmedis"
Agency Address: Lazdynų g. 77, Vilnius
Agency Phone Number: 844412345
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Vilnius	Antakalnis	Žolyno g.	3	Butas	2010	90	3	-	5
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	-	4
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	Geoterminis	-
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	-	3
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	Dujinis	-
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	Dujinis	-

```
Nuskaityti duomenys:
Agency Name: UAB "Molibdenas"
Agency Address: Taikos pr. 101, Klaipėda
Agency Phone Number: 862233445
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Klaipėda	Bandužiai	Smiltelės g.	18	Butas	2000	54	2	-	1
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	-	2
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	Geoterminis	-
Klaipėda	Bandužiai	Smiltelės g.	20	Butas	2001	60	2	-	3

```
Dažniausiai pasikartojančios gatvės:
```

Street	Count
Smiltelės g.	3

```
Seniausi objektai:
Agency Name:
Agency Address:
Agency Phone Number: 0
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Vilnius	Senamiestis	Didžioji g.	1	Butas	1990	65	2	-	3
Klaipėda	Centras	H. Manto g.	22	Butas	1990	67	3	-	2

```
Objektai kurie yra daugiau nei vienoje agentūroje:
Agency Name:
Agency Address:
Agency Phone Number: 0
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Kaunas	Žaliakalnis	Ąžuolų g.	7	Kotedžas	2012	140	4	Dujinis	-
Klaipėda	Smeltė	Gintaro g.	5	Kotedžas	2018	200	6	Geoterminis	-

```
Dideli objektai:
Agency Name:
Agency Address:
Agency Phone Number: 0
```

City	District	Street	House Number	Type	Build Year	Area	Number of Rooms	Heating Type	Floor
Kaunas	Centras	Laisvės al.	10	Butas	1998	8000	3	-	2
Kaunas	Dainava	Partizanų g.	56	Butas	2008	3000	2	-	4
Vilnius	Antakalnis	Smiltelės g.	5	Butas	2009	91	4	-	4
Vilnius	Naujamiestis	Vytenio g.	22	Kotedžas	2015	205	6	Dujinis	-

Pav.36 Rezultatai faile „ExternalData.txt“

4.8. Dėstytojo pastabos

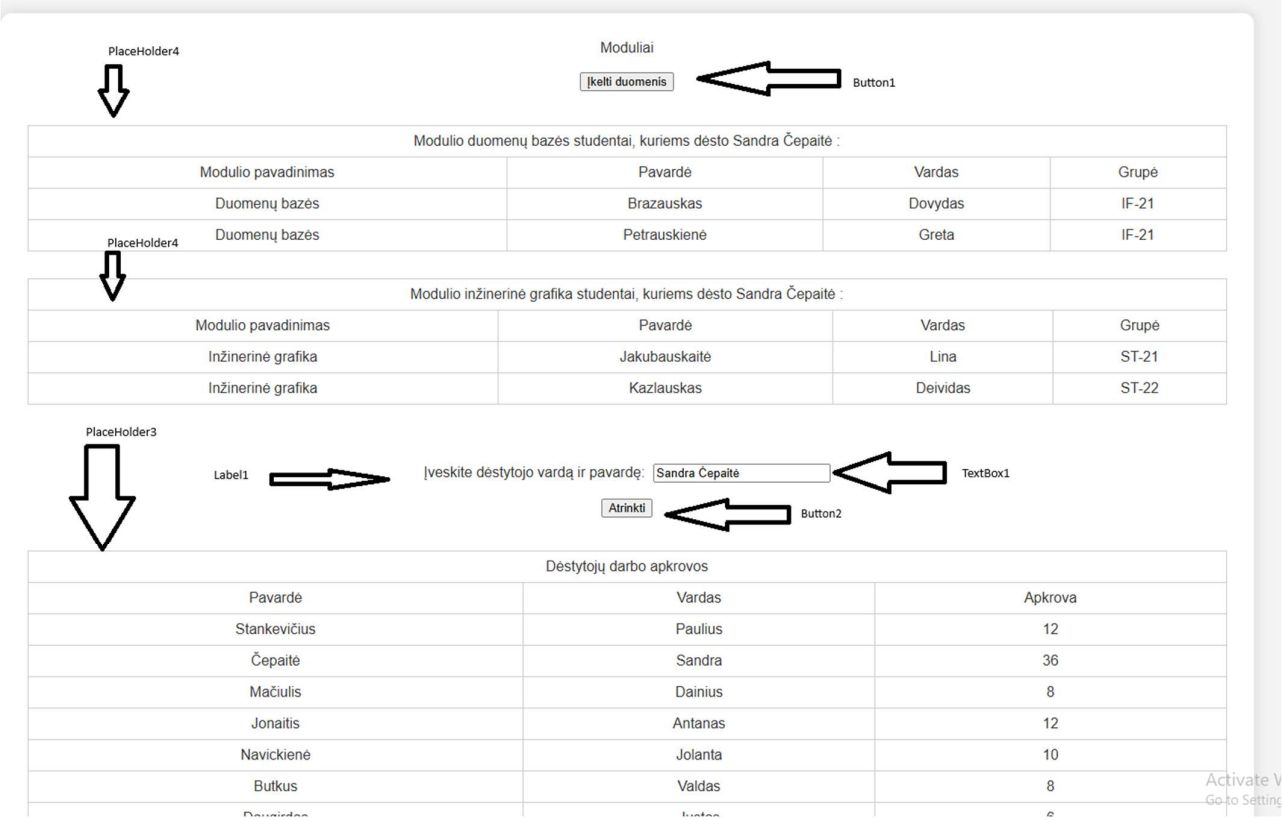
2 taškai už testą
4 taškai už darbą
0,4 taškai už ataskaitą

5. Deklaratyvusis programavimas (L5)

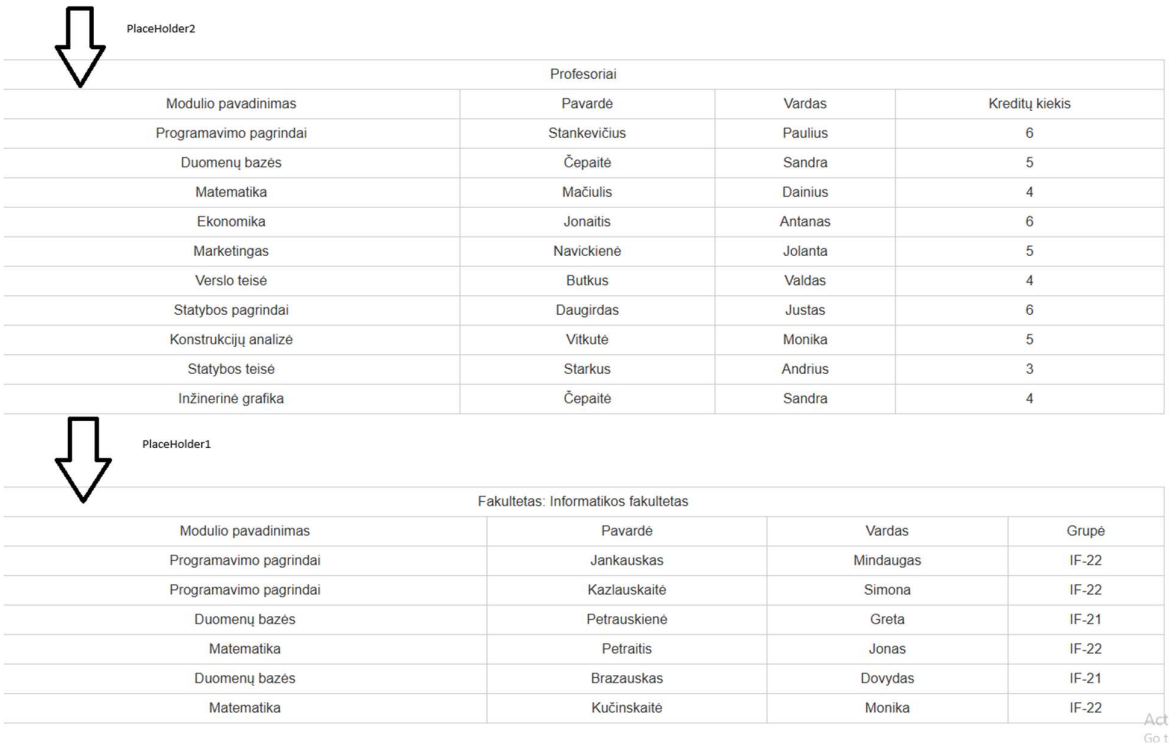
5.1. Darbo užduotis

U5_6. Moduliai.
Pirmoje failo eilutėje nurodytas fakulteto pavadinimas (failų daug). Tekstiniame faile duota informacija apie studentų pasirenkamus modulius: modulio pavadinimas, studento pavardė, vardas, grupė. Atskirame faile yra informacija apie modulius: modulio pavadinimas, atsakingo dėstytojo pavardė, vardas, kreditų kiekis. Suskaičiuoti kiekvieno dėstytojo darbo krūvį, jei modulio kreditų skaičius dauginamas iš jį pasirinkusių studentų skaičiaus. Atspausdinti nurodyto dėstytojo (įvedama klaviatūra) kiekvieno modulio studentų sąrašus pagal grupes ir studentų pavardes.

5.2. Grafinės naudotojo sąsajos schema



Pav.37 GUI schema

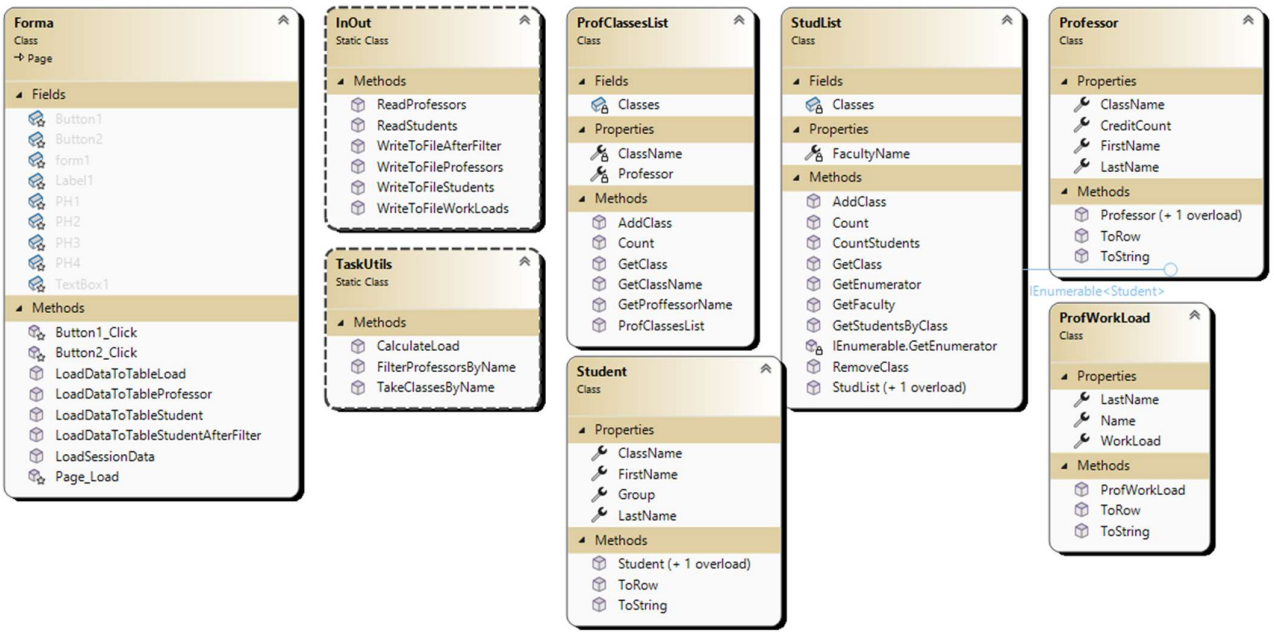


Pav.38 GUI schema

5.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė
Button1	Text	„Įkelti duomenis“
PlaceHolder4	ID	PH4
Label1	Text	„Įveskite dėstytojo vardą ir pavardę“
	Visible	False
TextBox1	Width	190px
	Visible	False
Button2	Visible	False
PlaceHolder3	ID	PH3
PlaceHolder2	ID	PH2
PlaceHolder1	ID	PH1

5.4. Klasių diagrama



Pav.39 Klasių diagrama

5.5. Programos naudotojo vadovas

Visi failai su sudento duomenimis tuyri būti sukelti į katalogą AppData, kurį rasime projekto kataloge. ProfData.txt turi būti įkeltas į šakninį projekto katalogą. Paspaudus mygtuką programoje „Įkelti duomenis“, programa automatiškai nusiskaito duomenis ir pateikia suskaičiuotą krūvį kiekvienam dėstytojui. Įvesdus į paieškos laukelį dėstytojo vardą ir pavardę ir paspaudus mygtuką „Atrinkti“, programa sudarys lenteles studentų, kurie lanko modulius pas nurodytą dėstytoją, sąrašus. Studento duomenų failų formatas:

- Pirmoje eilutėje fakulteto pavadinimas
- Likosios eilutės formatuojamos taip: Modulio _pavadinimas;Pavardė;Vardas;Grupė

Dsėstytojų failo duomenų formatas:

- Modulio_pavadinimas;Pavardė;Vardas;Kreditų_skaičius(Sveikasis skaičius).

5.6. Programos tekstas

Student.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI.WebControls;

namespace LD5
{
    /// <summary>
    /// Represents a student with class, name, and group information.
    /// </summary>
    public class Student
    {
        /// <summary>
        /// The class name.
```

```
        /// </summary>
        public string ClassName { get; set; }

        /// <summary>
        /// The last name.
        /// </summary>
        public string LastName { get; set; }

        /// <summary>
        /// The first name.
        /// </summary>
        public string FirstName { get; set; }

        /// <summary>
        /// The group.
        /// </summary>
        public string Group { get; set; }

        /// <summary>
        /// Creates a student with class name, last name, first name, and group.
        /// </summary>
        public Student(string classname, string lastName, string firstName, string group)
        {
            this.ClassName = classname;
            this.LastName = lastName;
            this.FirstName = firstName;
            this.Group = group;
        }

        /// <summary>
        /// Creates a student with last name, first name, and group.
        /// </summary>
        public Student(string lastName, string Name, string group)
        {
            this.LastName = lastName;
            this.FirstName = Name;
            this.Group = group;
        }

        /// <summary>
        /// Converts the student data to a table row.
        /// </summary>
        public TableRow ToRow()
        {
            TableRow row = new TableRow();
            row.Cells.Add(new TableCell() { Text = ClassName });
            row.Cells.Add(new TableCell() { Text = LastName });
            row.Cells.Add(new TableCell() { Text = FirstName });
            row.Cells.Add(new TableCell() { Text = Group });
            return row;
        }

        /// <summary>
        /// Returns a string with student data in table format.
        /// </summary>
        public override string ToString()
        {
            return string.Format($"| {this.ClassName,25} | {this.LastName,25} |
{this.FirstName,25} | {this.Group,15} |");
        }
    }
}
```

Professor.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI.WebControls;

namespace LD5
{
    /// <summary>
    /// Represents a professor with class name, last name, first name, and credit count.
    /// </summary>
    public class Professor
    {
        /// <summary>
        /// The class name.
        /// </summary>
        public string ClassName { get; set; }

        /// <summary>
        /// The professor's last name.
        /// </summary>
        public string LastName { get; set; }

        /// <summary>
        /// The professor's first name.
        /// </summary>
        public string FirstName { get; set; }

        /// <summary>
        /// The credit count.
        /// </summary>
        public int CreditCount { get; set; }

        /// <summary>
        /// Creates a professor with all details.
        /// </summary>
        public Professor(string classname, string lastName, string firstName, int
     creditCount)
        {
            this.ClassName = classname;
            this.LastName = lastName;
            this.FirstName = firstName;
            this.CreditCount = creditCount;
        }

        /// <summary>
        /// Creates a professor with only last and first name.
        /// </summary>
        public Professor(string lastName, string firstName)
        {
            this.LastName = lastName;
            this.FirstName = firstName;
        }

        /// <summary>
        /// Converts professor data to a table row.
        /// </summary>
        public TableRow ToRow()
        {
            TableRow row = new TableRow();
            row.Cells.Add(new TableCell() { Text = ClassName });
```

```
            row.Cells.Add(new TableCell() { Text = LastName });
            row.Cells.Add(new TableCell() { Text = FirstName });
            row.Cells.Add(new TableCell() { Text = CreditCount.ToString() });
            return row;
        }

        /// <summary>
        /// Returns a formatted string with professor data.
        /// </summary>
        public override string ToString()
        {
            return string.Format($"| {this.ClassName,25} | {this.LastName,25} |
     {this.FirstName,25} | {this.CreditCount,-15} |");
        }
    }
     }
```

ProfWorkLoad.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI.WebControls;

namespace LD5
{
    /// <summary>
    /// Holds information about a professor's workload.
    /// </summary>
    public class ProfWorkLoad
    {
        /// <summary>
        /// Professor's first name.
        /// </summary>
        public string Name { get; set; }

        /// <summary>
        /// Professor's last name.
        /// </summary>
        public string LastName { get; set; }

        /// <summary>
        /// Amount of work assigned to the professor.
        /// </summary>
        public int WorkLoad { get; set; }

        /// <summary>
        /// Creates a new ProfWorkLoad with name, last name, and workload.
        /// </summary>
        public ProfWorkLoad(string name, string lastName, int workLoad)
        {
            this.Name = name;
            this.LastName = lastName;
            this.WorkLoad = workLoad;
        }

        /// <summary>
        /// Converts the professor's data to a table row.
        /// </summary>
        public TableRow ToRow()
        {
            TableRow row = new TableRow();
```

```
            row.Cells.Add(new TableCell() { Text = Name });
            row.Cells.Add(new TableCell() { Text = LastName });
            row.Cells.Add(new TableCell() { Text = WorkLoad.ToString() });
            return row;
        }

        /// <summary>
        /// Returns a formatted string with the professor's data.
        /// </summary>
        public override string ToString()
        {
            return string.Format($"| {this.Name,25} | {this.LastName,25} | {this.WorkLoad,-
     15} |");
        }
    }
     }
```

StudList.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace LD5
{
    /// <summary>
    /// Stores a list of students for a faculty.
    /// Lets you add, remove, and find students by class.
    /// </summary>
    public class StudList : IEnumerable<Student>
    {

        private List<Student> Classes;
        private string FacultyName { get; set; } = string.Empty;

        /// <summary>
        /// Makes a new student list for a faculty.
        /// </summary>
        /// <param name="facultyName">Name of the faculty</param>
        public StudList(string facultyName)
        {
            Classes = new List<Student>();
            this.FacultyName = facultyName;
        }

        /// <summary>
        /// Makes a new empty student list.
        /// </summary>
        public StudList()
        {
            Classes = new List<Student>();
        }

        /// <summary>
        /// Gets the faculty name.
        /// </summary>
        /// <returns>Faculty name</returns>
        public string GetFaculty()
        {
            return FacultyName;
        }
```

```
/// <summary>
/// Adds a student to the list.
/// </summary>
/// <param name="newClass">Student to add</param>
public void AddClass(Student newClass)
{
    Classes.Add(newClass);
}

/// <summary>
/// Removes a student from the list.
/// </summary>
/// <param name="oldClass">Student to remove</param>
public void RemoveClass(Student oldClass)
{
    Classes.Remove(oldClass);
}

/// <summary>
/// Gets a student by index.
/// </summary>
/// <param name="index">Index of student</param>
/// <returns>Student at index</returns>
public Student GetClass(int index)
{
    return Classes[index];
}

/// <summary>
/// Gets the number of students in the list.
/// </summary>
/// <returns>Count of students</returns>
public int Count()
{
    return Classes.Count;
}

/// <summary>
/// Counts students in a class.
/// </summary>
/// <param name="cn">Class name</param>
/// <returns>Number of students in the class</returns>
public int CountStudents(string cn)
{
    return Classes.Count(s => s.ClassName == cn);
}

/// <summary>
/// Gets all students in a class.
/// </summary>
/// <param name="className">Class name</param>
/// <returns>List of students in the class</returns>
public List<Student> GetStudentsByClass(string className)
{
    return Classes.Where(s => s.ClassName == className).ToList();
}

/// <summary>
/// Gets an enumerator for the students.
/// </summary>
/// <returns>Enumerator for students</returns>
public IEnumerator<Student> GetEnumerator()
{
```

```
            return Classes.GetEnumerator();
        }

        IEnumerator IEnumerable.GetEnumerator()
        {
            return GetEnumerator();
        }
    }
     }
```

ProfClassesList.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace LD5.Lists
{
    /// <summary>
    /// Stores a list of students for a professor's class.
    /// Keeps the class name and professor information.
    /// </summary>
    public class ProfClassesList
    {
        /// <summary>
        /// List of students in the class.
        /// </summary>
        private List<Student> Classes;

        /// <summary>
        /// Name of the class.
        /// </summary>
        private string ClassName { get; set; }

        /// <summary>
        /// Professor who teaches the class.
        /// </summary>
        private Professor Professor { get; set; }

        /// <summary>
        /// Creates a new ProfClassesList with class name, students, and professor's name.
        /// </summary>
        /// <param name="className">Name of the class</param>
        /// <param name="afterFilter">List of students</param>
        /// <param name="Name">Professor's first name</param>
        /// <param name="LastName">Professor's last name</param>
        public ProfClassesList(string className, List<Student> afterFilter, string Name,
     string LastName)
        {
            this.Professor = new Professor(LastName, Name);
            Classes = afterFilter;
            this.ClassName = className;
        }

        /// <summary>
        /// Adds a student to the class.
        /// </summary>
        /// <param name="newClass">Student to add</param>
        public void AddClass(Student newClass)
        {
            Classes.Add(newClass);
        }
```

```
        /// <summary>
        /// Gets a student by index.
        /// </summary>
        /// <param name="index">Index of the student</param>
        /// <returns>Student at the given index</returns>
        public Student GetClass(int index)
        {
            return Classes[index];
        }

        /// <summary>
        /// Gets the number of students in the class.
        /// </summary>
        /// <returns>Count of students</returns>
        public int Count()
        {
            return Classes.Count;
        }

        /// <summary>
        /// Gets the class name.
        /// </summary>
        /// <returns>Class name</returns>
        public string GetClassName()
        {
            return ClassName;
        }

        /// <summary>
        /// Gets the professor's full name.
        /// </summary>
        /// <returns>Professor's first and last name</returns>
        public string GetProffessorName()
        {
            return Professor.FirstName + " " + Professor.LastName;
        }

    }
     }
```

TaskUtils.cs

```
using LD5.Lists;
using System;
using System.CodeDom;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Web;
using System.Web.UI.WebControls;

namespace LD5
{
    public static class TaskUtils
    {
        /// <summary>
        /// Calculates the workload for each professor.
        /// Workload = total students in all classes * total credits.
        /// </summary>
        /// <param name="profClasses">List of professors with their classes.</param>
        /// <param name="choises">List of student lists (choices).</param>
        /// <returns>List of professor workloads.</returns>
```

```
   public static List<ProfWorkLoad> CalculateLoad(List<Professor> profClasses,
List<StudList> choises)
   {
       return profClasses
           .GroupBy(prof => new { prof.LastName, prof.FirstName })
           .Select(profGroup =>
           {
               int totalStudents = profGroup.Sum(prof =>
                   choises.Sum(cl => cl.CountStudents(prof.ClassName))
               );
               int totalCredits = profGroup.Sum(prof => prof.CreditCount);
               int load = totalStudents * totalCredits;
               return new ProfWorkLoad(profGroup.Key.LastName,
profGroup.Key.FirstName, load);
           })
           .ToList();
   }

   /// <summary>
   /// Gets a list of classes for each professor with students in those classes.
   /// </summary>
   /// <param name="profs">List of professors.</param>
   /// <param name="studentsLists">List of student lists.</param>
   /// <returns>List of professor classes with students.</returns>
   public static List<ProfClassesList> TakeClassesByName(List<Professor> profs,
List<StudList> studentsLists)
   {
       List<ProfClassesList> tempList = profs
       .Select(prof => new ProfClassesList(
       prof.ClassName,
       studentsLists.SelectMany(s => s.GetStudentsByClass(prof.ClassName)).OrderBy(s
=> s.Group).ThenBy(s => s.LastName).ToList(),
       prof.FirstName,
       prof.LastName)).ToList();

       return tempList;
   }

   /// <summary>
   /// Finds professors by full name (FirstName LastName).
   /// Shows alert if input is wrong.
   /// </summary>
   /// <param name="Name_LastName">Professor's full name (FirstName LastName).</param>
   /// <param name="profesors">List of professors.</param>
   /// <returns>List of professors matching the name.</returns>
   public static List<Professor> FilterProfessorsByName(string Name_LastName,
List<Professor> profesors)
   {
       List<Professor> filteredProfessors = new List<Professor>();
       try
       {
           string[] parts = Name_LastName.Split(' ');
           string name = parts[0];
           string lastName = parts[1];
           filteredProfessors = profesors
               .Where(prof => prof.FirstName.Equals(name,
StringComparison.OrdinalIgnoreCase) &&
                              prof.LastName.Equals(lastName,
StringComparison.OrdinalIgnoreCase))
               .ToList();

       }
       catch
       {
```

```
                HttpContext.Current.Response.Write(
                   String.Format($"<script>alert('Neteisingai įvestas dėstytojas" +
                  $" {Name_LastName}. Formatas: Vardas Pavardė')</script>"));
            }
            return filteredProfessors;
        }
    }
     }
```

InOutUtils.cs

```
using LD5.Lists;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Web;

namespace LD5
{
    /// <summary>
    /// Handles reading and writing student and professor data to files.
    /// </summary>
    public static class InOut
    {
        /// <summary>
        /// Reads students from a file and returns a StudList.
        /// </summary>
        /// <param name="fileName">The file to read from.</param>
        /// <returns>List of students.</returns>
        public static StudList ReadStudents(string fileName)
        {
            StudList students = null;
            using (StreamReader reader = new StreamReader(fileName, Encoding.UTF8))
            {
                string line;
                int lineCounter = 1;
                line = reader.ReadLine();
                students = new StudList(line);
                while ((line = reader.ReadLine()) != null)
                {
                    try
                    {
                        string[] parts = line.Split(';');
                        string className = parts[0];
                        string lastname = parts[1];
                        string name = parts[2];
                        string group = parts[3];

                        Student student = new Student(className, lastname, name, group);
                        students.AddClass(student);
                    }
                    catch
                    {
                        HttpContext.Current.Response.Write(
                            String.Format($"<script>alert('Klaida failo" +
                            $" {fileName} eilutėje {lineCounter}')</script>"));
                        continue;
                    }
                    finally { lineCounter++; }
                }
            }
```

```
        return students;
    }

    /// <summary>
    /// Reads professors from a file and returns a list of Professor objects.
    /// </summary>
    /// <param name="fileName">The file to read from.</param>
    /// <returns>List of professors.</returns>
    public static List<Professor> ReadProfessors(string fileName)
    {
        List<Professor> profs = null;
        using (StreamReader reader = new StreamReader(fileName, Encoding.UTF8))
        {
            string line;
            int lineCounter = 1;
            profs = new List<Professor>();
            while ((line = reader.ReadLine()) != null)
            {
                try
                {
                    string[] parts = line.Split(';');
                    string className = parts[0];
                    string lastname = parts[1];
                    string name = parts[2];
                    int credits = int.Parse(parts[3]);
                    Professor professor = new Professor(className, lastname, name,
credits);
                    profs.Add(professor);
                }
                catch
                {
                    HttpContext.Current.Response.Write(
                        String.Format($"<script>alert('Klaida failo" +
                        $" {fileName} eilutėje {lineCounter}')</script>"));
                    continue;
                }
                finally { lineCounter++; }
            }
        }
        return profs;
    }

    /// <summary>
    /// Writes a list of student lists to a file with a header.
    /// </summary>
    /// <param name="fileName">The file to write to.</param>
    /// <param name="list">List of student lists.</param>
    /// <param name="header">Header text.</param>
    public static void WriteToFileStudents(string fileName, List<StudList> list, string
header)
    {
        using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
        {
            writer.WriteLine(header);
            foreach (StudList studList in list)
            {
                writer.WriteLine("Fakultetas: " + studList.GetFaculty());
                writer.WriteLine(new string('-', 103));
                writer.WriteLine($"| {"Modulio pavadinimas",25} | {"Pavardė",25} |
{"Vardas",25} | {"Grupė",15} |");
                writer.WriteLine(new string('-', 103));
                foreach (Student s in studList)
                {
                    writer.WriteLine(s.ToString());
```

```
                }
                writer.WriteLine(new string('-', 103));
            }
        }
    }

    /// <summary>
    /// Writes a list of professors to a file with a header.
    /// </summary>
    /// <param name="fileName">The file to write to.</param>
    /// <param name="list">List of professors.</param>
    /// <param name="header">Header text.</param>
    public static void WriteToFileProfessors(string fileName, List<Professor> list,
string header)
    {
        using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
        {
            writer.WriteLine(header);
            writer.WriteLine(new string('-', 103));
            writer.WriteLine($"| {"Modulio pavadinimas",25} | {"Pavardė",25} |
{"Vardas",25} | {"Kreditų kiekis",15} |");
            writer.WriteLine(new string('-', 103));
            foreach (Professor s in list)
            {
                writer.WriteLine(s.ToString());
            }
            writer.WriteLine(new string('-', 103));
        }
    }

    /// <summary>
    /// Writes a list of professor workloads to a file with a header.
    /// </summary>
    /// <param name="fileName">The file to write to.</param>
    /// <param name="list">List of professor workloads.</param>
    /// <param name="header">Header text.</param>
    public static void WriteToFileWorkLoads(string fileName, List<ProfWorkLoad> list,
string header)
    {
        using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
        {
            writer.WriteLine(header);
            writer.WriteLine(new string('-', 75));
            writer.WriteLine($"| {"Vardas",25} | {"Pavardė",25} | {"Grupė",15} |");
            writer.WriteLine(new string('-', 75));
            foreach (ProfWorkLoad s in list)
            {
                writer.WriteLine(s.ToString());
            }
            writer.WriteLine(new string('-', 75));
        }
    }

    /// <summary>
    /// Writes filtered professor classes to a file with a header.
    /// </summary>
    /// <param name="fileName">The file to write to.</param>
    /// <param name="listintList">List of filtered professor classes.</param>
    /// <param name="header">Header text.</param>
    public static void WriteToFileAfterFilter(string fileName, List<ProfClassesList>
listintList, string header)
    {
        using (StreamWriter writer = new StreamWriter(fileName, true, Encoding.UTF8))
        {
```

```
                writer.WriteLine(header);
                writer.WriteLine(new string('-', 103));
                writer.WriteLine($"| {"Modulio pavadinimas",25} | {"Pavardė",25} |
     {"Vardas",25} | {"Grupė",15} |");
                writer.WriteLine(new string('-', 103));
                for (int i = 0; i < listintList.Count(); i++)
                {
                    ProfClassesList profClass = listintList[i];
                    for (int j = 0; j < profClass.Count(); j++)
                    {
                        writer.WriteLine(profClass.GetClass(j).ToString());
                    }
                }
                writer.WriteLine(new string('-', 103));
            }
        }
    }
     }
```

Forma.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Forma.aspx.cs"
     Inherits="LD5.Forma" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <link href="Styles.css" rel="stylesheet" type="text/css" />
    <title></title>
</head>
<body>
    <form id="form1" runat="server" visible="True">
        <div>
            Moduliai<br />
            <br />
            <asp:Button ID="Button1" runat="server" OnClick="Button1_Click" Text="Įkelti
     duomenis" />
            <br />
            <br />
            <asp:PlaceHolder ID="PH4" runat="server"></asp:PlaceHolder>
            <br />
            <br />
            <asp:Label ID="Label1" runat="server" Text="Įveskite dėstytojo vardą ir
     pavardę:" Visible="False"></asp:Label>
 <asp:TextBox ID="TextBox1" runat="server" Visible="False"
     Width="190px"></asp:TextBox>
            <br />
            <br />
            <asp:Button ID="Button2" runat="server" OnClick="Button2_Click" Text="Atrinkti"
     Visible="False" />
            <br />
            <br />
            <asp:PlaceHolder ID="PH3" runat="server"></asp:PlaceHolder>
            <br />
            <br />
            <asp:PlaceHolder ID="PH2" runat="server"></asp:PlaceHolder>
            <br />
            <br />
            <asp:PlaceHolder ID="PH1" runat="server"></asp:PlaceHolder>
        </div>
    </form>
</body>
```

```
</html>
```

Forma.aspx.cs

```
using LD5.Lists;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace LD5
{
    public partial class Forma : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                Button2.Visible = false;
                Label1.Visible = false;
                TextBox1.Visible = false;
                if (File.Exists(Server.MapPath("~/ExternalData.txt")))
     File.Delete(Server.MapPath("~/ExternalData.txt"));
            }
            else
            {
                LoadSessionData();
            }

        }

        protected void Button1_Click(object sender, EventArgs e)
        {
            string folderPath = Server.MapPath("~/AppData/");
            string[] filePaths = Directory.GetFiles(folderPath, "*.txt");
            List<StudList> studentsChoises = new List<StudList>();
            List<Professor> professors = new List<Professor>();
            if (filePaths.Length == 0)
            {
                HttpContext.Current.Response.Write("<script>alert('Nerasta jokių failų
     AppData aplanke.')</script>");
                return;
            }
            try
            {
                studentsChoises = filePaths.Select(path =>
     InOut.ReadStudents(path)).ToList();
                InOut.WriteToFileStudents(Server.MapPath("~/ExternalData.txt"),
     studentsChoises, "Nuskaityti studentų duomenys");
            }
            catch
            {
                HttpContext.Current.Response.Write("<script>alert('Klaida skaitant studentų
     duomenis. Patikrinkite failus AppData aplanke.')</script>");
            }
            try
            {
               professors = InOut.ReadProfessors(Server.MapPath("~/ProfData.txt"));
                InOut.WriteToFileProfessors(Server.MapPath("~/ExternalData.txt"),
```

```
professors, "Nuskaityti dėstytojų duomenys");
        }
        catch
        {
            HttpContext.Current.Response.Write("<script>alert('Klaida skaitant
dėstytojų duomenis. Patikrinkite ProfData.txt failą.')</script>");
            return;
        }


        studentsChoises.ForEach(students =>
        {
            LoadDataToTableStudent(students, $"Fakultetas: {students.GetFaculty()}",
PH1);
        });


        LoadDataToTableProfessor(professors, "Profesoriai", PH2);
        Session["students"] = studentsChoises;
        Session["professors"] = professors;
        List<ProfWorkLoad> LoadCalc = new List<ProfWorkLoad>();
        try
        {
            LoadCalc = TaskUtils.CalculateLoad(professors, studentsChoises);
            InOut.WriteToFileWorkLoads(Server.MapPath("~/ExternalData.txt"), LoadCalc,
"Dėstytojų darbo apkrovos");
        }
        catch
        {
            HttpContext.Current.Response.Write("<script>alert('Klaida skaičiuojant
dėstytojų apkrovą.')</script>");
            return;
        }
        LoadDataToTableLoad(LoadCalc, "Dėstytojų darbo apkrovos", PH3);
        Session["Load"] = LoadCalc;

        Button2.Visible = true;
        Label1.Visible = true;
        TextBox1.Visible = true;
    }

    protected void Button2_Click(object sender, EventArgs e)
    {
        string Name_LastName = TextBox1.Text.Trim();
        List<Professor> filteredProfessors =
TaskUtils.FilterProfessorsByName(Name_LastName,
(List<Professor>)Session["professors"]);
        List<ProfClassesList> filteredByClass =
TaskUtils.TakeClassesByName(filteredProfessors, (List<StudList>)Session["students"]);
        filteredByClass.ForEach(ProfClass =>
        {
            LoadDataToTableStudentAfterFilter(ProfClass, "Modulio "
            + ProfClass.GetClassName().ToLower() + " studentai, kuriems dėsto " +
ProfClass.GetProffessorName() + " : ", PH4);
        });
        InOut.WriteToFileAfterFilter(Server.MapPath("~/ExternalData.txt"),
filteredByClass, Name_LastName + " studentai: ");

    }
}
 }
```

FormaMethods.cs

```
using LD5.Lists;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Routing;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;

namespace LD5
{
    public partial class Forma : System.Web.UI.Page
    {
        /// <summary>
        /// Loads filtered student data into a table and adds it to the given placeholder.
        /// </summary>
        /// <param name="classes">List of filtered student classes.</param>
        /// <param name="header">Table header text.</param>
        /// <param name="ph">Placeholder to add the table to.</param>
        public void LoadDataToTableStudentAfterFilter(ProfClassesList classes, string
     header, PlaceHolder ph)
        {
            Table table = new Table();
            TableRow headerRow = new TableRow();
            headerRow.Cells.Add(new TableCell() { Text = string.Format(header), ColumnSpan
     = 4 });
            table.Rows.Add(headerRow);
            TableRow hRow1 = new TableRow();
            hRow1.Cells.Add(new TableCell() { Text = "Modulio pavadinimas" });
            hRow1.Cells.Add(new TableCell() { Text = "Pavardė" });
            hRow1.Cells.Add(new TableCell() { Text = "Vardas" });
            hRow1.Cells.Add(new TableCell() { Text = "Grupė" });
            table.Controls.Add(hRow1);
            for (int i = 0; i < classes.Count(); i++)
            {
                table.Rows.Add(classes.GetClass(i).ToRow());
            }
            ph.Controls.Add(table);
        }

        /// <summary>
        /// Loads student data into a table and adds it to the given placeholder.
        /// </summary>
        /// <param name="classes">List of student classes.</param>
        /// <param name="header">Table header text.</param>
        /// <param name="ph">Placeholder to add the table to.</param>
        public void LoadDataToTableStudent(StudList classes, string header, PlaceHolder ph)
        {
            Table table = new Table();
            TableRow headerRow = new TableRow();
            headerRow.Cells.Add(new TableCell() { Text = string.Format(header), ColumnSpan
     = 4 });
            table.Rows.Add(headerRow);
            TableRow hRow1 = new TableRow();
            hRow1.Cells.Add(new TableCell() { Text = "Modulio pavadinimas" });
            hRow1.Cells.Add(new TableCell() { Text = "Pavardė" });
            hRow1.Cells.Add(new TableCell() { Text = "Vardas" });
            hRow1.Cells.Add(new TableCell() { Text = "Grupė" });
            table.Controls.Add(hRow1);
            for (int i = 0; i < classes.Count(); i++)
            {
                table.Rows.Add(classes.GetClass(i).ToRow());
```

```
        }
        ph.Controls.Add(table);
    }

    /// <summary>
    /// Loads professor data into a table and adds it to the given placeholder.
    /// </summary>
    /// <param name="classes">List of professors.</param>
    /// <param name="header">Table header text.</param>
    /// <param name="ph">Placeholder to add the table to.</param>
    public void LoadDataToTableProfessor(List<Professor> classes, string header,
PlaceHolder ph)
    {
        Table table = new Table();
        TableRow headerRow = new TableRow();
        headerRow.Cells.Add(new TableCell() { Text = string.Format(header), ColumnSpan
= 4 });
        table.Rows.Add(headerRow);
        TableRow hRow1 = new TableRow();
        hRow1.Cells.Add(new TableCell() { Text = "Modulio pavadinimas" });
        hRow1.Cells.Add(new TableCell() { Text = "Pavardė" });
        hRow1.Cells.Add(new TableCell() { Text = "Vardas" });
        hRow1.Cells.Add(new TableCell() { Text = "Kreditų kiekis" });
        table.Controls.Add(hRow1);
        foreach (Professor p in classes)
        {
            table.Rows.Add(p.ToRow());
        }
        ph.Controls.Add(table);
    }

    /// <summary>
    /// Loads professor workload data into a table and adds it to the given
placeholder.
    /// </summary>
    /// <param name="loads">List of professor workloads.</param>
    /// <param name="header">Table header text.</param>
    /// <param name="ph">Placeholder to add the table to.</param>
    public void LoadDataToTableLoad(List<ProfWorkLoad> loads, string header,
PlaceHolder ph)
    {
        Table table = new Table();
        TableRow headerRow = new TableRow();
        headerRow.Cells.Add(new TableCell() { Text = string.Format(header), ColumnSpan
= 3 });
        table.Rows.Add(headerRow);
        TableRow hRow1 = new TableRow();
        hRow1.Cells.Add(new TableCell() { Text = "Pavardė" });
        hRow1.Cells.Add(new TableCell() { Text = "Vardas" });
        hRow1.Cells.Add(new TableCell() { Text = "Apkrova" });
        table.Controls.Add(hRow1);
        foreach (ProfWorkLoad l in loads)
        {
            table.Rows.Add(l.ToRow());
        }
        ph.Controls.Add(table);
    }

    /// <summary>
    /// Loads data from session and fills tables for students, professors, and
workloads.
    /// </summary>
    public void LoadSessionData()
    {
```

```
            try
            {
                if (Session["students"] != null)
                {
                    List<StudList> studentsChoises = (List<StudList>)Session["students"];
                    studentsChoises.ForEach(students =>
                    {
                        LoadDataToTableStudent(students, $"Fakultetas:
     {students.GetFaculty()}", PH1);
                    });
                }
                if (Session["professors"] != null)
                {
                    List<Professor> professors = (List<Professor>)Session["professors"];
                    LoadDataToTableProfessor(professors, "Profesoriai", PH2);
                }
                if (Session["Load"] != null)
                {
                    List<ProfWorkLoad> LoadCalc = (List<ProfWorkLoad>)Session["Load"];
                    LoadDataToTableLoad(LoadCalc, "Dėstytojų darbo apkrovos", PH3);
                }
            }
            catch (Exception)
            {
                HttpContext.Current.Response.Write("<script>alert('Klaida įkeliant sesijos
     duomenis.')</script>");
            }
        }
    }
     }
```

Styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    color: #333;
    margin: 0;
    padding: 20px;
}

form {
    background-color: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    text-align: center;
}

h1 {
    text-align: center;
    color: #2c3e50;
    margin-bottom: 30px;
}

form {
    width: 95%;
    max-width: 1400px;
    margin: auto;
    background-color: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 0 15px rgba(0, 0, 0, 0.15);
```

```
    text-align: center;
    box-sizing: border-box;
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
    margin-bottom: 30px;
}

table, th, td {
    border: 1px solid #ccc;
}

th, td {
    padding: 8px;
    text-align: center;
}

caption {
    font-weight: bold;
    margin-bottom: 10px;
}
```

5.7. Pradiniai duomenys ir rezultatai

Duomenys Nr. 1

IT.txt

```
Informatikos fakultetas
Programavimo pagrindai;Jankauskas;Mindaugas;IF-22
Programavimo pagrindai;Kazlauskaitė;Simona;IF-22
Duomenų bazės;Petrauskienė;Greta;IF-21
Matematika;Petraitis;Jonas;IF-22
Duomenų bazės;Brazauskas;Dovydas:21
Duomenų bazės;Brazauskas;Dovydas;IF-21
Matematika;Kučinskaitė;Monika;IF-22
```

Verslas.txt

```
Verslo fakultetas
Ekonomika;Bartkus;Matas;VE-21
Marketingas;Stonytė;Laura;VE-22
Verslo teisė;Navickas;Andrius;VE-22
Ekonomika;Grigaitė;Ema;VE-21
Verslo teisė;Rakauskas;Tomas;VE-21
Marketingas;Balčiūnaitė;Ieva;VE-22
```

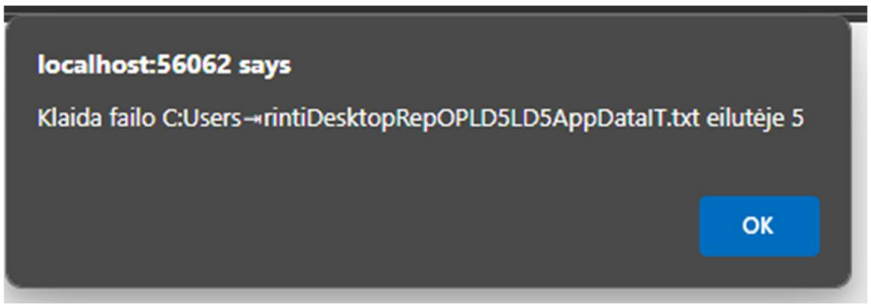
Statyba.txt

```
Statybos fakultetas
Statybos pagrindai;Mickevičius;Lukas;ST-21
Konstrukcijų analizė;Mažeikaitė;Indrė;ST-22
Statybos teisė;Vaitkus;Mindaugas;ST-21
Konstrukcijų analizė;Jasaitis;Tadas;ST-22
Inžinerinė grafika;Jakubauskaitė;Lina;ST-21
Inžinerinė grafika;Kazlauskas;Deividas;ST-22
```

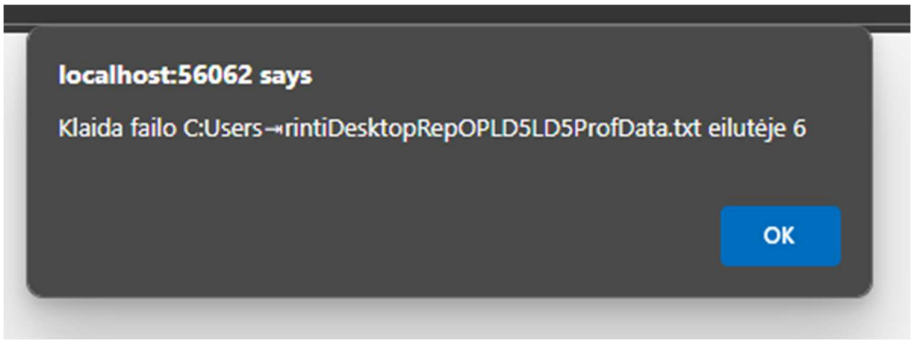
ProfData.txt

```
Programavimo pagrindai;Stankevičius;Paulius;6
Duomenų bazės;Čepaitė;Sandra;5
Matematika;Mačiulis;Dainius;4
Ekonomika;Jonaitis;Antanas;6
Marketingas;Navickienė;Jolanta;5
;;sd;;ds;'''as;;

Verslo teisė;Butkus;Valdas;4
Statybos pagrindai;Daugirdas;Justas;6
Konstrukcijų analizė;Vitkutė;Monika;5
Statybos teisė;Starkus;Andrius;3
sdsd;;s;;a;d;;;
Inžinerinė grafika;Čepaitė;Sandra;4
```

Rezultatai



Pav.40 Klaida



Pav.41 Klaida

Modulio duomenų bazės studentai, kuriems dėsto Sandra Čepaitė :			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Duomenų bazės	Brazauskas	Dovydas	IF-21
Duomenų bazės	Petrauskienė	Greta	IF-21

Modulio inžinerinė grafika studentai, kuriems dėsto Sandra Čepaitė :			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Įveskite dėstytojo vardą ir pavardę: Sandra Čepaitė

Atrinkti

Pav. 42 Rezultatai po filtracijos

Dėstytojų darbo apkrovos		
Pavardė	Vardas	Apkrova
Stankevičius	Paulius	12
Čepaitė	Sandra	36
Mačiulis	Dainius	8
Jonaitis	Antanas	12
Navickienė	Jolanta	10
Butkus	Valdas	8
Daugirdas	Justas	6
Viškutė	Monika	10
Starkus	Andrius	3

Pav.43 Dėstytojų krūvių rezultatai

Profesoriai			
Modulio pavadinimas	Pavardė	Vardas	Kreditų kiekis
Programavimo pagrindai	Stankevičius	Paulius	6
Duomenų bazės	Čepaitė	Sandra	5
Matematika	Mačiulis	Dainius	4
Ekonomika	Jonaitis	Antanas	6
Marketingas	Navickienė	Jolanta	5
Verslo teisė	Butkus	Valdas	4
Statybos pagrindai	Daugirdas	Justas	6
Konstrukcijų analizė	Vitkutė	Monika	5
Statybos teisė	Starkus	Andrius	3
Inžinerinė grafika	Čepaitė	Sandra	4

Fakultetas: Informatikos fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimo pagrindai	Jankauskas	Mindaugas	IF-22
Programavimo pagrindai	Kazlauskaitė	Simona	IF-22
Duomenų bazės	Petrauskienė	Greta	IF-21
Matematika	Petraitis	Jonas	IF-22
Duomenų bazės	Brazauskas	Dovydas	IF-21
Matematika	Kučinskaitė	Monika	IF-22

Fakultetas: Statybos fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Statybos pagrindai	Mickevičius	Lukas	ST-21
Konstrukcijų analizė	Mažeikaitė	Indrė	ST-22
Statybos teisė	Vaitkus	Mindaugas	ST-21
Konstrukcijų analizė	Jasaitis	Tadas	ST-22
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Fakultetas: Verslo fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Ekonomika	Bartkus	Matas	VE-21
Marketingas	Stonytė	Laura	VE-22
Verslo teisė	Navickas	Andrius	VE-22
Ekonomika	Grigaitė	Ema	VE-21
Verslo teisė	Rakauskas	Tomas	VE-21
Marketingas	Balčiūnaitė	Ieva	VE-22

Pav.44 Nuskaityti duomenys

Nuskaityti studentų duomenys
Fakultetas: Informatikos fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimo pagrindai	Jankauskas	Mindaugas	IF-22
Programavimo pagrindai	Kazlauskaitė	Simona	IF-22
Duomenų bazės	Petrauskienė	Greta	IF-21
Matematika	Petraitis	Jonas	IF-22
Duomenų bazės	Brazauskas	Dovydas	IF-21
Matematika	Kučinskaitė	Monika	IF-22

Fakultetas: Statybos fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Statybos pagrindai	Mickevičius	Lukas	ST-21
Konstrukcijų analizė	Mažeikaitė	Indrė	ST-22
Statybos teisė	Vaitkus	Mindaugas	ST-21
Konstrukcijų analizė	Jasaitis	Tadas	ST-22
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Fakultetas: Verslo fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Ekonomika	Bartkus	Matas	VE-21
Marketingas	Stonytė	Laura	VE-22
Verslo teisė	Navickas	Andrius	VE-22
Ekonomika	Grigaitė	Ema	VE-21
Verslo teisė	Rakauskas	Tomas	VE-21
Marketingas	Balčiūnaitė	Ieva	VE-22

Nuskaityti dėstytojų duomenys

Modulio pavadinimas	Pavardė	Vardas	Kreditų kiekis
Programavimo pagrindai	Stankevičius	Paulius	6
Duomenų bazės	Čepaitė	Sandra	5
Matematika	Mačiulis	Dainius	4
Ekonomika	Jonaitis	Antanas	6
Marketingas	Navickienė	Jolanta	5
Verslo teisė	Butkus	Valdas	4
Statybos pagrindai	Daugirdas	Justas	6
Konstrukcijų analizė	Vitkutė	Monika	5
Statybos teisė	Starkus	Andrius	3
Inžinerinė grafika	Čepaitė	Sandra	4

Dėstytojų darbo apkrovos

Vardas	Pavardė	Grupė
Stankevičius	Paulius	12
Čepaitė	Sandra	36
Mačiulis	Dainius	8
Jonaitis	Antanas	12
Navickienė	Jolanta	10
Butkus	Valdas	8
Daugirdas	Justas	6
Vitkutė	Monika	10
Starkus	Andrius	3

Sandra Čepaitė studentai:

Modulio pavadinimas	Pavardė	Vardas	Grupė
Duomenų bazės	Brazauskas	Dovydas	IF-21
Duomenų bazės	Petrauskienė	Greta	IF-21
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Pav.45 Rezultatai „ExternalData.txt“

Duomenys Nr. 2

IT.txt

```
Informatikos fakultetas
Programavimo pagrindai;Jankauskas;Mindaugas;IF-22
Programavimo pagrindai;Kazlauskaitė;Simona;IF-22
Duomenų bazės;Petrauskienė;Greta;IF-21
Matematika;Petraitis;Jonas;IF-22
Duomenų bazės;Brazauskas;Dovydas:21
Duomenų bazės;Brazauskas;Dovydas;IF-21
Matematika;Kučinskaitė;Monika;IF-22
```

Verslas.txt

```
Verslo fakultetas
Ekonomika;Bartkus;Matas;VE-21
Marketingas;Stonytė;Laura;VE-22
Verslo teisė;Navickas;Andrius;VE-22
Ekonomika;Grigaitė;Ema;VE-21
Verslo teisė;Rakauskas;Tomas;VE-21
Marketingas;Balčiūnaitė;Ieva;VE-22
```

Statyba.txt

```
Statybos fakultetas
Statybos pagrindai;Mickevičius;Lukas;ST-21
Konstrukcijų analizė;Mažeikaitė;Indrė;ST-22
Statybos teisė;Vaitkus;Mindaugas;ST-21
Konstrukcijų analizė;Jasaitis;Tadas;ST-22
Inžinerinė grafika;Jakubauskaitė;Lina;ST-21
Inžinerinė grafika;Kazlauskas;Deividas;ST-22
```

Itsort.txt

```
Informatikos fakultetas
Programavimas;Balčiūnas;Tomas;IF-21
Programavimas;Adamkevičius;Lukas;IF-22
Programavimas;Žukauskas;Marius;IF-22
Programavimas;Petraitis;Jonas;IF-21
Programavimas;Kazlauskas;Andrius;IF-21
Programavimas;Dulkytė;Eglė;IF-23
Programavimas;Bartkevičius;Simonas;IF-23
Programavimas;Jakubauskaitė;Laura;IF-22
Programavimas;Jurgelevičius;Dainius;IF-21
Programavimas;Vasiliauskaitė;Rasa;IF-23
```

ProfData.txt

```
Programavimo pagrindai;Stankevičius;Paulius;6
Duomenų bazės;Čepaitė;Sandra;5
Matematika;Mačiulis;Dainius;4
Ekonomika;Jonaitis;Antanas;6
Marketingas;Navickienė;Jolanta;5
Verslo teisė;Butkus;Valdas;4
Statybos pagrindai;Daugirdas;Justas;6
Konstrukcijų analizė;Vitkutė;Monika;5
Statybos teisė;Starkus;Andrius;3
Inžinerinė grafika;Čepaitė;Sandra;4
```

Rezultatai

Moduliai

Įkelti duomenis

Modulio programavimas studentai, kuriems dėsto Lukas Naujokaitis :			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimas	Balčiūnas	Tomas	IF-21
Programavimas	Jurgelevičius	Dainius	IF-21
Programavimas	Kazlauskas	Andrius	IF-21
Programavimas	Petraitis	Jonas	IF-21
Programavimas	Adamkevičius	Lukas	IF-22
Programavimas	Jakubauskaitė	Laura	IF-22
Programavimas	Žukauskas	Marius	IF-22
Programavimas	Bartkevičius	Simonas	IF-23
Programavimas	Dulkytė	Eglė	IF-23
Programavimas	Vasiliauskaitė	Rasa	IF-23

Įveskite dėstytojo vardą ir pavardę: Lukas Naujokaitis

Atrinkti

Pav.46 Rezultatai puslapyje

Dėstytojų darbo apkrovos		
Pavardė	Vardas	Apkrova
Stankevičius	Paulius	12
Čepaitė	Sandra	36
Mačiulis	Dainius	8
Jonaitis	Antanas	12
Navickienė	Jolanta	10
Butkus	Valdas	8
Naujokaitis	Lukas	60
Daugirdas	Justas	6
Vitkutė	Monika	10
Starkus	Andrius	3

Profesoriai			
Modulio pavadinimas	Pavardė	Vardas	Kreditų kiekis
Programavimo pagrindai	Stankevičius	Paulius	6
Duomenų bazės	Čepaitė	Sandra	5
Matematika	Mačiulis	Dainius	4
Ekonomika	Jonaitis	Antanas	6
Marketingas	Navickienė	Jolanta	5
Verslo teisė	Butkus	Valdas	4
Programavimas	Naujokaitis	Lukas	6
Statybos pagrindai	Daugirdas	Justas	6
Konstrukcijų analizė	Vitkutė	Monika	5
Statybos teisė	Starkus	Andrius	3
Inžinerinė grafika	Čepaitė	Sandra	4

Pav.47 Rezultatai puslapyje

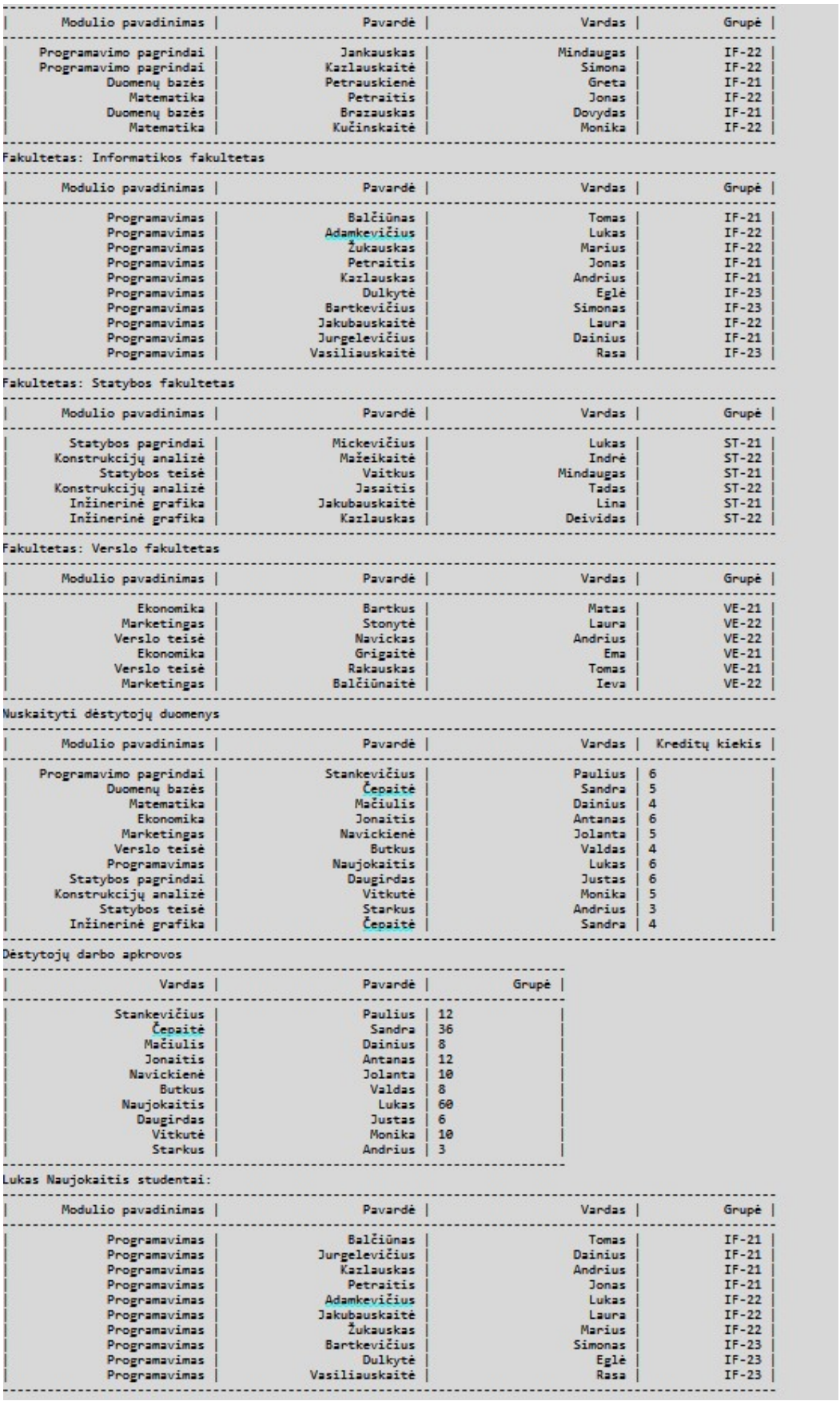
Fakultetas: Informatikos fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimo pagrindai	Jankauskas	Mindaugas	IF-22
Programavimo pagrindai	Kazlauskaitė	Simona	IF-22
Duomenų bazės	Petrauskienė	Greta	IF-21
Matematika	Petraitis	Jonas	IF-22
Duomenų bazės	Brazauskas	Dovydas	IF-21
Matematika	Kučinskaitė	Monika	IF-22

Fakultetas: Informatikos fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimas	Balčiūnas	Tomas	IF-21
Programavimas	Adamkevičius	Lukas	IF-22
Programavimas	Žukauskas	Marius	IF-22
Programavimas	Petraitis	Jonas	IF-21
Programavimas	Kazlauskas	Andrius	IF-21
Programavimas	Dulkytė	Eglė	IF-23
Programavimas	Bartkevičius	Simonas	IF-23
Programavimas	Jakubauskaitė	Laura	IF-22
Programavimas	Jurgelevičius	Dainius	IF-21
Programavimas	Vasiliauskaitė	Rasa	IF-23

Fakultetas: Statybos fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Statybos pagrindai	Mickevičius	Lukas	ST-21
Konstrukcijų analizė	Mažeikaitė	Indrė	ST-22
Statybos teisė	Vaitkus	Mindaugas	ST-21
Konstrukcijų analizė	Jasaitis	Tadas	ST-22
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Fakultetas: Verslo fakultetas			
Modulio pavadinimas	Pavardė	Vardas	Grupė
Ekonomika	Bartkus	Matas	VE-21
Marketingas	Stonytė	Laura	VE-22
Verslo teisė	Navickas	Andrius	VE-22
Ekonomika	Grigaitė	Ema	VE-21
Verslo teisė	Rakauskas	Tomas	VE-21
Marketingas	Balčiūnaitė	Ieva	VE-22

Pav.48 Rezultatai puslapyje



Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimo pagrindai	Jankauskas	Mindaugas	IF-22
Programavimo pagrindai	Kazlauskaitė	Simona	IF-22
Duomenų bazės	Petrauskienė	Greta	IF-21
Matematika	Petraitis	Jonas	IF-22
Duomenų bazės	Brazauskas	Dovydas	IF-21
Matematika	Kučinskaitė	Monika	IF-22

Fakultetas: Informatikos fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimas	Balčiūnas	Tomas	IF-21
Programavimas	Adamkevičius	Lukas	IF-22
Programavimas	Žukauskas	Marius	IF-22
Programavimas	Petraitis	Jonas	IF-21
Programavimas	Kazlauskas	Andrius	IF-21
Programavimas	Dulkytė	Eglė	IF-23
Programavimas	Bartkevičius	Simonas	IF-23
Programavimas	Jakubauskaitė	Laura	IF-22
Programavimas	Jurgelevičius	Dainius	IF-21
Programavimas	Vasiliauskaitė	Rasa	IF-23

Fakultetas: Statybos fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Statybos pagrindai	Mickevičius	Lukas	ST-21
Konstrukcijų analizė	Mažeikaitė	Indrė	ST-22
Statybos teisė	Vaitkus	Mindaugas	ST-21
Konstrukcijų analizė	Jasaitis	Tadas	ST-22
Inžinerinė grafika	Jakubauskaitė	Lina	ST-21
Inžinerinė grafika	Kazlauskas	Deividas	ST-22

Fakultetas: Verslo fakultetas

Modulio pavadinimas	Pavardė	Vardas	Grupė
Ekonomika	Bartkus	Matas	VE-21
Marketingas	Stonytė	Laura	VE-22
Verslo teisė	Navickas	Andrius	VE-22
Ekonomika	Grigaitė	Ema	VE-21
Verslo teisė	Rakauskas	Tomas	VE-21
Marketingas	Balčiūnaitė	Ieva	VE-22

Nuskaityti dėstytojų duomenys

Modulio pavadinimas	Pavardė	Vardas	Kreditų kiekis
Programavimo pagrindai	Stankevičius	Paulius	6
Duomenų bazės	Čepaitė	Sandra	5
Matematika	Mačiulis	Dainius	4
Ekonomika	Jonaitis	Antanas	6
Marketingas	Navickienė	Jolanta	5
Verslo teisė	Butkus	Valdas	4
Programavimas	Naujokaitis	Lukas	6
Statybos pagrindai	Daugirdas	Justas	6
Konstrukcijų analizė	Vitkutė	Monika	5
Statybos teisė	Starkus	Andrius	3
Inžinerinė grafika	Čepaitė	Sandra	4

Dėstytojų darbo apkrovos

Vardas	Pavardė	Grupė
Stankevičius	Paulius	12
Čepaitė	Sandra	36
Mačiulis	Dainius	8
Jonaitis	Antanas	12
Navickienė	Jolanta	10
Butkus	Valdas	8
Naujokaitis	Lukas	60
Daugirdas	Justas	6
Vitkutė	Monika	10
Starkus	Andrius	3

Lukas Naujokaitis studentai:

Modulio pavadinimas	Pavardė	Vardas	Grupė
Programavimas	Balčiūnas	Tomas	IF-21
Programavimas	Jurgelevičius	Dainius	IF-21
Programavimas	Kazlauskas	Andrius	IF-21
Programavimas	Petraitis	Jonas	IF-21
Programavimas	Adamkevičius	Lukas	IF-22
Programavimas	Jakubauskaitė	Laura	IF-22
Programavimas	Žukauskas	Marius	IF-22
Programavimas	Bartkevičius	Simonas	IF-23
Programavimas	Dulkytė	Eglė	IF-23
Programavimas	Vasiliauskaitė	Rasa	IF-23

Pav.49 Rezultatai „ExternalData.txt“

5.8. Dėstytojo pastabos